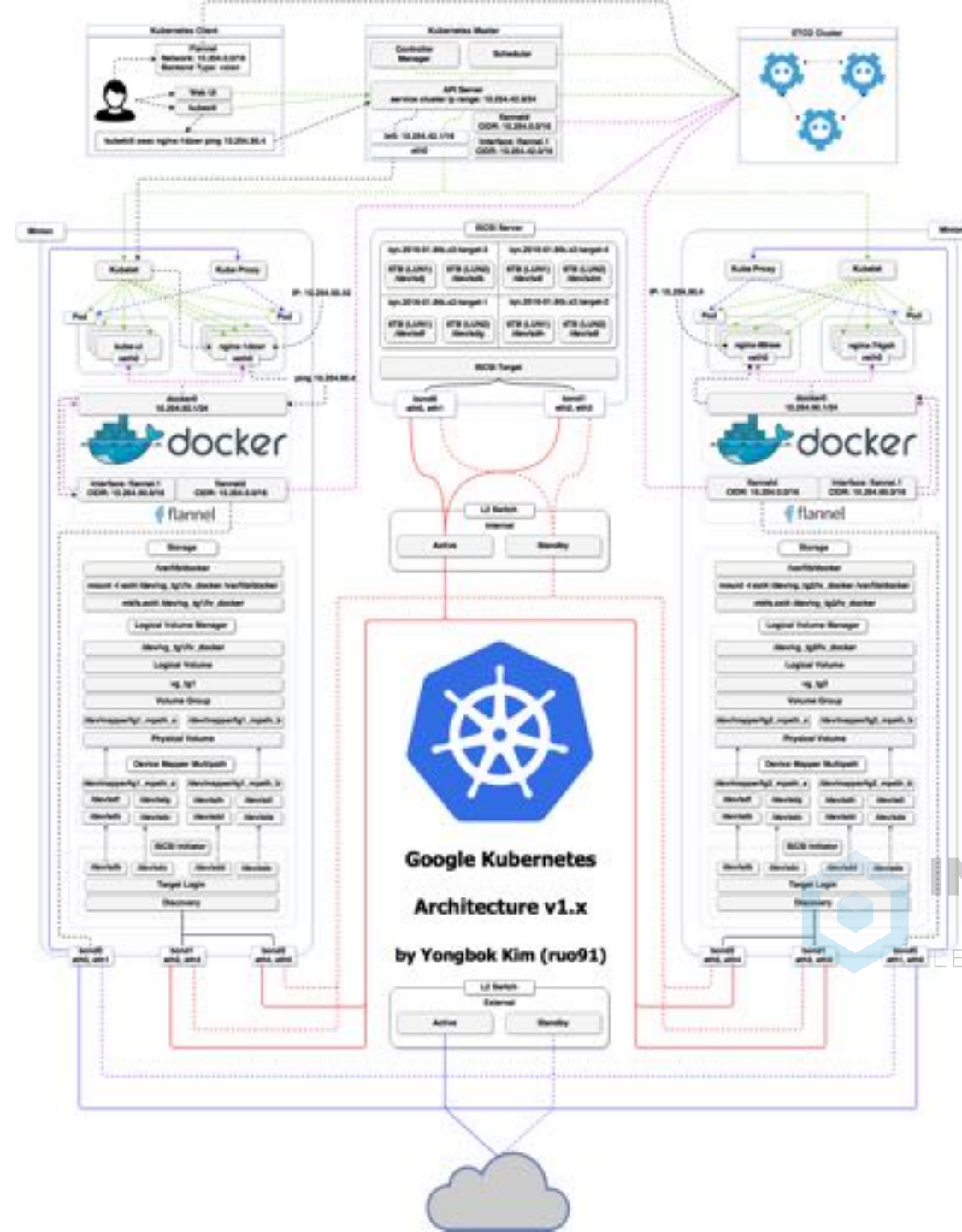


Kubernetes Architecture

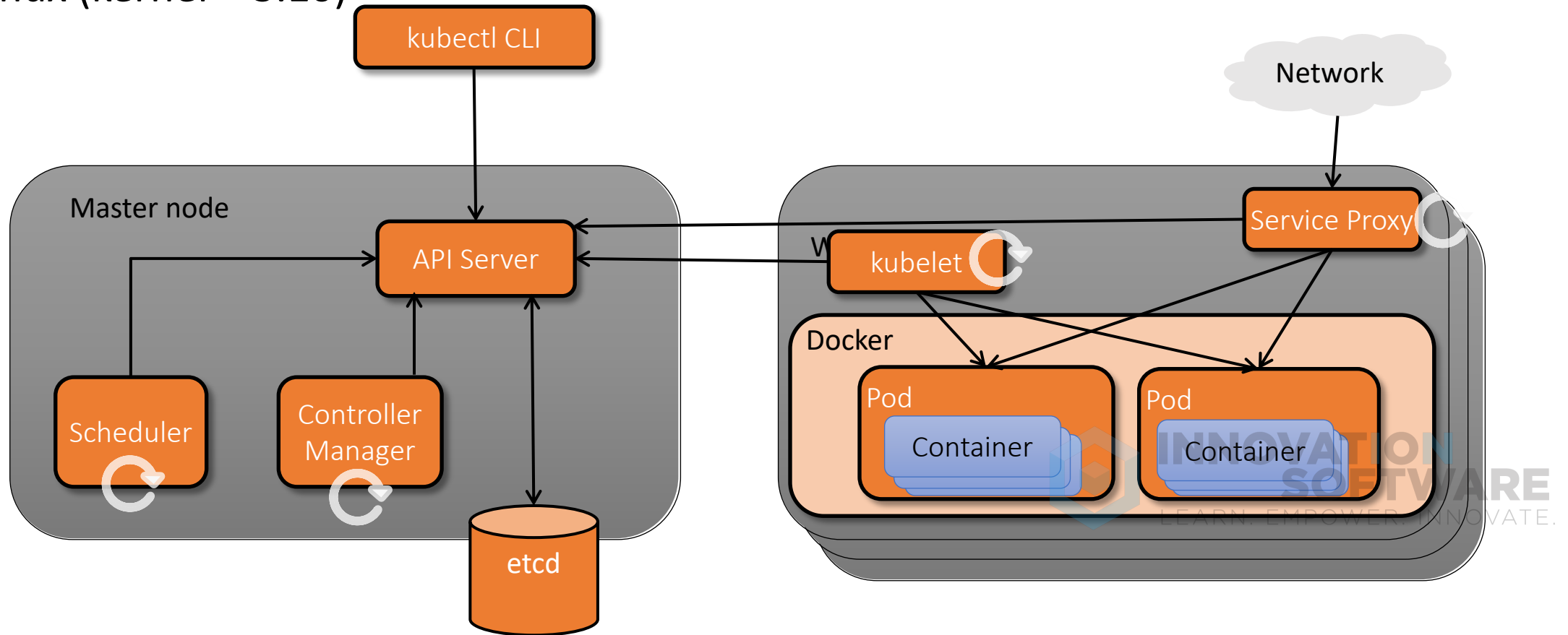


INNOVATION
STACK
LEARN. EMPLOY. INNOVATE.



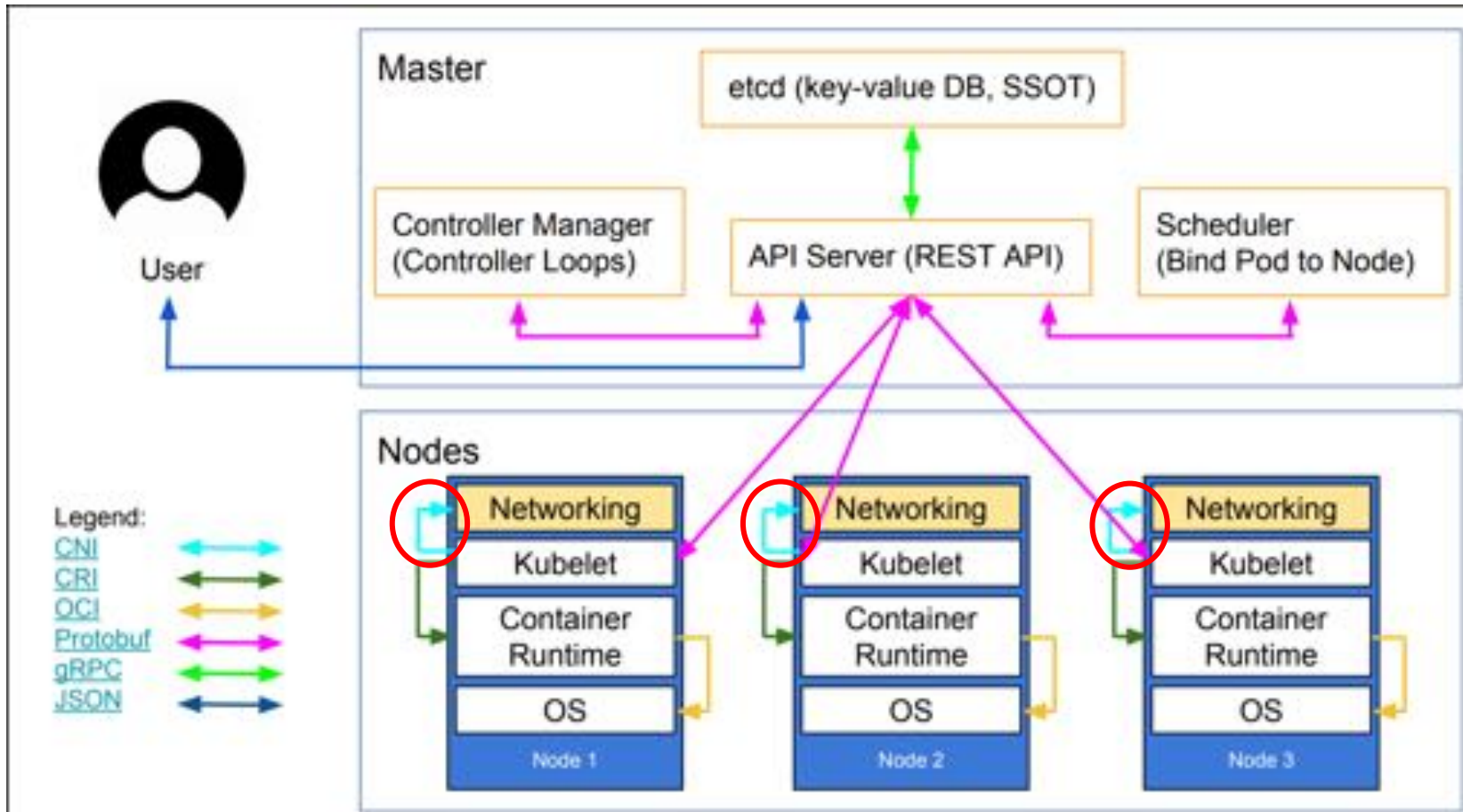
Kubernetes Cluster Architecture

- Kubernetes nodes can be physical hosts or VM's running a container-friendly Linux (kernel > 3.10)



Components Communication

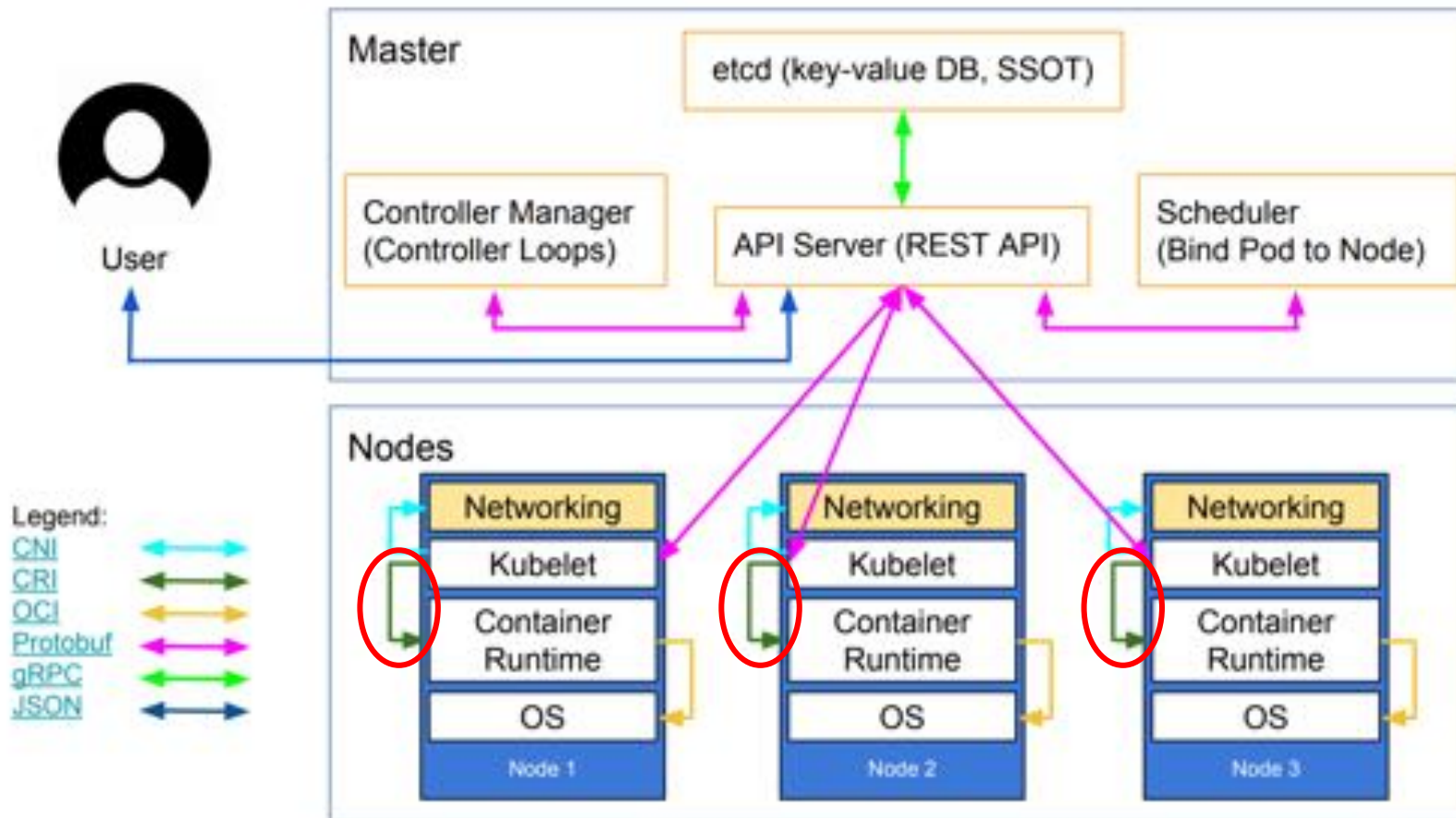
- CNI = Container Network Interface



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Components Communication

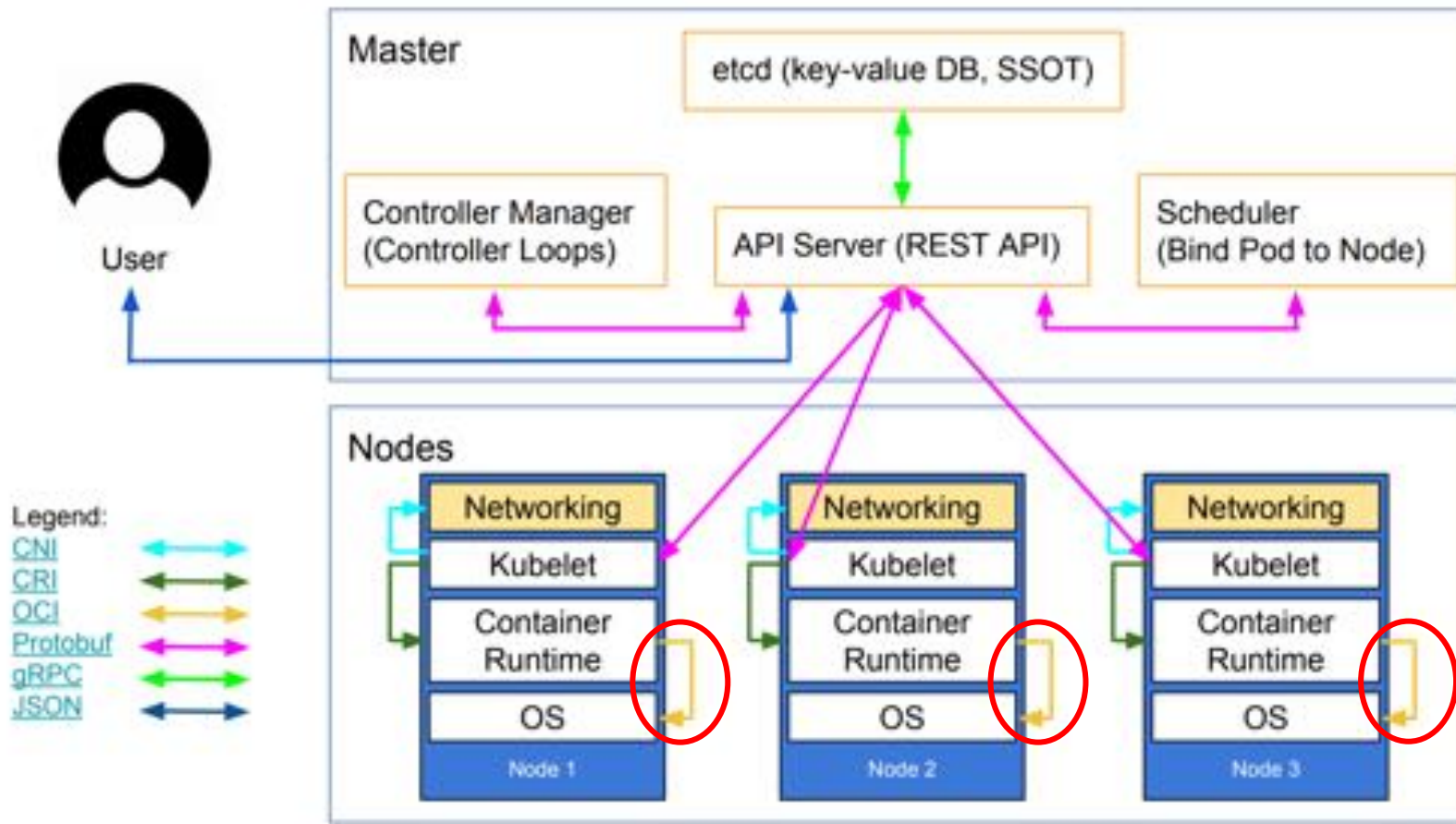
- CNI = Container Network Interface
- CRI = Container Runtime Interface
 - Docker, CRI-O, Containerd



INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.

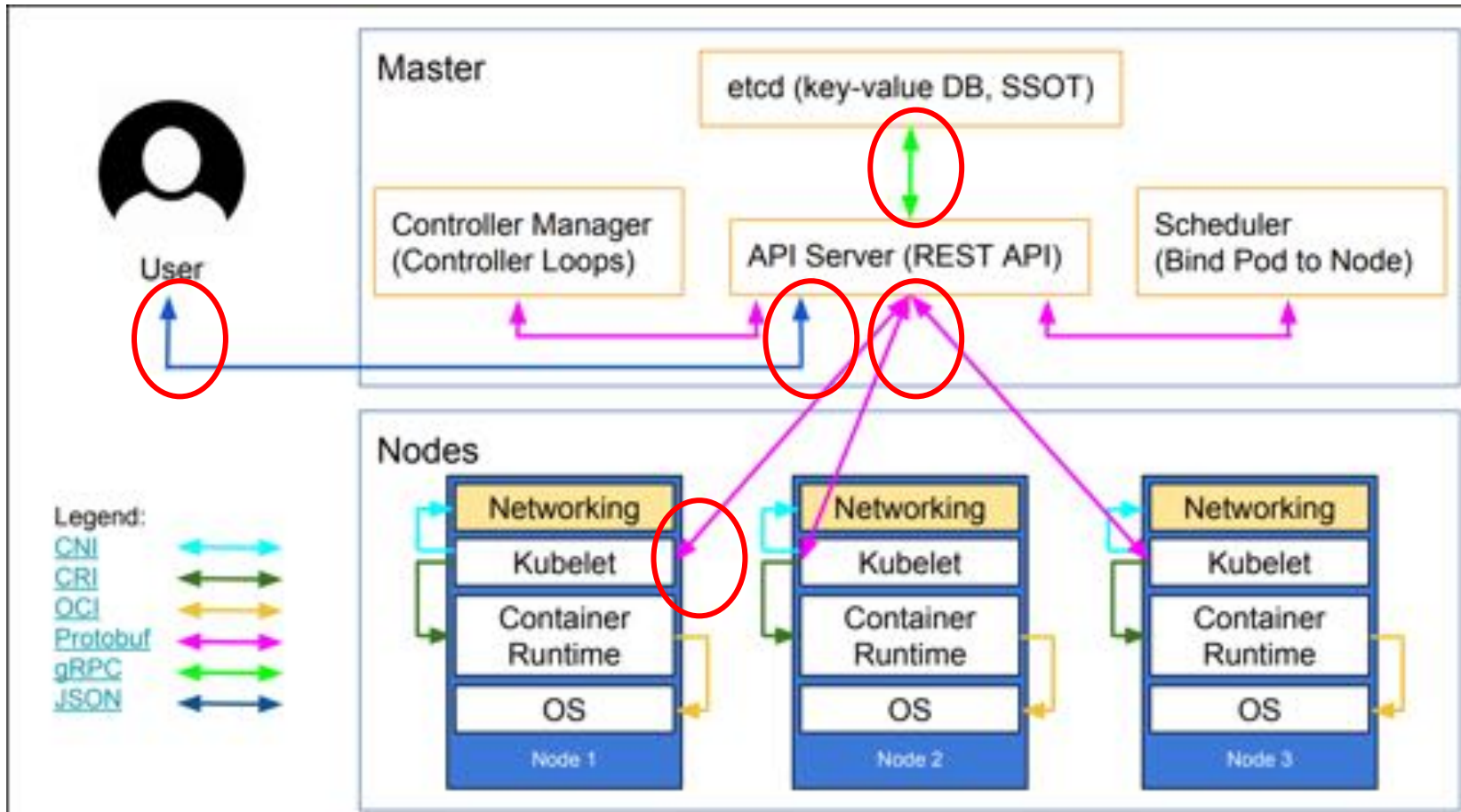
Components Communication

- CNI = Container Network Interface
- CRI = Container Runtime Interface
 - Docker, CRI-O, Containerd
- OCI = Open Container Initiative



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Components Communication

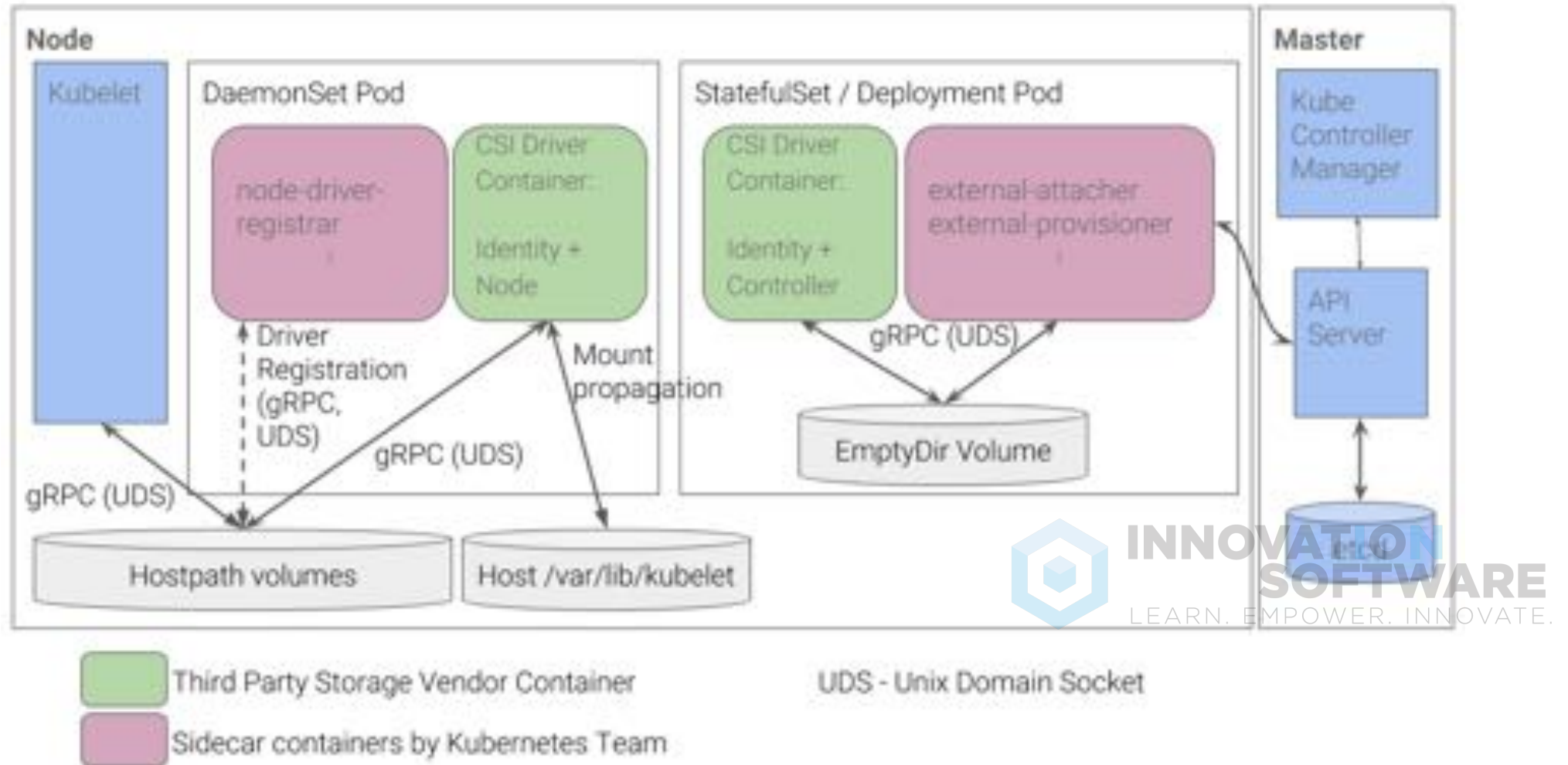


- CNI = Container Network Interface
- CRI = Container Runtime Interface
 - Docker, CRI-O, Containerd
- OCI = Open Container Initiative
- Protobuf
- gRPC
- JSON

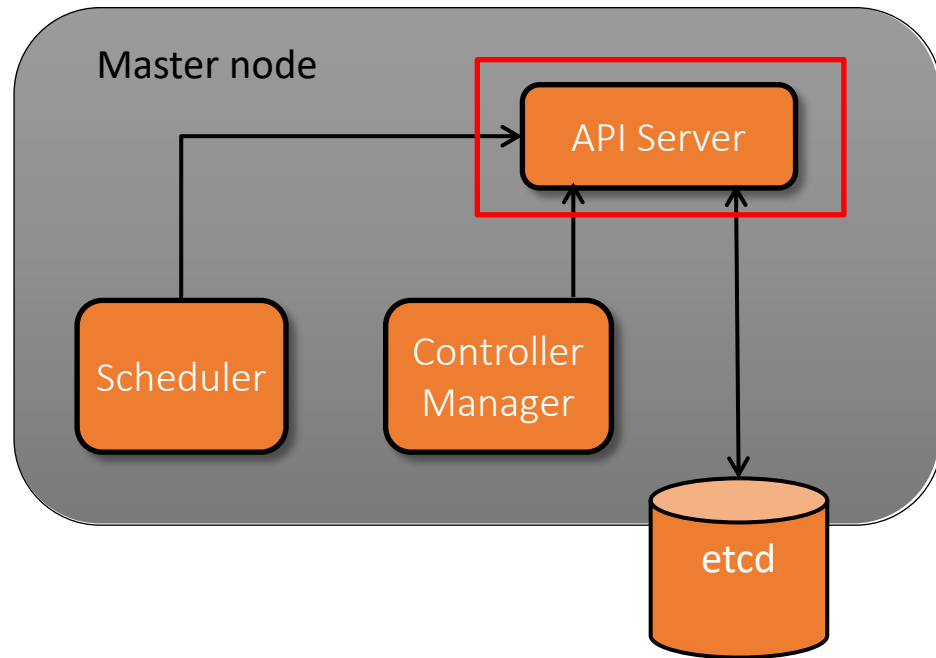


INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.

Container Storage Interface



Kubernetes Master Node Components



- **API Server (kube-apiserver):** exposes the Kubernetes REST API, and can be scaled horizontally
- **Scheduler (kube-scheduler):** selects nodes for newly created pods to run on
- **Controller manager (kube-controller-manager):** runs background controller processes for the system to enforce declared object states, e.g. Node Controller, Replication Controller, ...
- **Persistent data store (etcd):** all K8s system data is stored in a distributed, reliable key-value store. etcd may run on separate nodes from the master

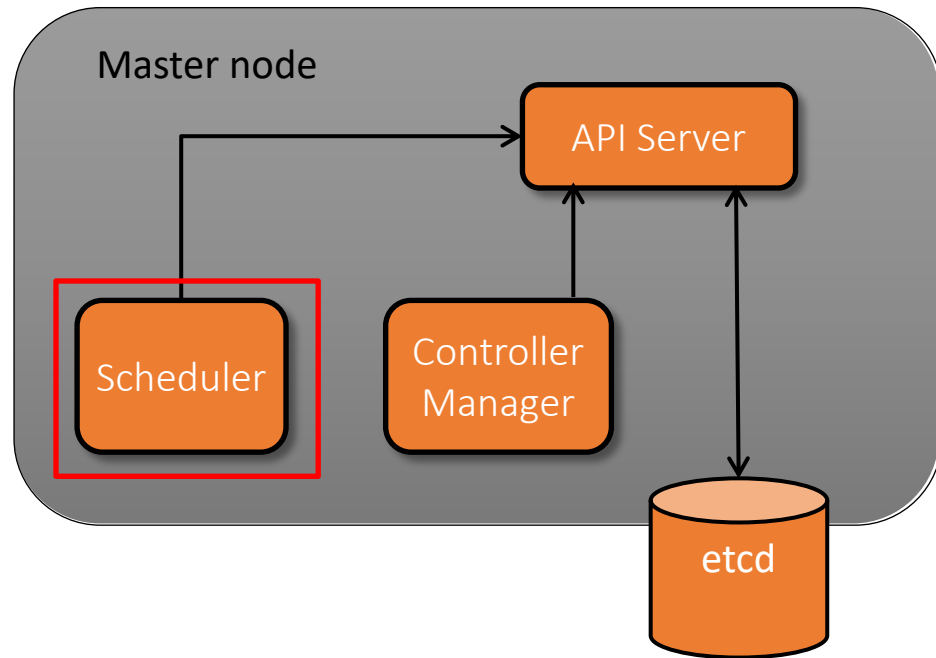


K8s components
written in Go
(golang.org)



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Kubernetes Master Node Components



- **API Server (kube-apiserver):** exposes the Kubernetes REST API, and can be scaled horizontally
- **Scheduler (kube-scheduler):** selects nodes for newly created pods to run on
- **Controller manager (kube-controller-manager):** runs background controller processes for the system to enforce declared object states, e.g. Node Controller, Replication Controller, ...
- **Persistent data store (etcd):** all K8s system data is stored in a distributed, reliable key-value store. etcd may run on separate nodes from the master

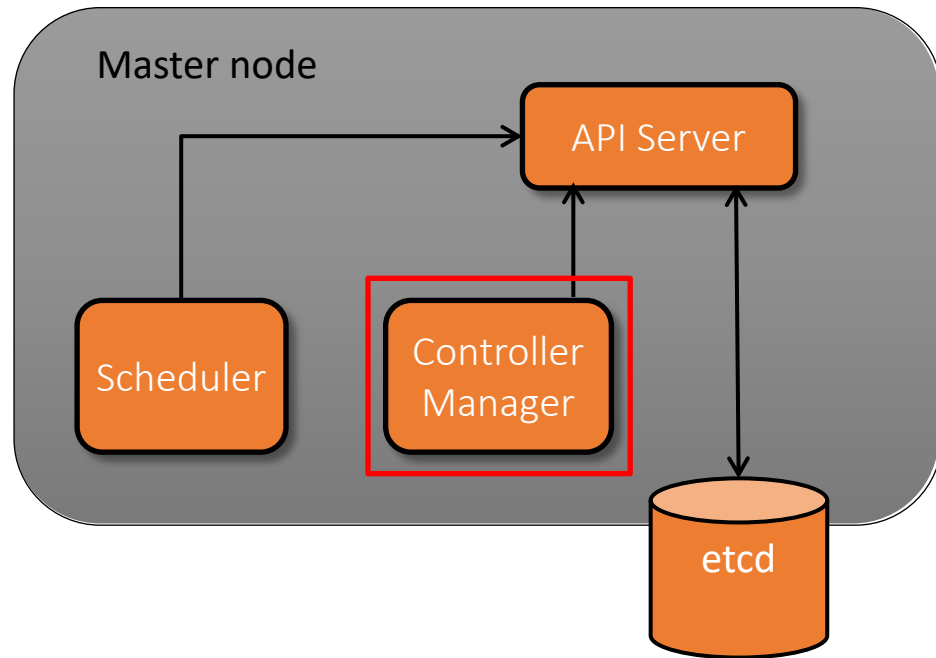


K8s components
written in Go
(golang.org)



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Kubernetes Master Node Components



- **API Server (kube-apiserver):** exposes the Kubernetes REST API, and can be scaled horizontally
- **Scheduler (kube-scheduler):** selects nodes for newly created pods to run on
- **Controller manager (kube-controller-manager):** runs background controller processes for the system to enforce declared object states, e.g. Node Controller, Replication Controller, ...
- **Persistent data store (etcd):** all K8s system data is stored in a distributed, reliable key-value store. etcd may run on separate nodes from the master

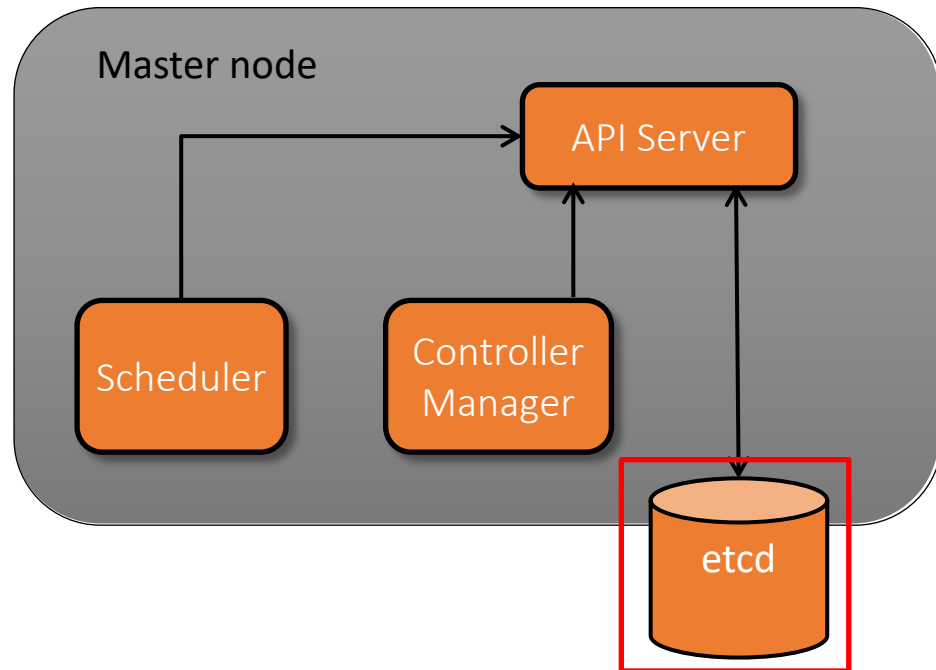


K8s components
written in Go
(golang.org)



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Kubernetes Master Node Components



- **API Server (kube-apiserver):** exposes the Kubernetes REST API, and can be scaled horizontally
- **Scheduler (kube-scheduler):** selects nodes for newly created pods to run on
- **Controller manager (kube-controller-manager):** runs background controller processes for the system to enforce declared object states, e.g. Node Controller, Replication Controller, ...
- **Persistent data store (etcd):** all K8s system data is stored in a distributed, reliable key-value store. etcd may run on separate nodes from the master

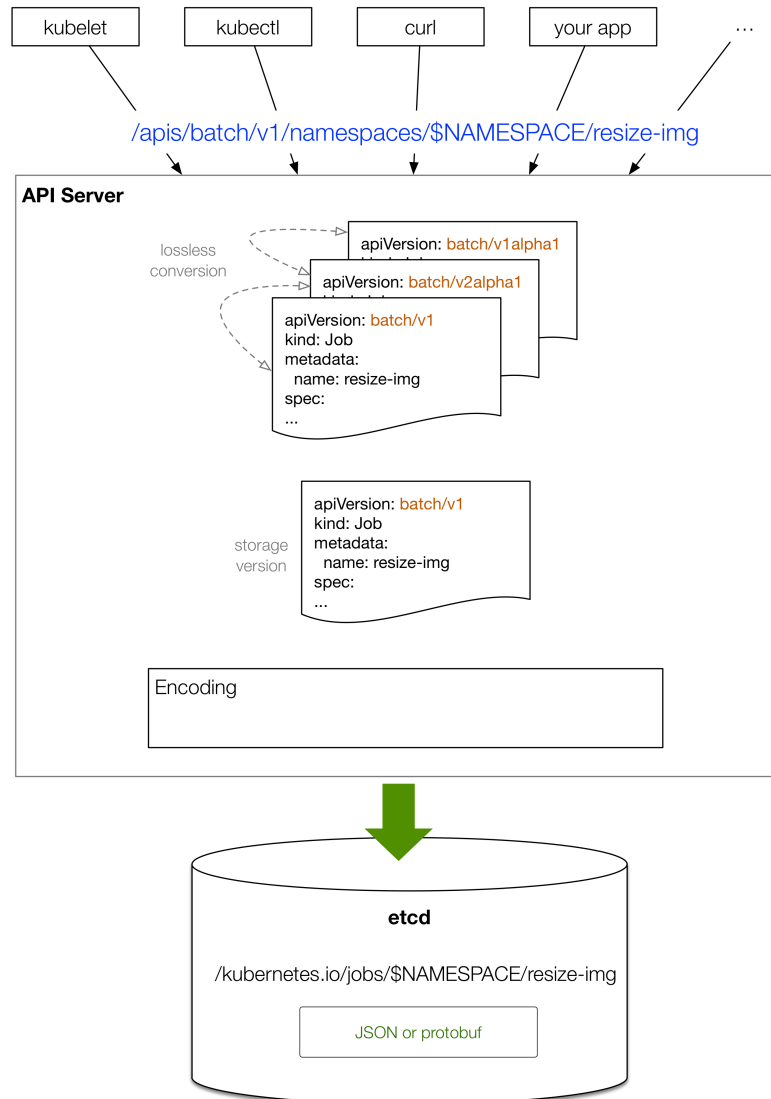


K8s components
written in Go
(golang.org)



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

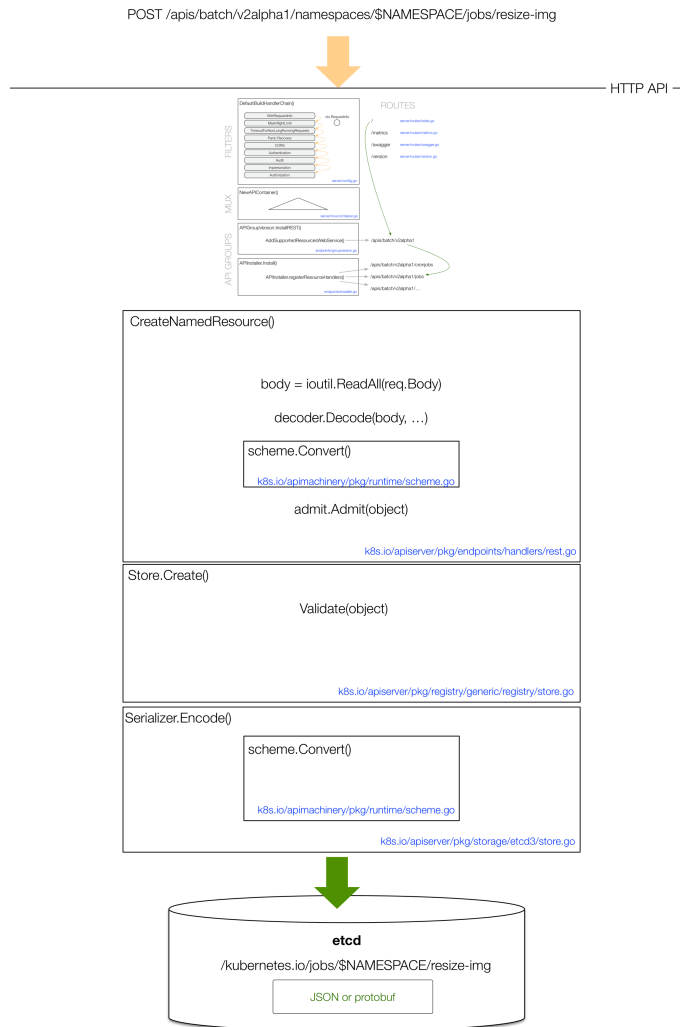
Etcd flow



- **Etcd flow:**

- client provides desired object state in YAML or JSON
- kubectl converts YAML to JSON and sends it to API
- API server turns input object state into canonical storage version
- storage process in etcd, at a certain key, into a value with the encoding to JSON or protobuf.

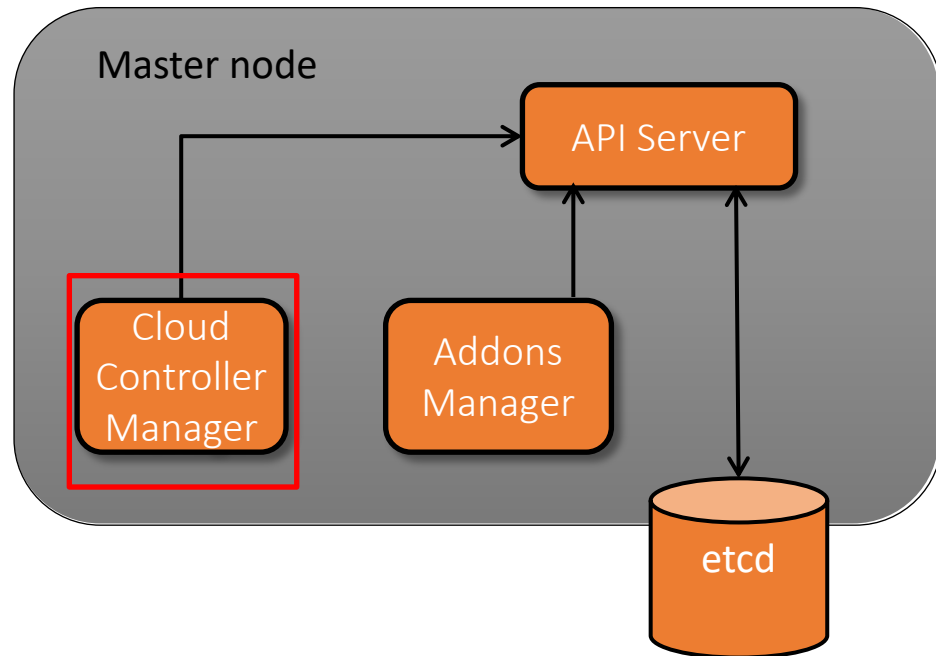
Serialization of State Flow



- **Serialization flow:**

- API Server keeps all known k8s object kinds in a Go registry (Scheme).
- Version of kinds defined along with how they can be:
 - converted
 - new objects created
 - objects encoded/decoded to JSON/protobuf

Master Node Additional Components



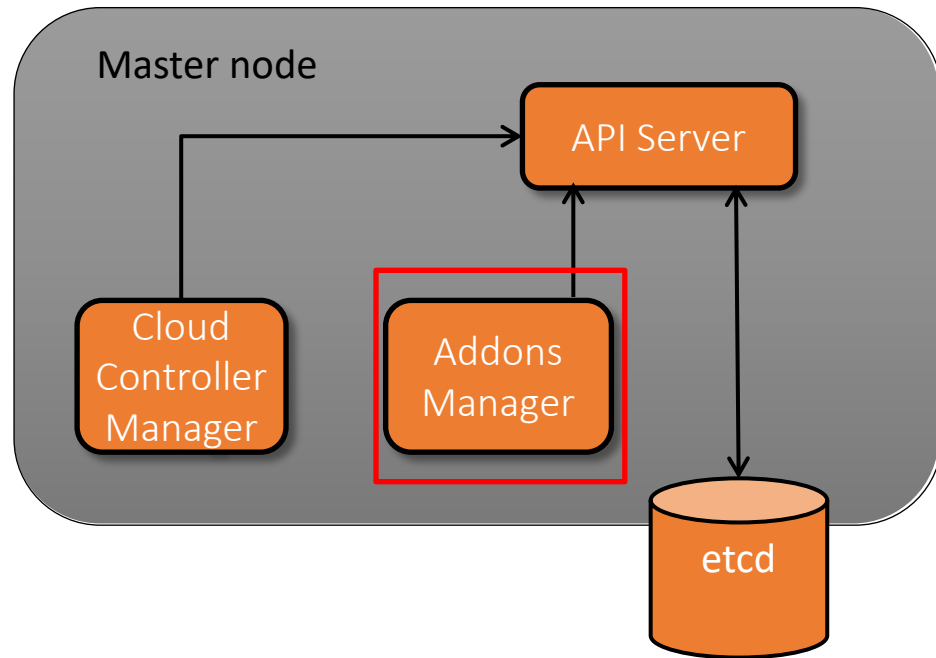
K8s components
written in Go
(golang.org)

- **cloud-controller-manager**: runs controllers interacting with underlying IaaS providers – alpha feature
Allows cloud vendor-specific code to be separate from main K8s system components
- **addons-manager**: creates and maintains cluster addon resources in 'kube-system' namespace, e.g.
 - **Kubernetes Dashboard**: general web UI for application and cluster management
 - **kube-dns**: serves DNS records for K8s services and resources
 - Container resource monitoring and cluster-level logging



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Master Node Additional Components



K8s components
written in Go
(golang.org)

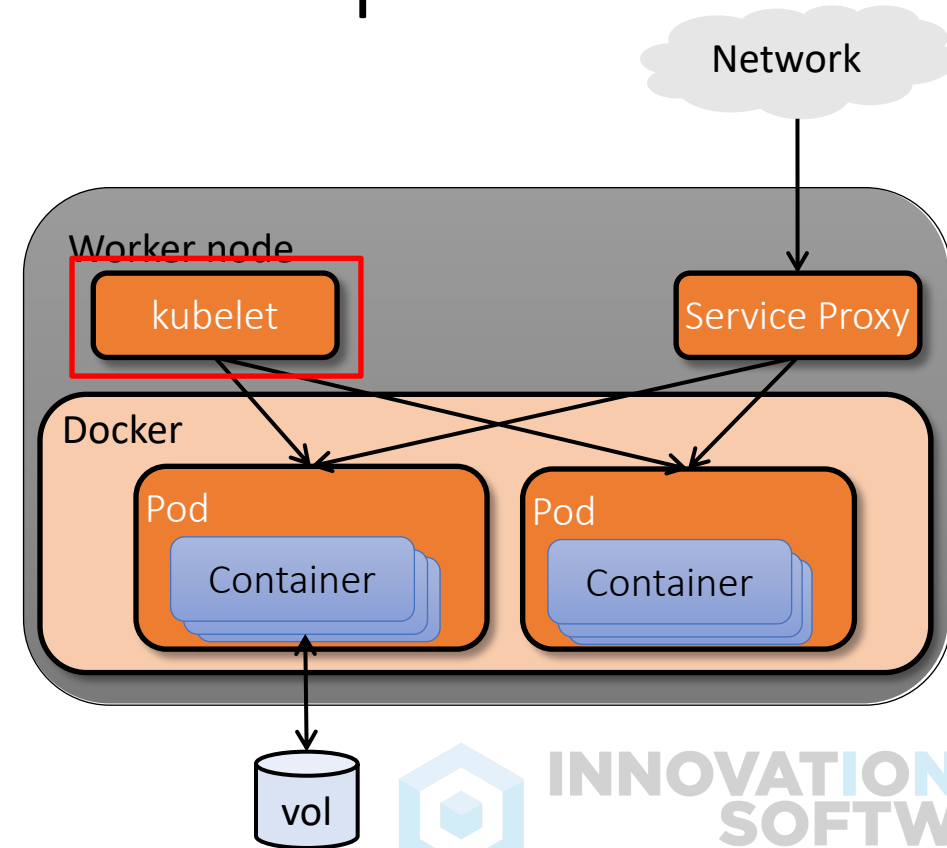
- **cloud-controller-manager**: runs controllers interacting with underlying IaaS providers – alpha feature
 - Allows cloud vendor-specific code to be separate from main K8s system components
- **addons-manager**: creates and maintains cluster add-on resources in 'kube-system' namespace, e.g.
 - **Kubernetes Dashboard**: general web UI for application and cluster management
 - **kube-dns**: serves DNS records for K8s services and resources
 - Container resource monitoring and cluster-level logging



INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.

Kubernetes Worker Node Components

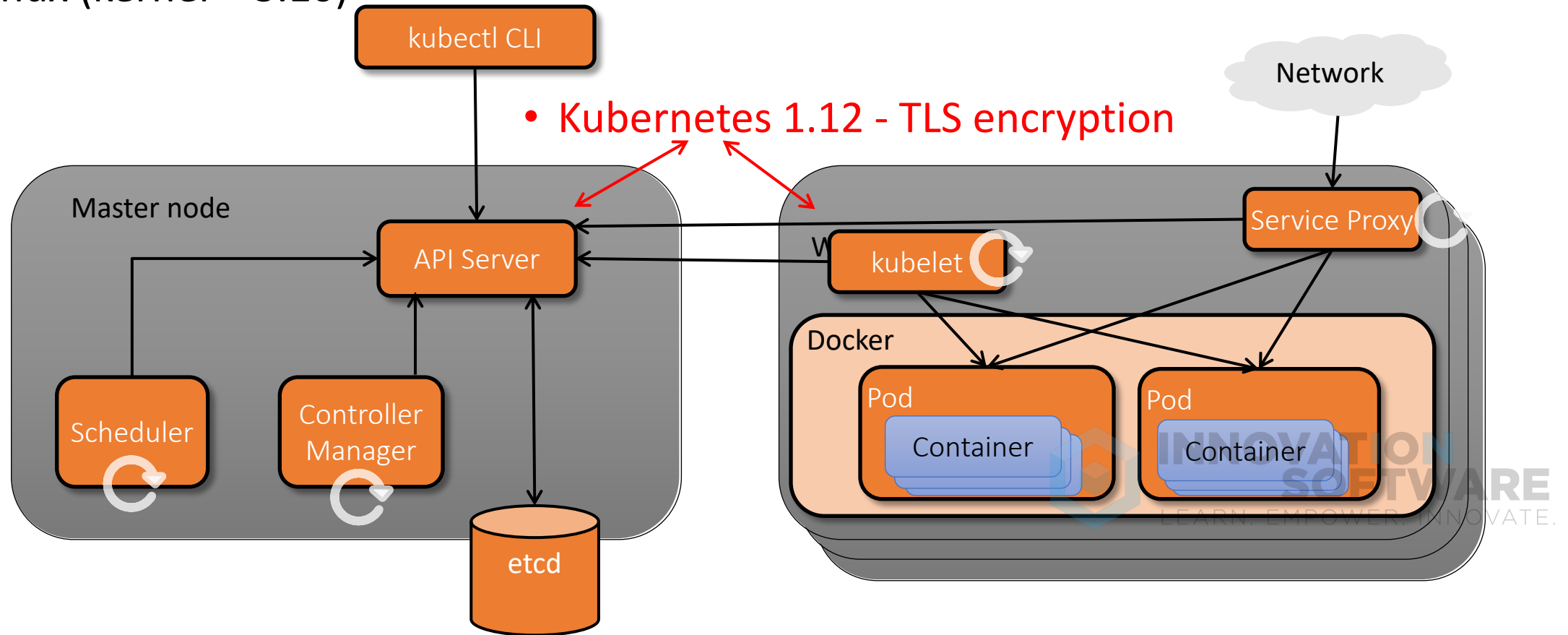
- **kubelet**: local K8s agent that is responsible for operations on the node, including
 - Watching for pod assignments
 - Mounting pod required volumes
 - Running a pod's containers
 - Executing container liveness probes
 - Reporting pod status to system
 - Reporting node status to system
- **Service proxy (kube-proxy)**: enables K8s service abstractions by maintaining host network rules and forwarding connections
- **Docker**: runs the containers



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

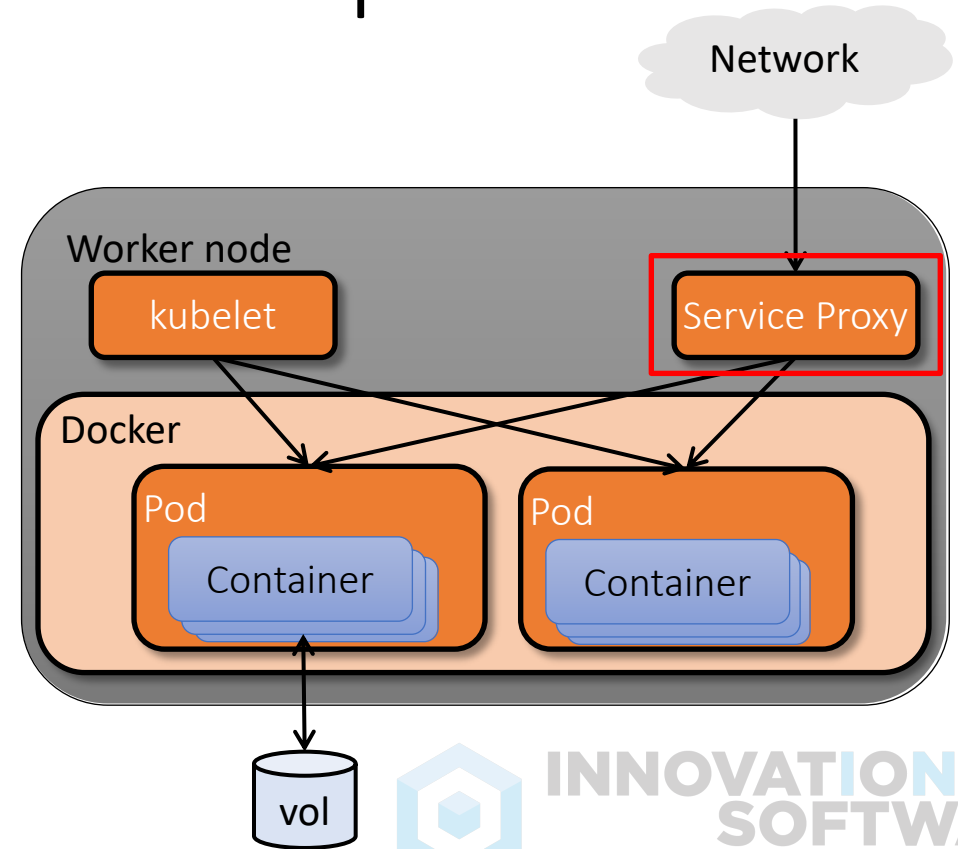
Kubernetes Cluster Architecture

- Kubernetes nodes can be physical hosts or VM's running a container-friendly Linux (kernel > 3.10)



Kubernetes Worker Node Components

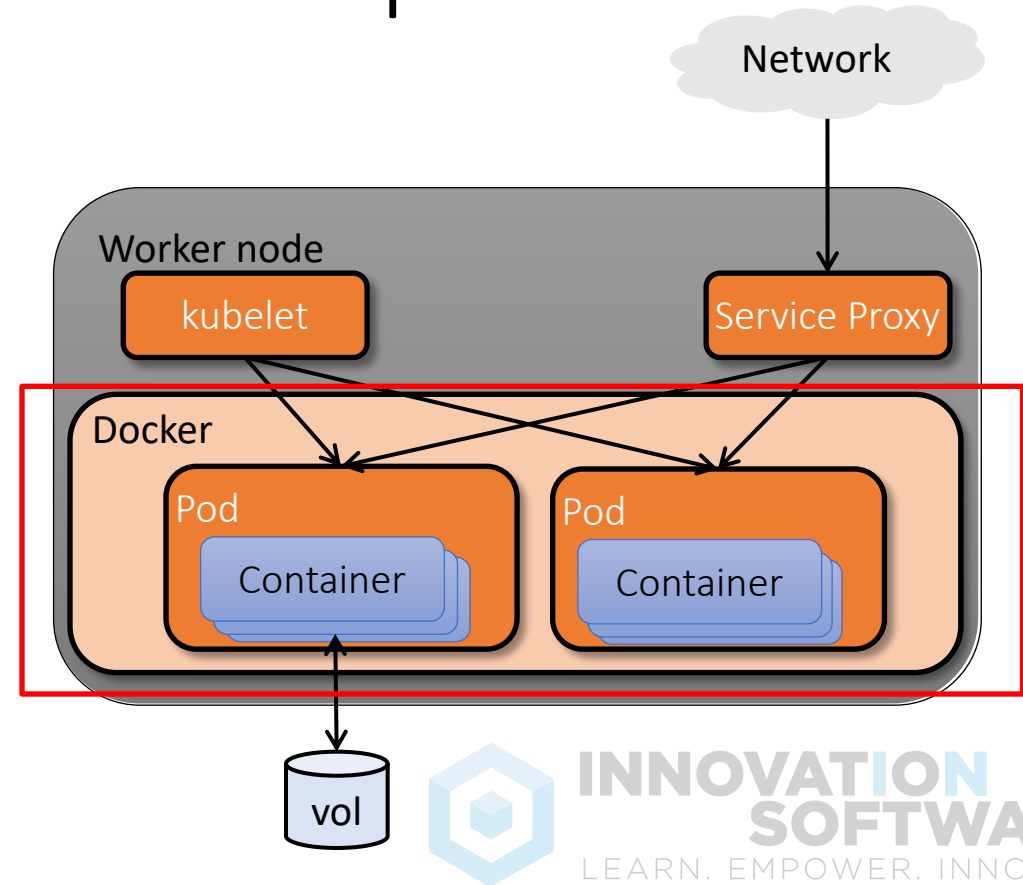
- **kubelet**: local K8s agent that is responsible for operations on the node, including
 - Watching for pod assignments
 - Mounting pod required volumes
 - Running a pod's containers
 - Executing container liveness probes
 - Reporting pod status to system
 - Reporting node status to system
- **Service proxy (kube-proxy)**: enables K8s service abstractions by maintaining host network rules and forwarding connections
- **Docker**: runs the containers



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

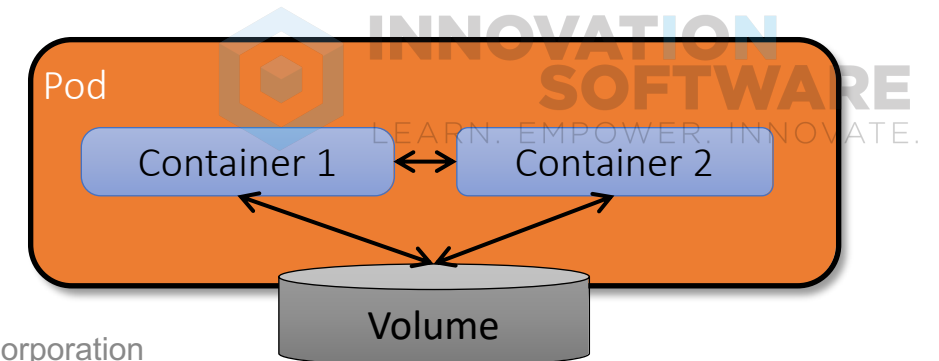
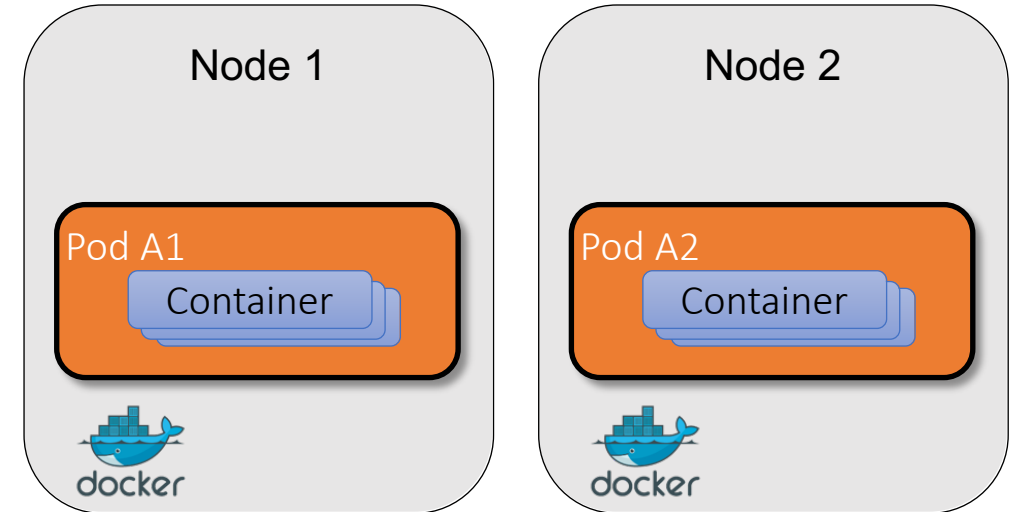
Kubernetes Worker Node Components

- **kubelet**: local K8s agent that is responsible for operations on the node, including
 - Watching for pod assignments
 - Mounting pod required volumes
 - Running a pod's containers
 - Executing container liveness probes
 - Reporting pod status to system
 - Reporting node status to system
- **Service proxy (kube-proxy)**: enables K8s service abstractions by maintaining host network rules and forwarding connections
- **Docker**: container runtime



Kubernetes Pods

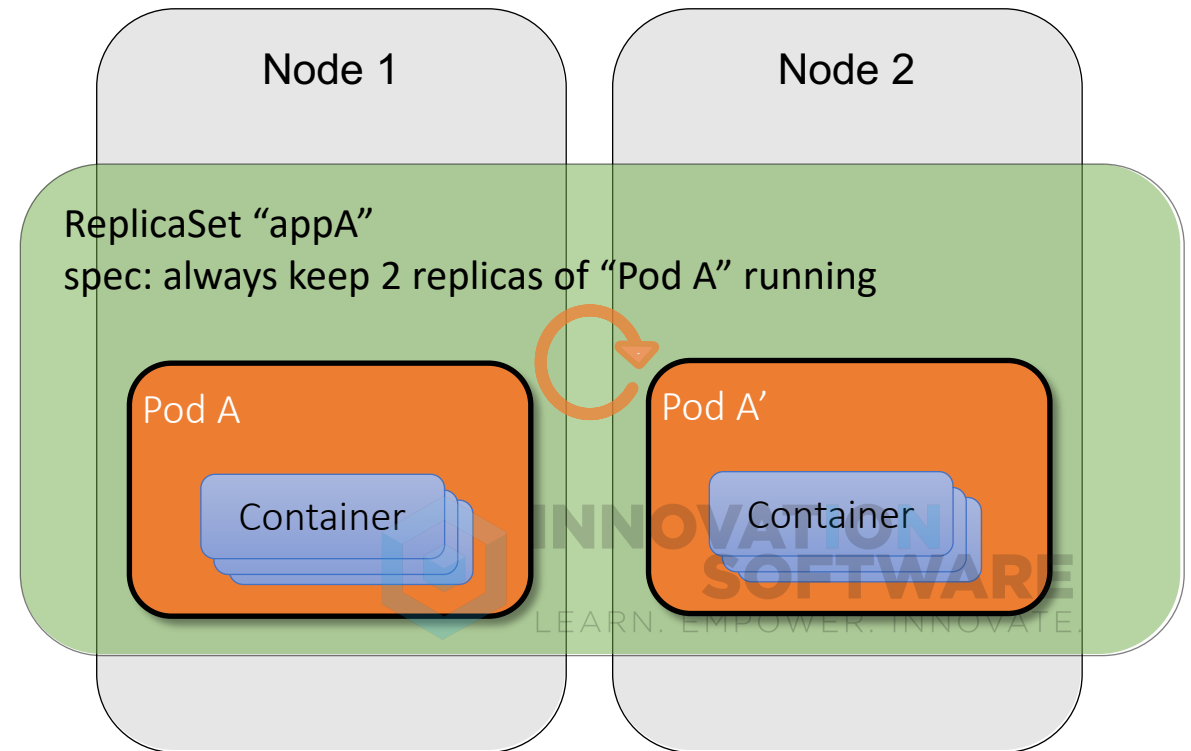
- Smallest K8s workload unit is the **Pod**, a set of co-scheduled containers
- A Pod == an application instance
- Pods can include more than one container, for tightly-coupled application components
- Containers in the same Pod share networking and storage resources
- Kubernetes handles efficient placement of Pods across available Nodes
- Pods and other K8s objects carry user-defined labels



Controllers for Different Application Patterns

- K8s controller objects used to create and manage Pods according to different application patterns => control loops
 - **ReplicaSets** manage sets of replicas of stateless workloads to ensure availability
 - **StatefulSets** manage stateful workloads on stable storage to ensure consistency
 - **DaemonSets** manage workloads that must run on every node, or set of nodes
 - **Jobs** manage parallel batch processing workloads

Controller example: ReplicaSet



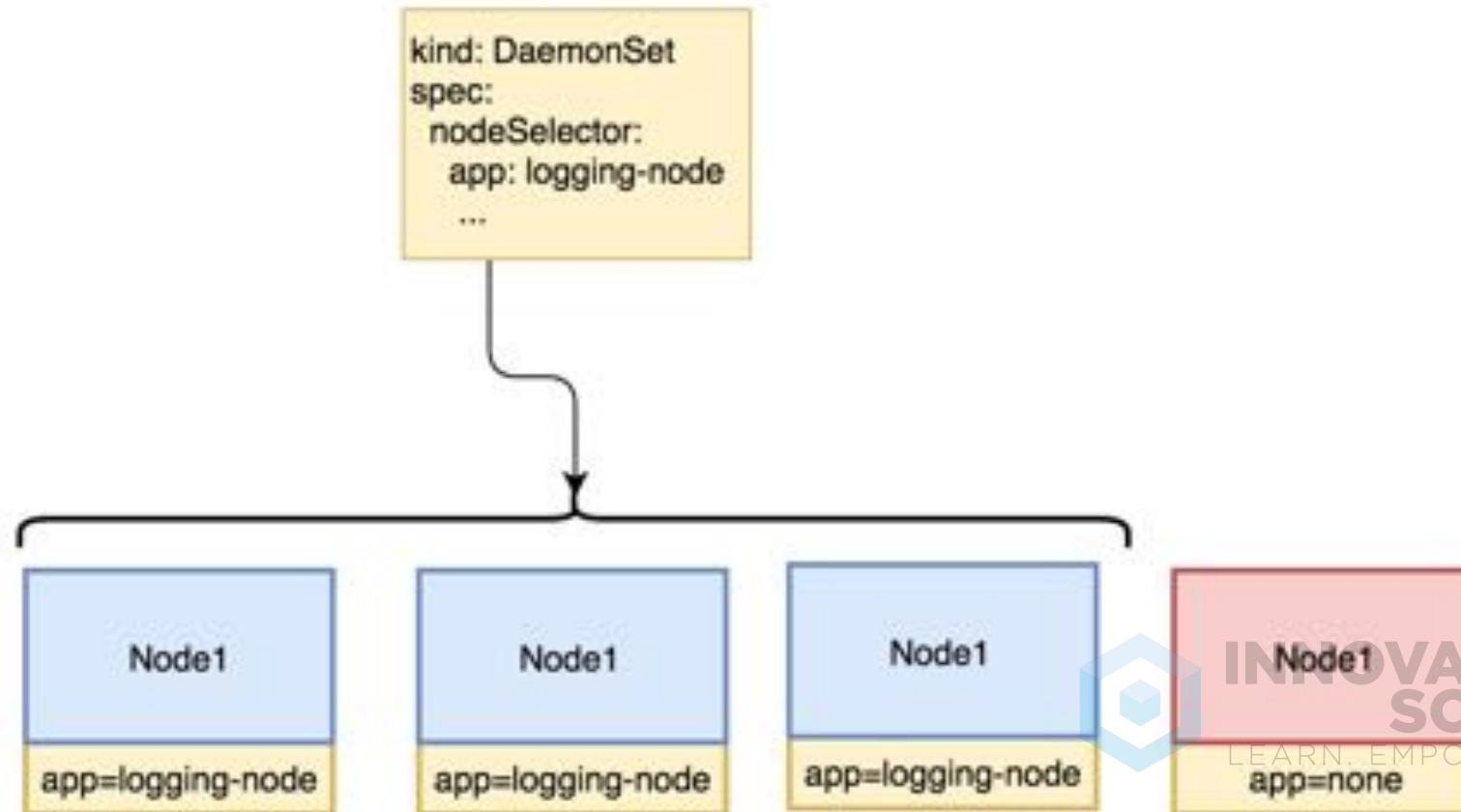
DaemonSets



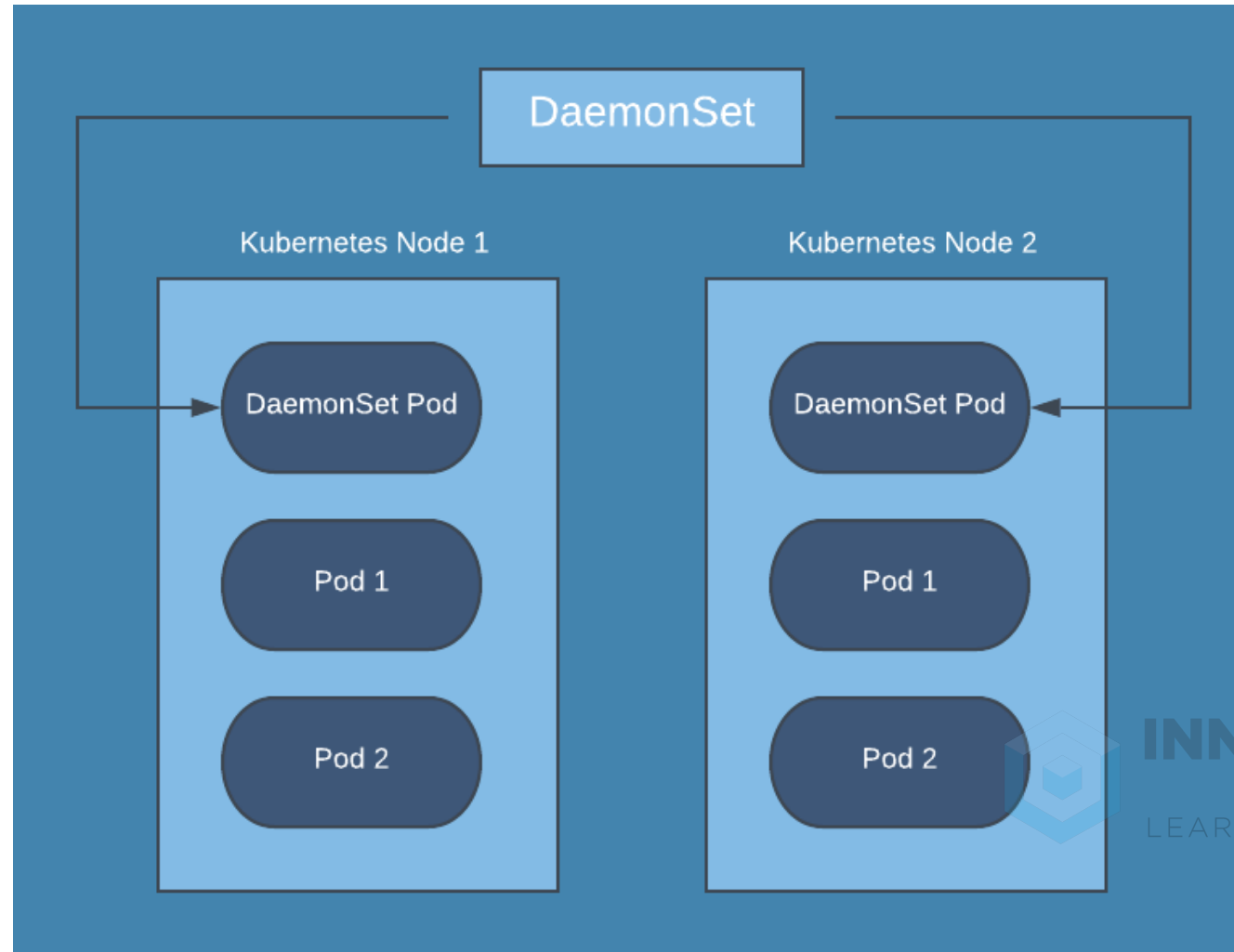
**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.



DaemonSets



DaemonSets



DaemonSets

Configure DaemonSet

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: logging
spec:
  selector:
    matchLabels:
      app: logging-app
  template:
    metadata:
      labels:
        app: logging-app
spec:
  containers:
    - name: webserver
      image: nginx
      ports:
        - containerPort: 80
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

DaemonSets

Configure DaemonSet

```
...
  template:
    metadata:
      labels:
        app: logging-app
  spec:
    nodeSelector:
      type: prod
    containers:
      - name: webserver
        image: nginx
        ports:
          - containerPort: 80
```

- **nodeSelector:** which node(s) the DaemonSet runs on.



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

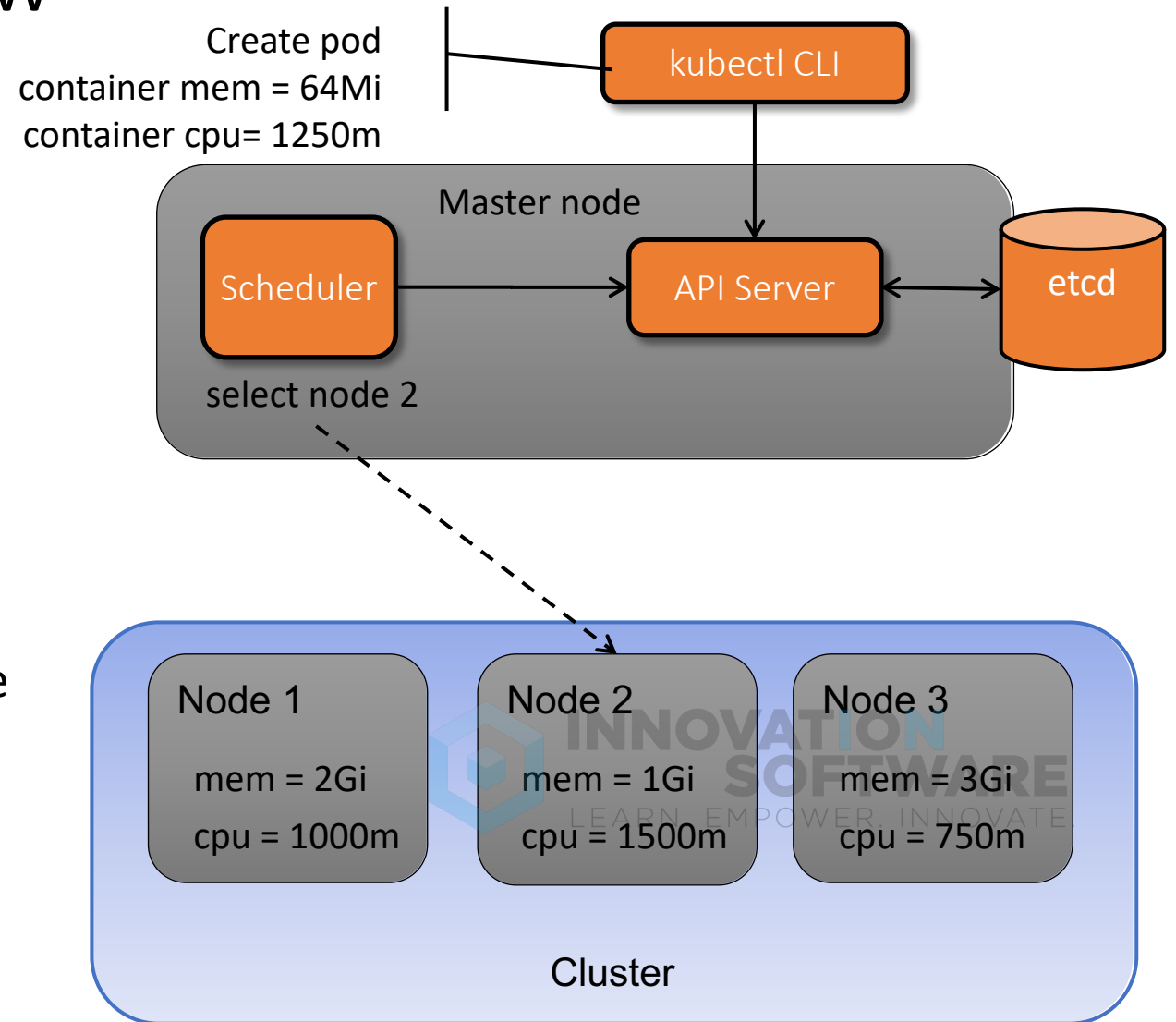
Kubernetes POD Scheduling



INNOVATION
XPERTS
LEARN. EMPOWER. SCALE.

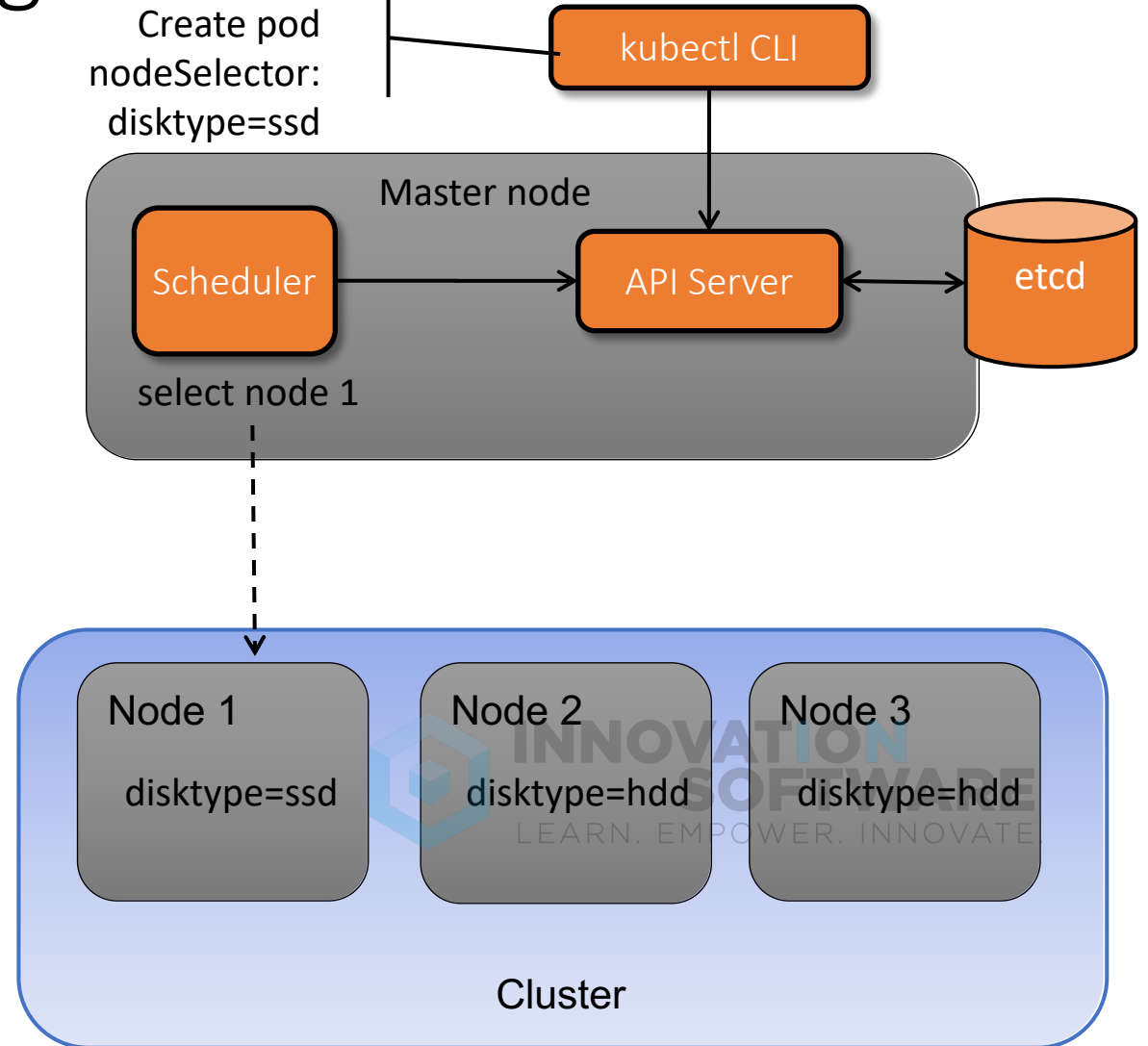
Pod Scheduling Overview

- kube-scheduler on the master node assigns new pod requests to appropriate worker nodes
- Default scheduler takes account of
 - Available node CPU/RAM
 - Resource requests from new pod – sum of resource requests of pod containers
- Default scheduler will automatically
 - Schedule pod on node with sufficient free resources
 - Spread pods across nodes in the cluster
- Can specify custom schedulers for pods



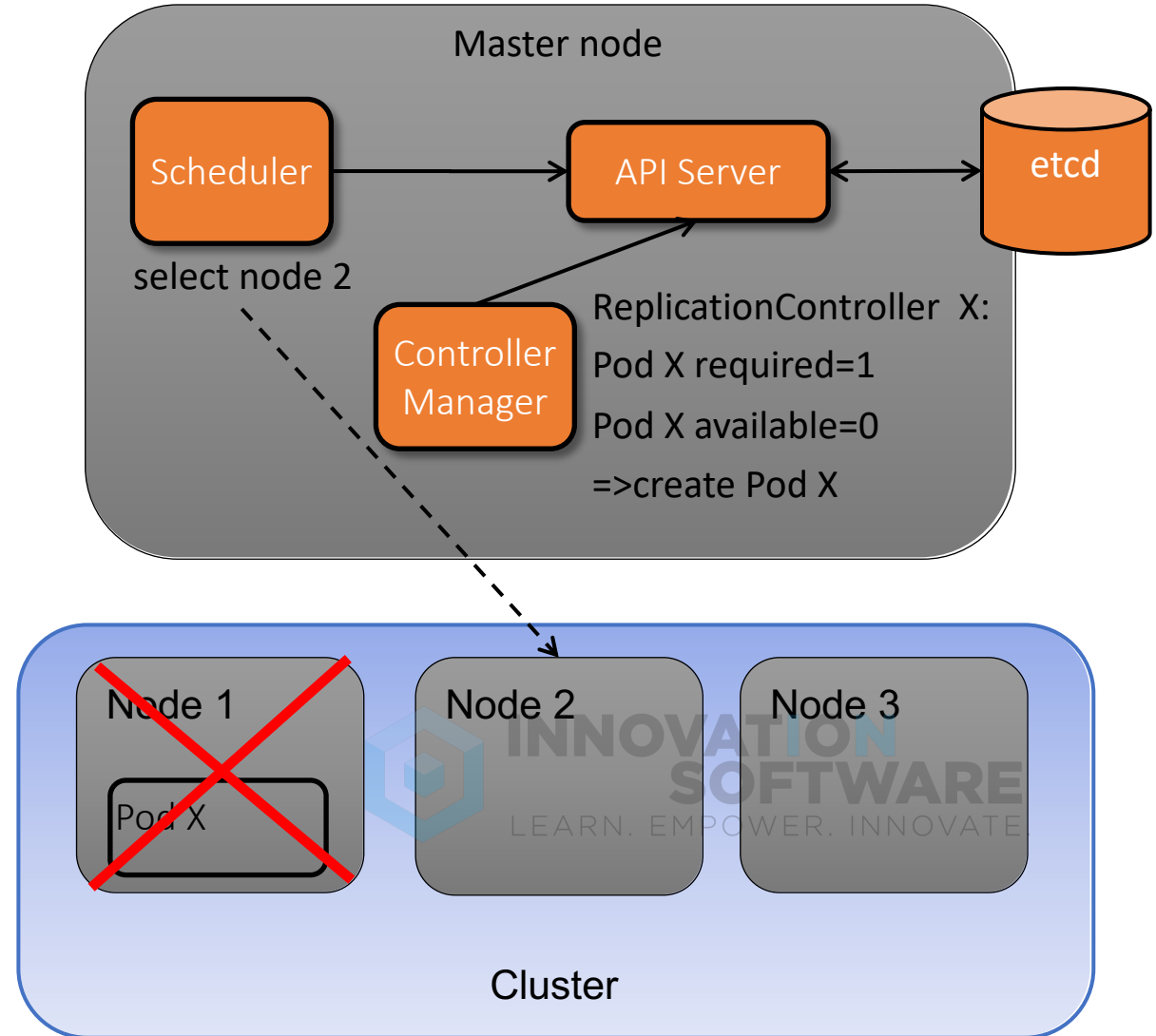
Controlling Pod Scheduling with Default Scheduler

- Nodes carry labels indicating topology and other resource notes
- Users can require pods to be scheduled on nodes with specific label(s) via a nodeSelector in container spec



Scheduling Pod Re-creation Driven by Control Loops

- kube-scheduler performs same node selection operation when new pod created due to e.g. node loss
- kube-controller-manager runs controllers like ReplicationController managing number of pod instances available
- kube-controller-manager will initiate request for new pods as needed, which will be scheduled by kube-scheduler per pod/container spec



Kubernetes Scheduling

Node Selector

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  container:
  - name: nginx
    image: nginx
    nodeSelector:
      disktype: ssd
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Scheduler preemption



- **Preemption**
 - Lower priority Pods are destroyed to free up resources for higher priority Pods.



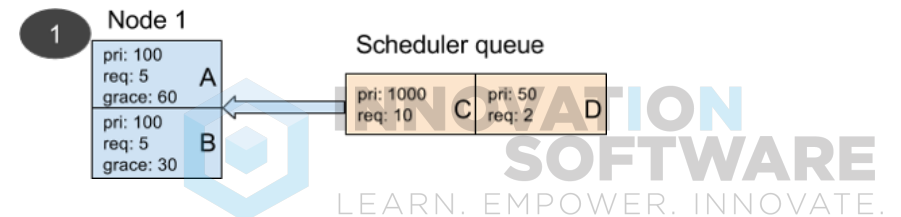
**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Scheduler preemption



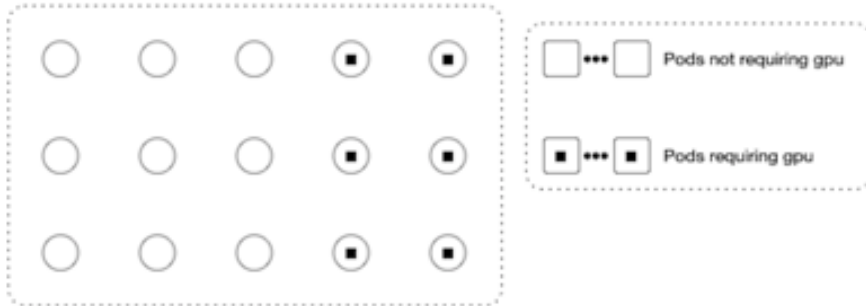
- **Example**

- 1 node in the cluster with capacity 10 units
- 2 pods (A,B) running on the node with priority 100 and each using 5 units
- Scheduler has 2 pods (C,D) in queue. Pod C needs 10 units and priority is 1000. Pod D needs 2 units and it's priority is 50
- Scheduler determines that Pod C has highest priority and destroys (A,B) so it can schedule.



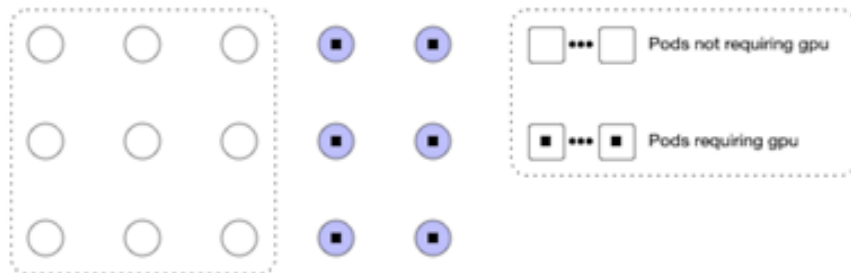
Scheduler placements

Initial State



- Non-GPU Pods eligible to run on any Node
- GPU Pods eligible to run on any Node

With Node Taints



- Non-GPU Pods eligible to run on any Non-GPU Node
- GPU Pods eligible to run on any Non-GPU Node

Case Study

- 9 nodes without GPUs
- 6 nodes with GPUs
- We need to schedule Pods that require GPUs only on nodes with GPUs and Pods that do not require GPUs on nodes without GPUs.

Node Taints

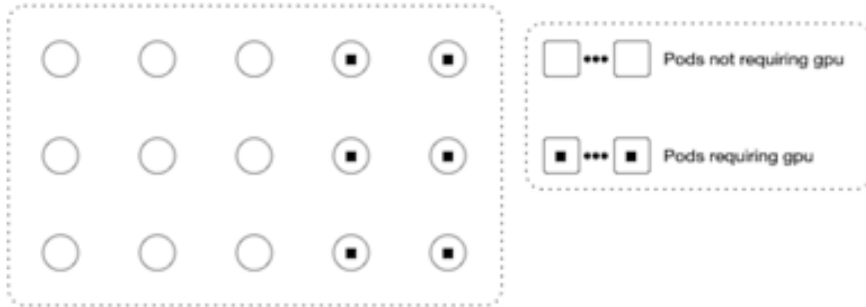
- An option is to use taints which will allow you to stop Pods from being deployed on certain nodes.



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

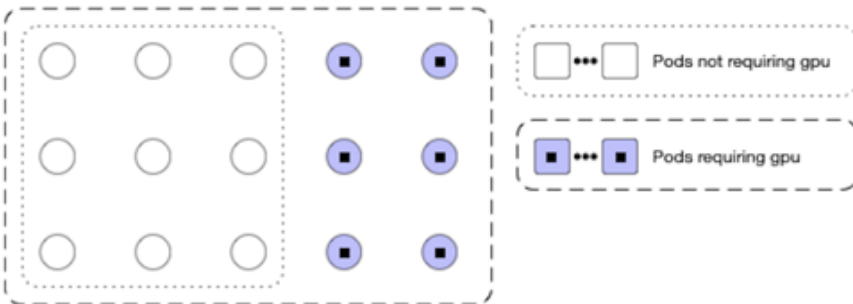
Scheduler placements

Initial State



- Non-GPU Pods eligible to run on any Node
- GPU Pods eligible to run on any Node

With Pod Tolerations



- Non-GPU Pods eligible to run on any Non-GPU Node
- GPU Pods eligible to run on any Node

Case Study

- 9 nodes without GPUs
- 6 nodes with GPUs
- We need to schedule Pods that require GPUs only on nodes with GPUs and Pods that do not require GPUs on nodes without GPUs.

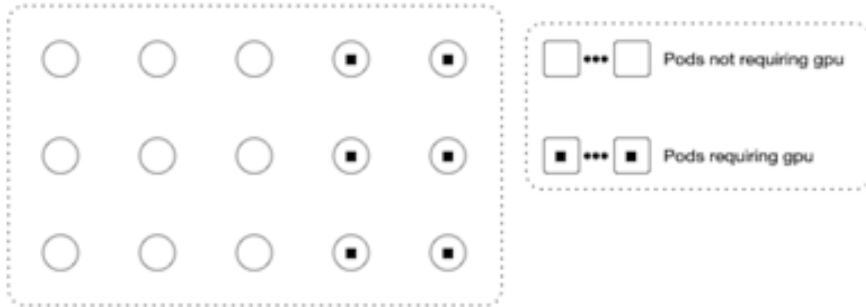
Tolerations

- Tolerations allow us to say specific Pods can run on tainted nodes.



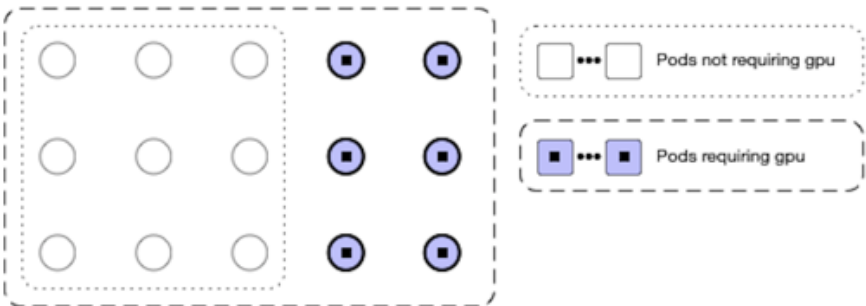
Scheduler placements

Initial State



- Non-GPU Pods eligible to run on any Node
- GPU Pods eligible to run on any Node

With Node Labels



- Non-GPU Pods eligible to run on any Non-GPU Node
- GPU Pods eligible to run on any Node

Case Study

- 9 nodes without GPUs
- 6 nodes with GPUs
- We need to schedule Pods that require GPUs only on nodes with GPUs and Pods that do not require GPUs on nodes without GPUs.

Node labels

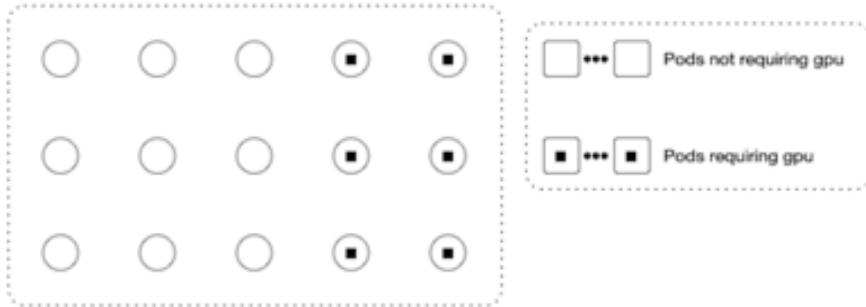
- Labels tell Pods which nodes they should be scheduled on.



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

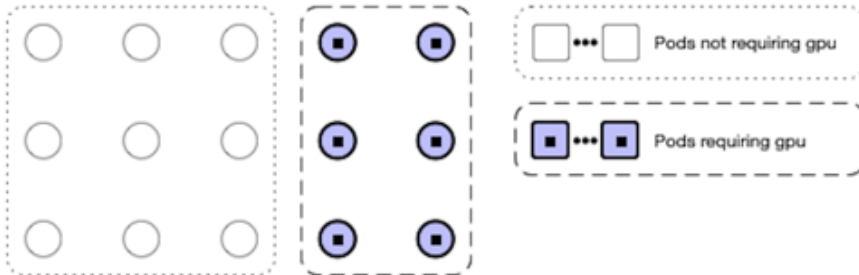
Scheduler placements

Initial State



- Non-GPU Pods eligible to run on any Node
- GPU Pods eligible to run on any Node

With Node Affinity



- Non-GPU Pods eligible to run on any Non-GPU Node
- GPU Pods eligible to run on any GPU Node

Case Study

- 9 nodes without GPUs
- 6 nodes with GPUs
- We need to schedule Pods that require GPUs only on nodes with GPUs and Pods that do not require GPUs on nodes without GPUs.

Node affinity

- Similar to node labels but more expressive
 - In/NotIn/Exists/DoesNotExist/Lt,Gt
 - Required or preferred
 - Schedule based on Pods running on other nodes.

Built-in Node Labels

- `kubernetes.io/hostname`
- `failure-domain.beta.kubernetes.io/zone`
- `failure-domain.beta.kubernetes.io/region`
- `beta.kubernetes.io/instance-type`
- `beta.kubernetes.io/os`
- `beta.kubernetes.io/arch`

Kubernetes Advanced POD Scheduling



Kubernetes Scheduling

- Node Selector
- Node Affinity
- Pod Affinity
- Pod Anti Affinity
- 3rd party schedulers



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Kubernetes Scheduling

- K8s 1.6 adds more advanced scheduling options, including
 - Node affinity/anti-affinity
 - generalizing the nodeSelector feature
 - Node taints
 - prevent scheduling of pods to nodes unless pod 'tolerates' the taint
 - Pod affinity/anti-affinity
 - control relative placement of pods

Node Affinity

The affinity/anti-affinity feature greatly expands the types of constraints you can express.

The key enhancements are:

- More expressive language (not just “AND of exact match”)
- Rule is “soft”/”preference” so if it doesn’t match Pods are still scheduled.

Kubernetes Scheduling

Node Affinity

```
affinity:  
  nodeAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: "failure-domain.beta.kubernetes.io/zone"  
            operator: In  
            values: ["us-central1-a"]
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Kubernetes Scheduling

Node Affinity

```
affinity:  
  nodeAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: "failure-domain.beta.kubernetes.io/zone"  
            operator: In  
            values: ["us-central1-a"]
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Kubernetes Scheduling

Weight example

affinity:

nodeAffinity:

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 5

preference:

- matchExpressions:

- key: env

- operator: In

- values:

- dev

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 1

preference:

- matchExpressions:

- key: team

- operator: In

- values:

- engineering



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Node Affinity

Kubernetes supports "weighted scheduling"

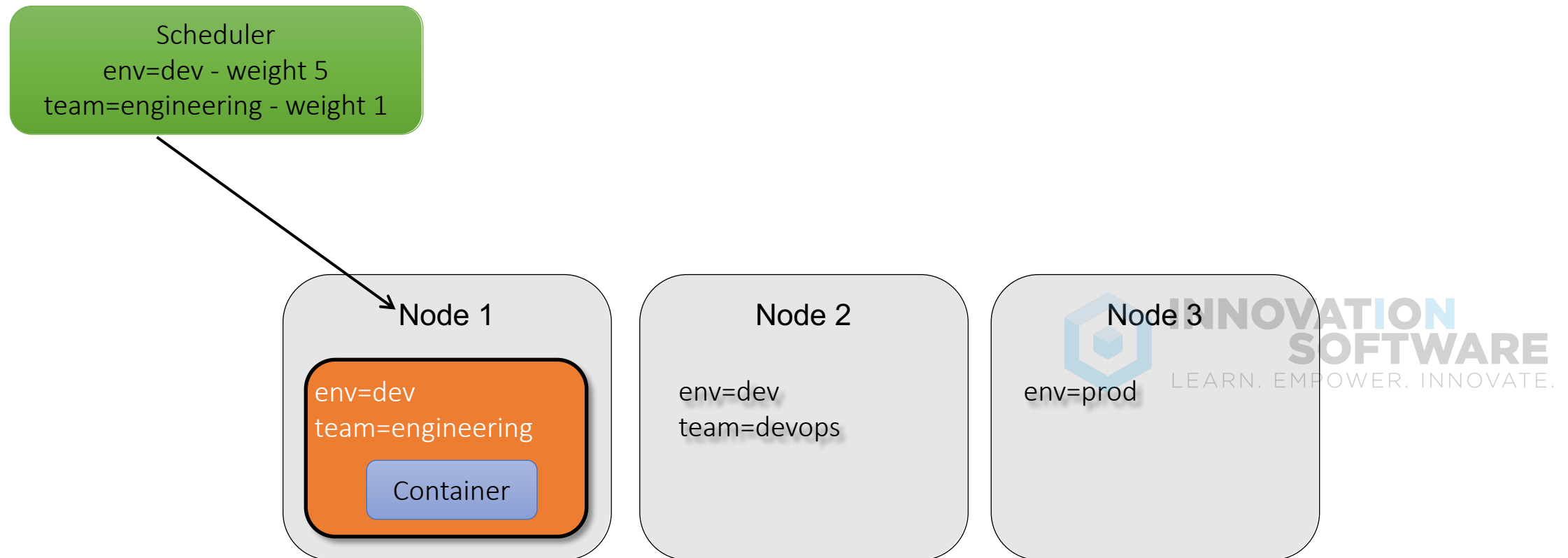
- The higher this weighting the more weight is given to that rule.
- Scoring is done by summarizing weighting per node.
 - 2 rules weights 1 and 5
 - If both match = score of 6
 - If only 1 rule matches = score of 1
- Node with highest total weight score receives Pod.



INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.

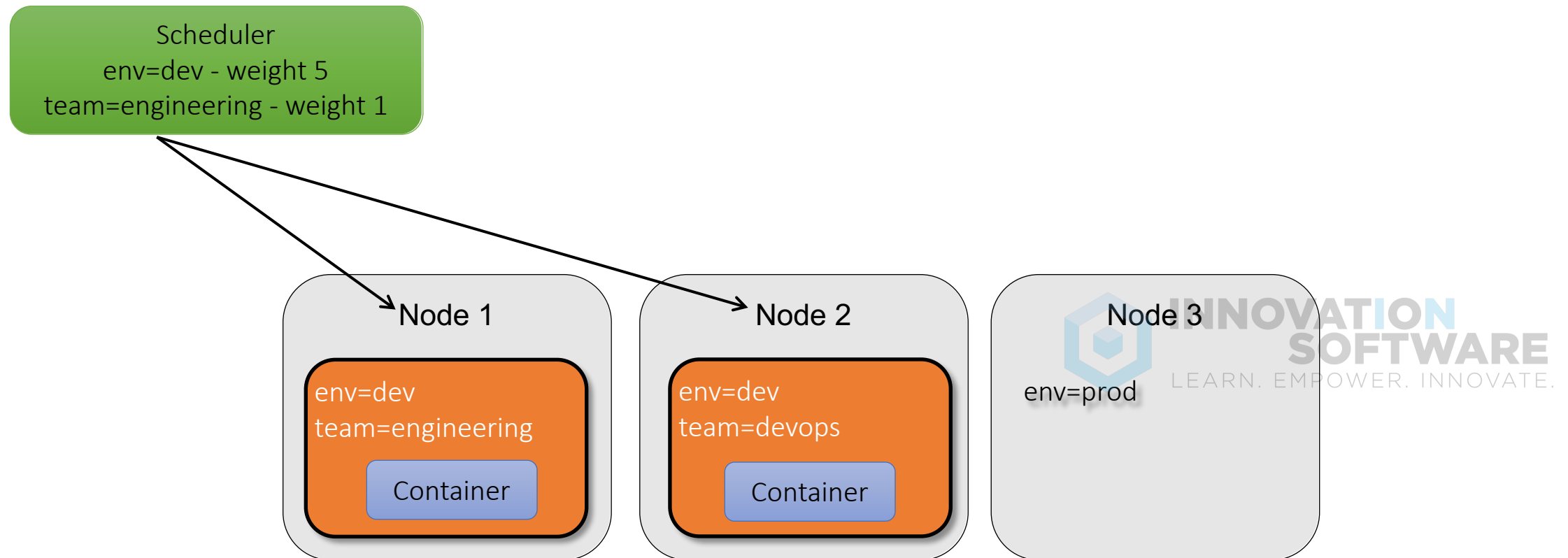
Node Affinity: Weight

- Schedule Pods onto nodes that have labels
 - env = dev
 - team = engineering



Node Affinity: Weight

- Schedule Pods onto nodes that have labels
 - env = dev
 - team = engineering



Kubernetes Scheduling

Node Anti-Affinity

```
affinity:  
  nodeAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: "failure-domain.beta.kubernetes.io/zone"  
            operator: NotIn  
            values: ["us-central1-a"]
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Operators

- Valid operators
 - In
 - NotIn
 - Exists
 - DoesNotExist
 - Gt (Greater Than)
 - Lt (Less Than)

Node Management



INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.

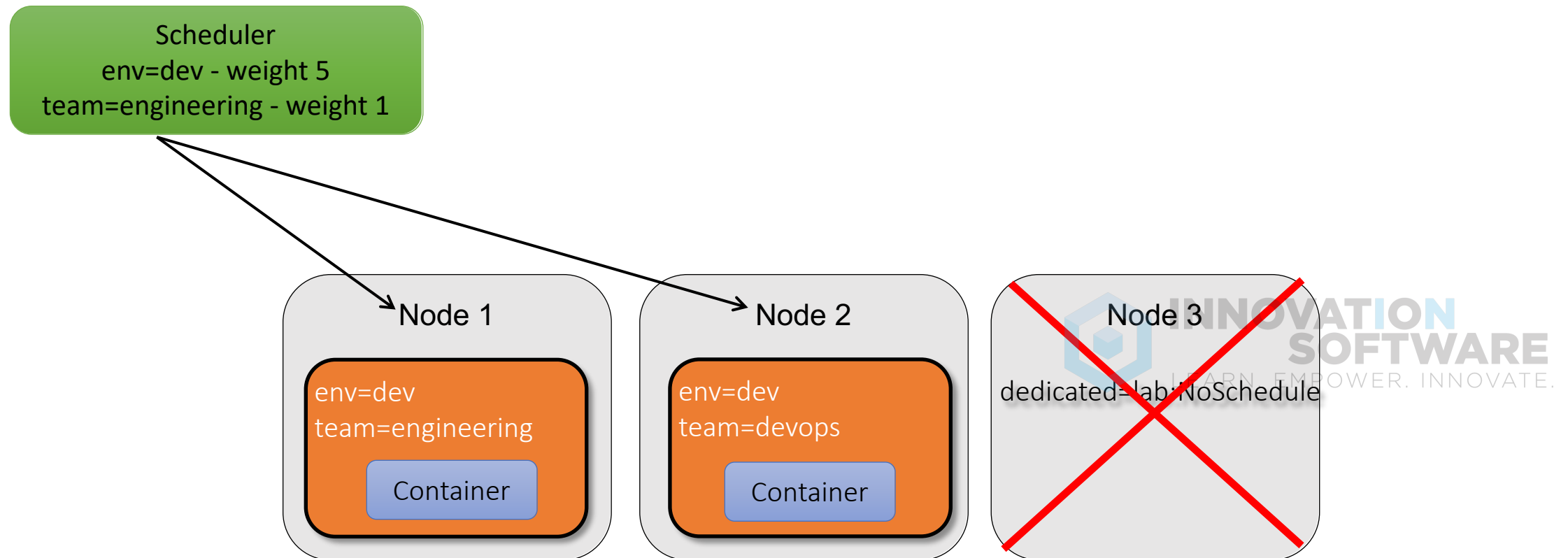


Node Taints

- Allows you to mark (“taint”) a node
 - No pods can be scheduled onto tainted nodes
 - Useful when all or most PODs should not be scheduled on a node
 - Mark your master node as schedulable only by Kubernetes system components
 - Keep regular pods away from nodes that have special hardware so as to leave room for pods that need the special hardware.

Node Management: Taint

- Scheduler will not deploy pods to Node 3
 - dedicated=lab:NoSchedule
 - team = engineering



Node Tolerations

- To allow a POD to be scheduled onto a 'tainted' node it must have:

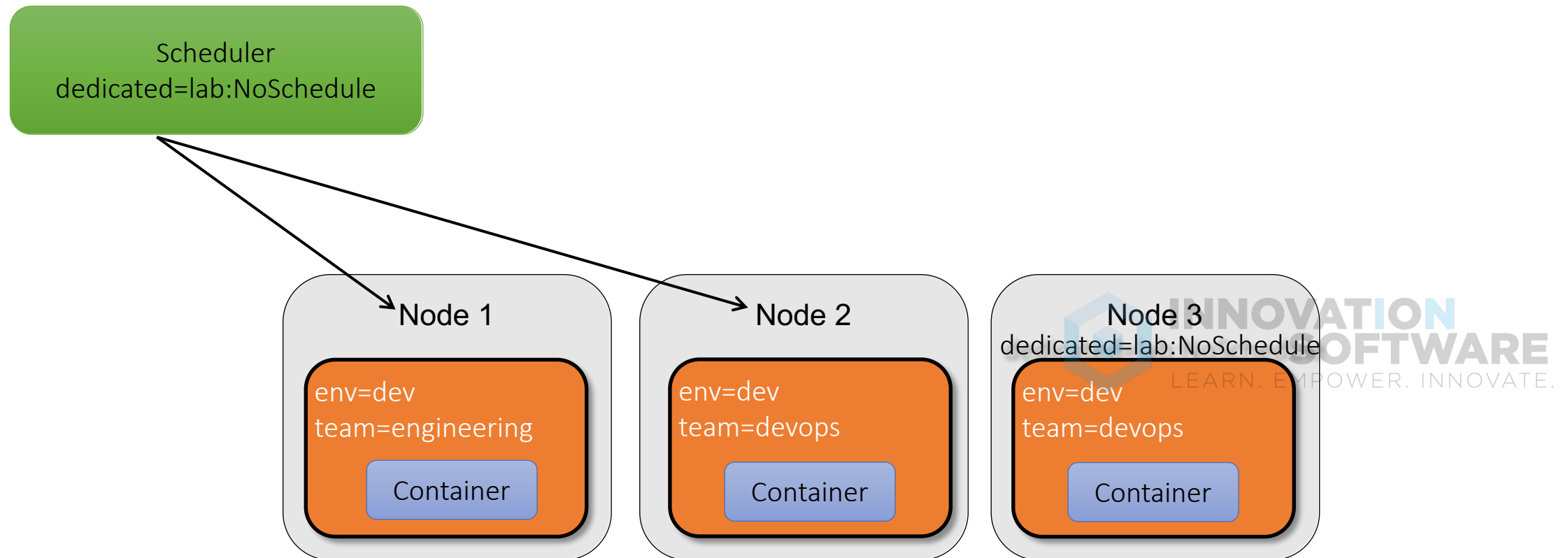
```
tolerations:  
- key: "key"  
  operator: "Equal"  
  value: "value"  
  effect: "NoSchedule"
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Node Management: Toleration

- Apply toleration to Node 3
 - `dedicated=lab:NoSchedule`

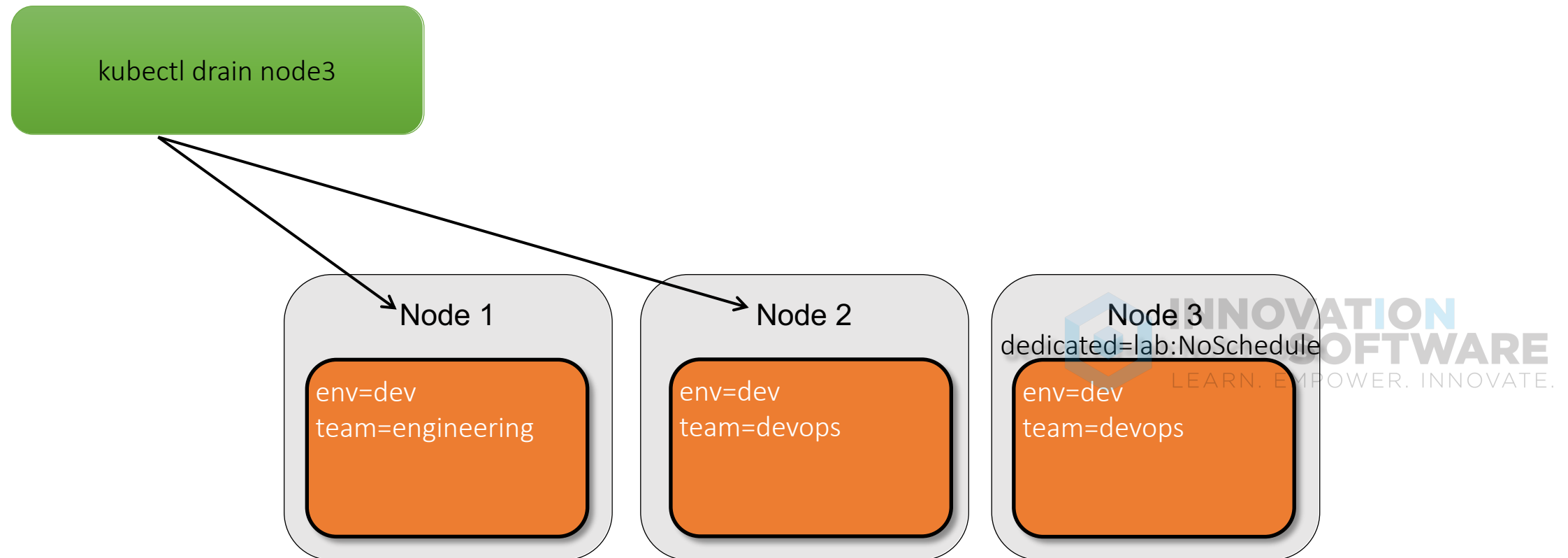


Drain Node(s)

- Evict all pods from node(s)
 - Pods managed by controller (ReplicaSet, StatefulSet, DaemonSet) are recreated on active nodes if specified in configuration.
 - No pods are scheduled to node(s) while in 'cordon' mode.
 - Respects PodDisruptionBudget
 - Uncordon node when ready to schedule pods to it.

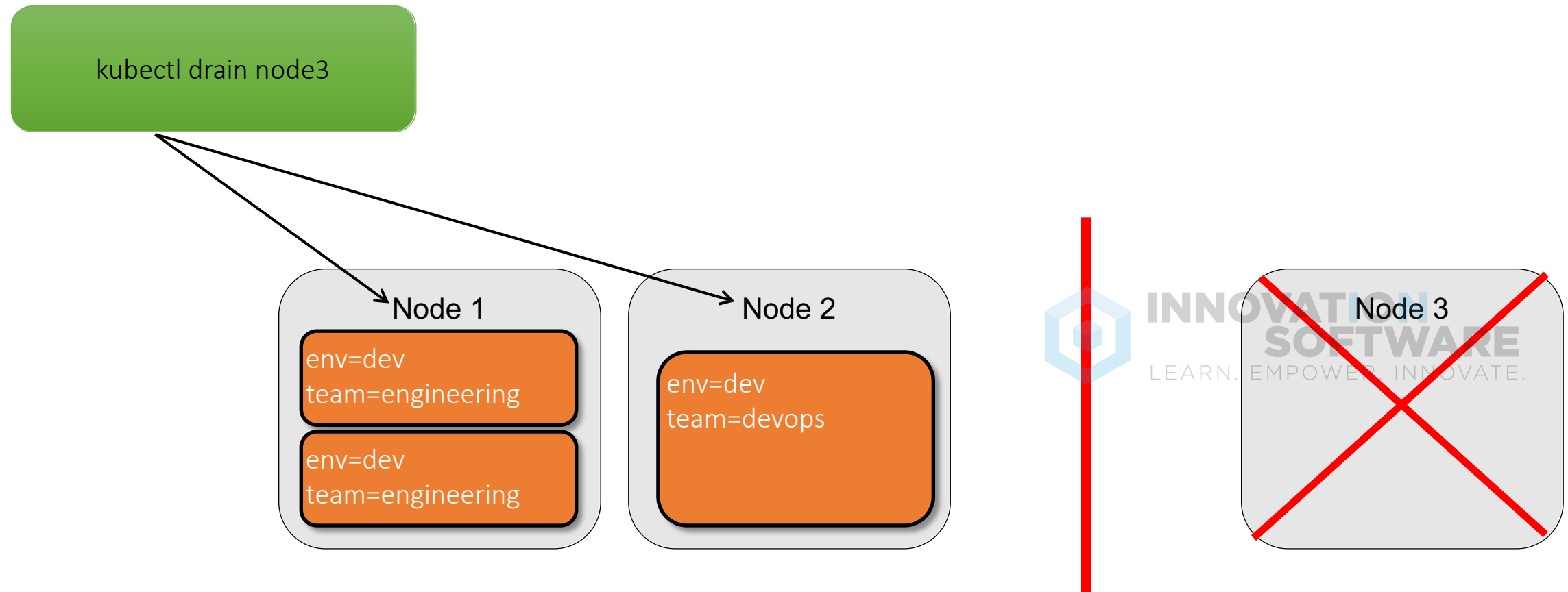
Node Management: Drain

- Evict pods from Node 3



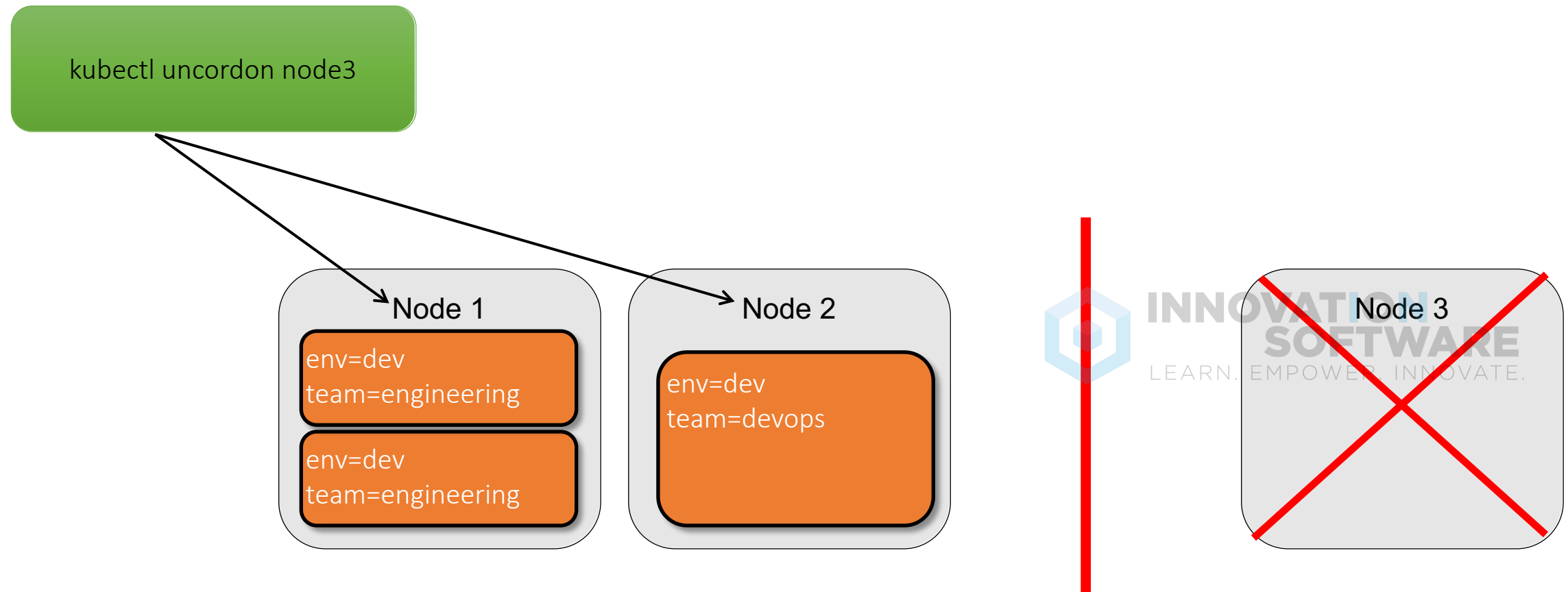
Node Management: Drain

- Evict pods from Node 3



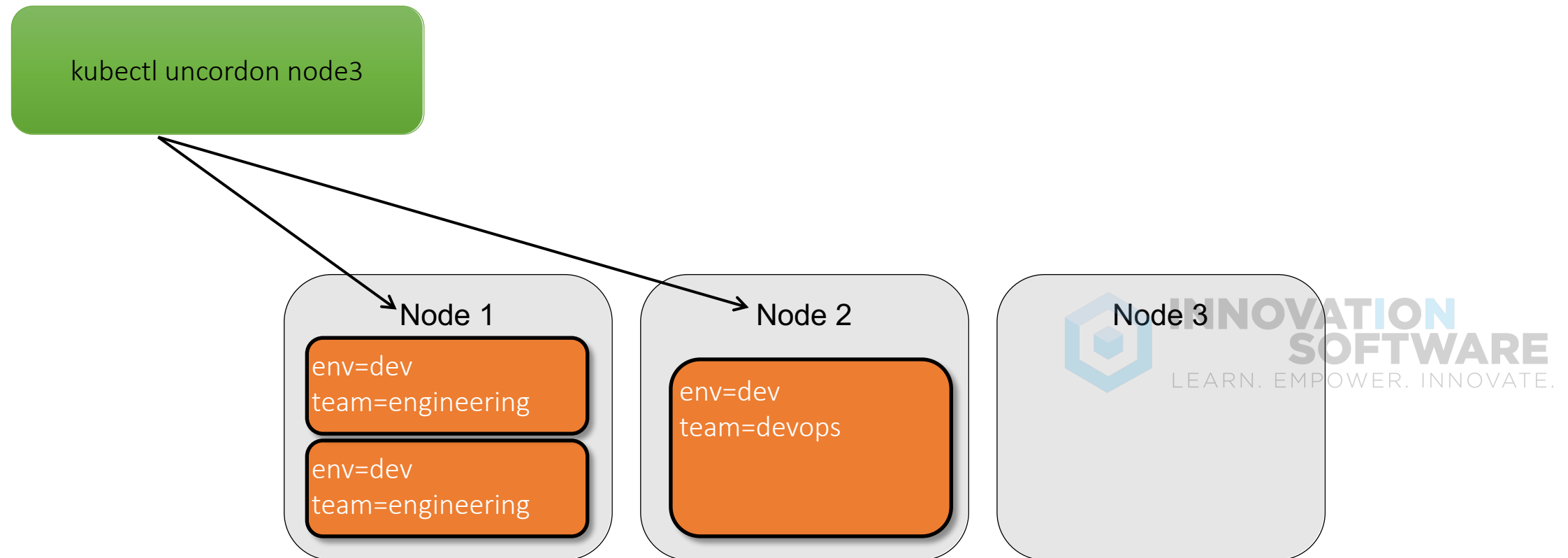
Node Management: Uncordon

- Return Node 3 to active state



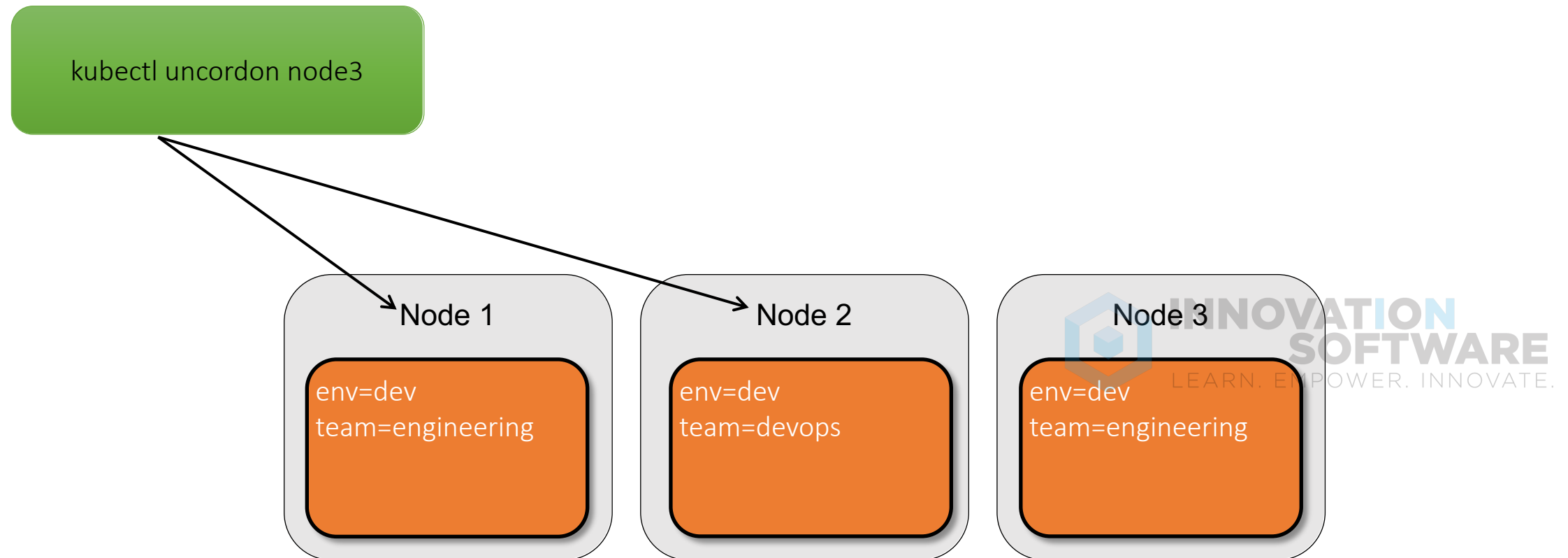
Node Management: Uncordon

- Return Node 3 to active state



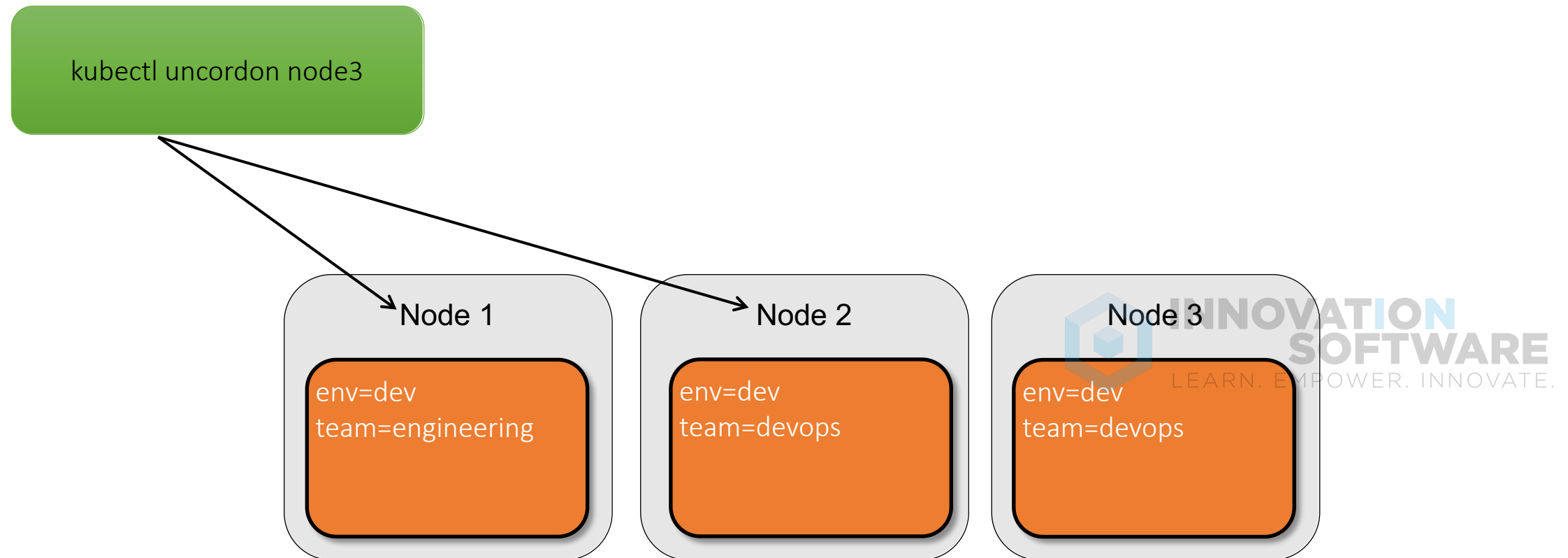
Node Management: Uncordon

- Return Node 3 to active state



Node Management: Uncordon

- Evict pods from Node 3



Pod Affinity

- Define how pods should be placed relative to one another
- Spread or pack pods within a service or relative to pods in other services

```
affinity:  
  podAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
        matchExpressions:  
          - key: service  
            operator: In  
            values: ["S1"]  
        topologyKey: failure-domain.beta.kubernetes.io/zone
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Affinity

- Define how pods should be placed relative to one another
- Spread or pack pods within a service or relative to pods in other services

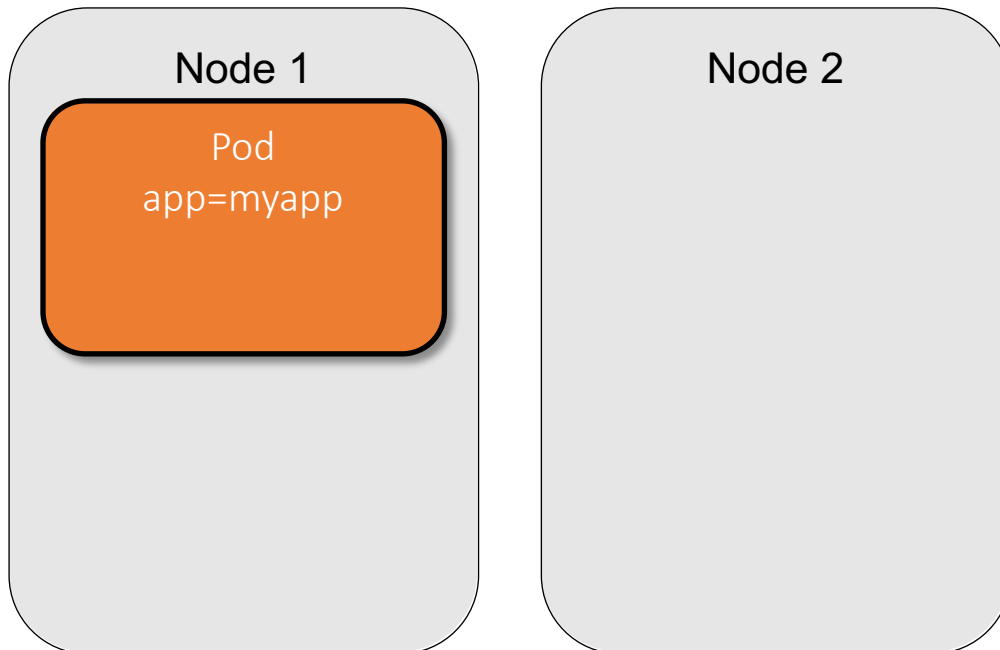
```
affinity:  
  podAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: service  
              operator: In  
              values: ["S1"]  
          topologyKey: failure-domain.beta.kubernetes.io/zone
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Affinity

- Schedule Pods onto nodes that have same labels
 - app=myapp
 - operator: In
 - kubernetes.io/hostname



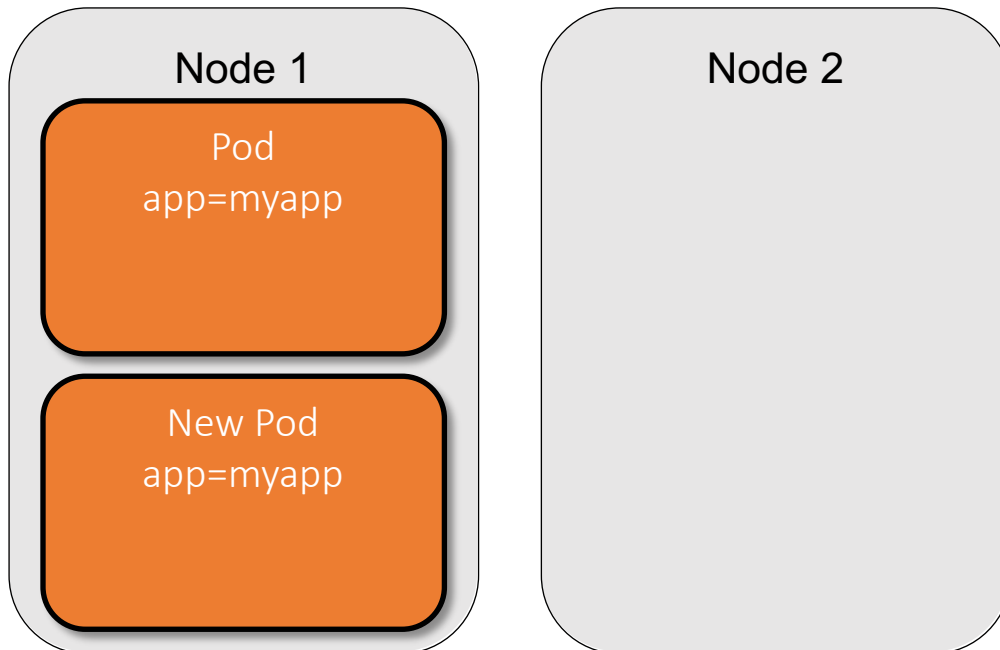
```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Affinity

- Schedule Pods onto nodes that have same labels
 - app=myapp
 - operator: In
 - kubernetes.io/hostname



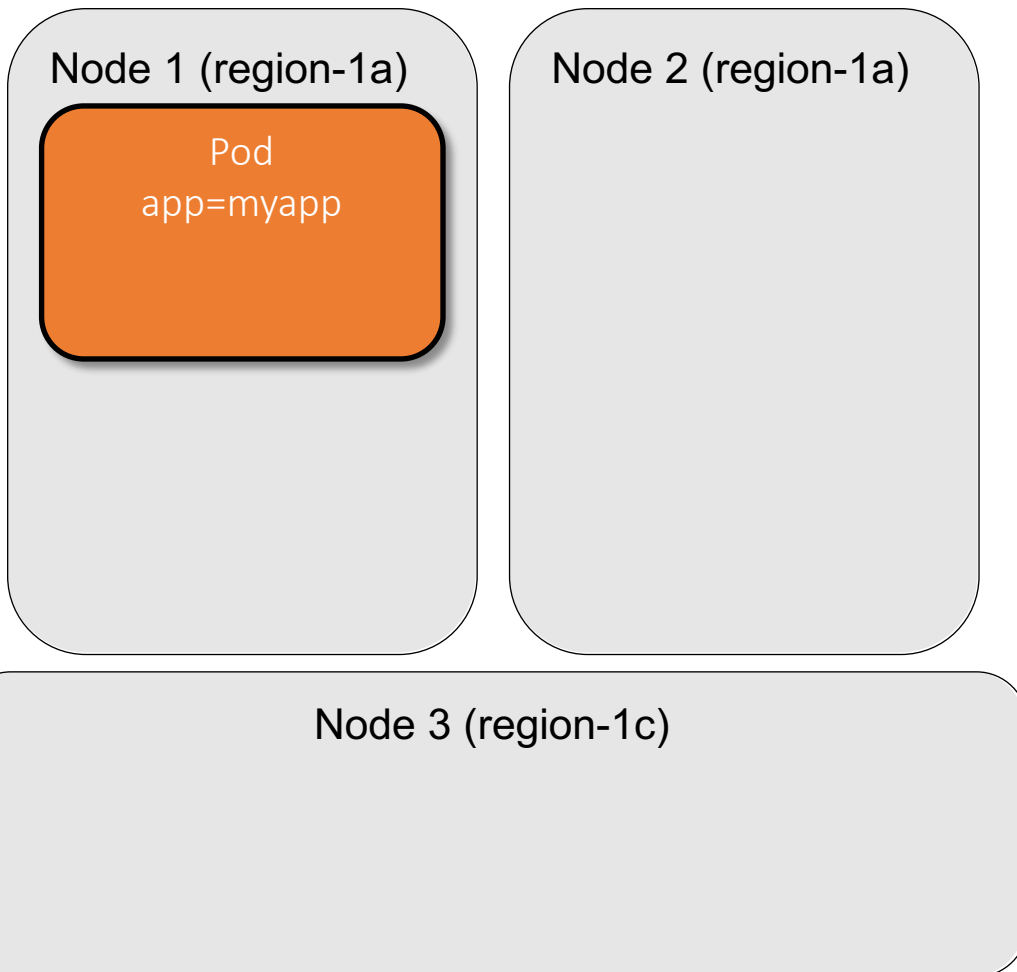
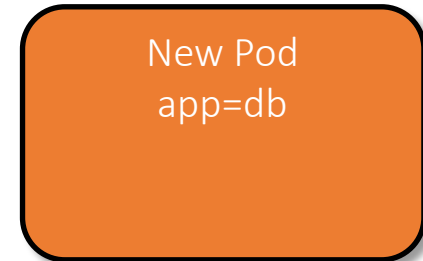
```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Affinity

- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



affinity:

podAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

- labelSelector:

matchExpressions:

- key: "app"

operator: In

values:

- myapp

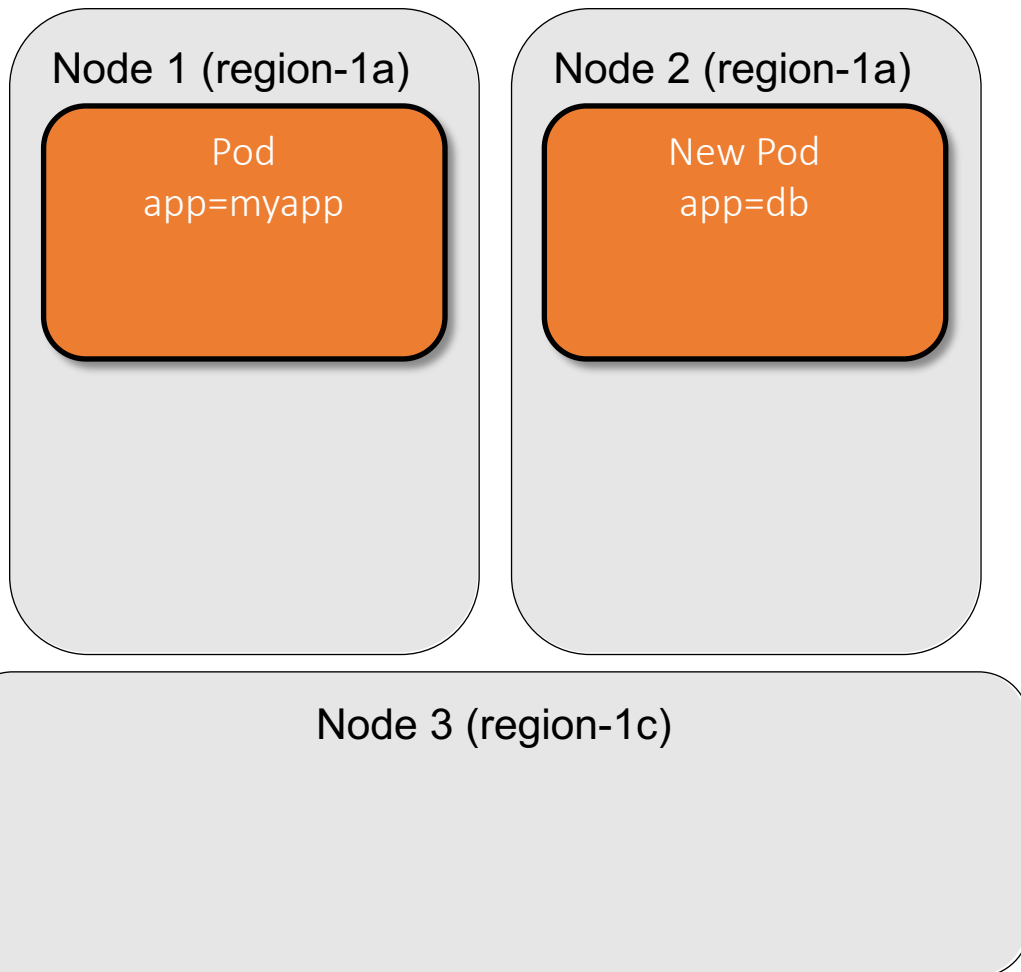
topologyKey: " failure-domain.beta.kubernetes.io/zone"



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Affinity

- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



affinity:

podAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

- labelSelector:

matchExpressions:

- key: "app"

operator: In

values:

- myapp

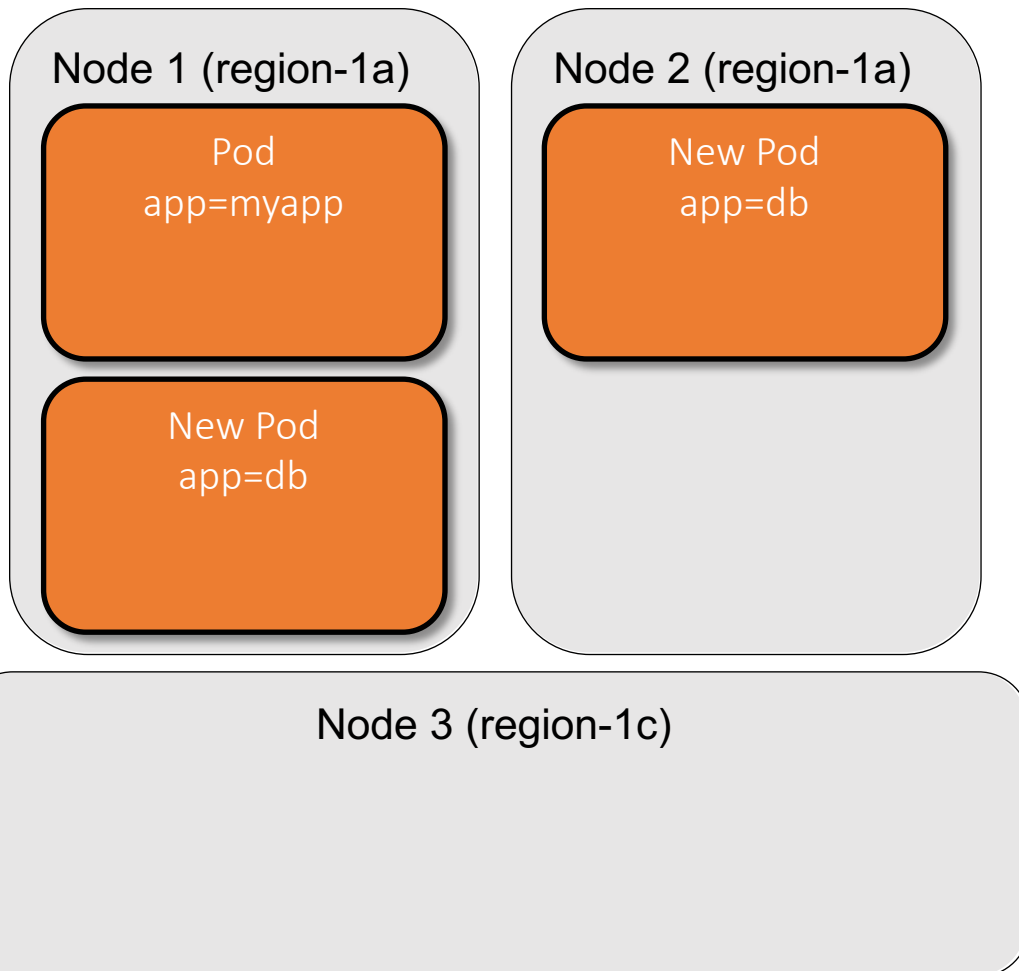
topologyKey: " failure-domain.beta.kubernetes.io/zone"



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Affinity

- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



affinity:

podAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

- labelSelector:

matchExpressions:

- key: "app"

operator: In

values:

- myapp

topologyKey: " failure-domain.beta.kubernetes.io/zone"



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Anti-Affinity

- Define how pods should be placed relative to one another
- PODs of different services run on different nodes.

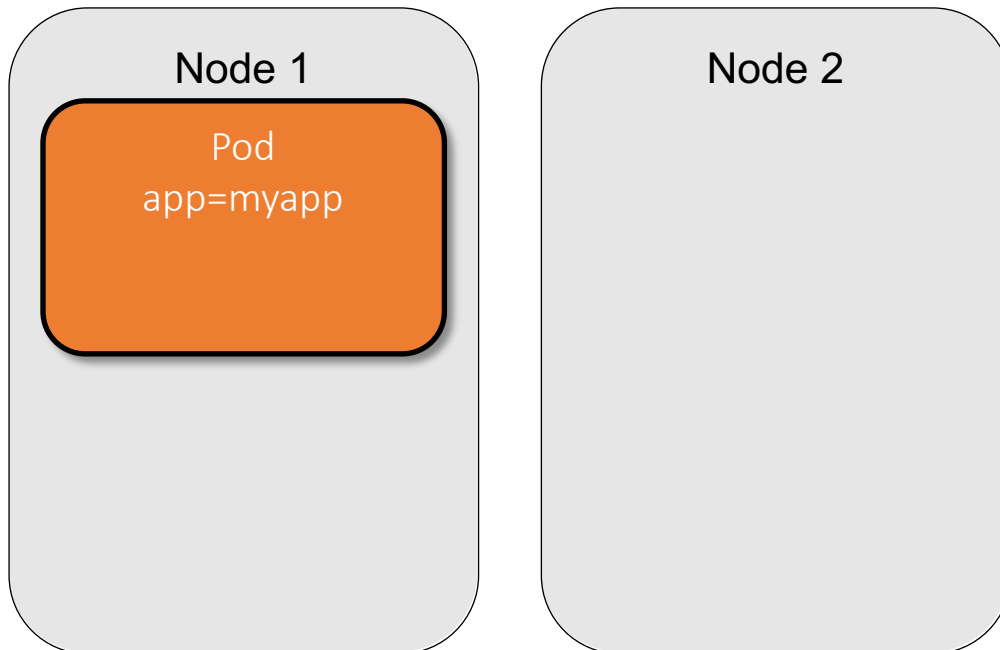
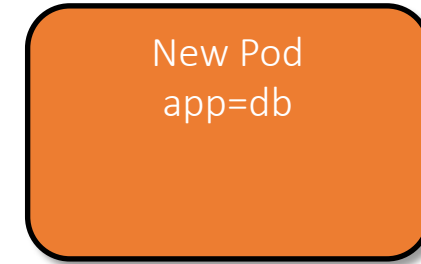
```
affinity:  
  podAntiAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: service  
              operator: In  
              values: ["S1"]  
        topologyKey: kubernetes.io/hostname
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Anti-Affinity

- Do not schedule Pods onto nodes with matching labels.
 - app=db
 - operator: In
 - kubernetes.io/hostname



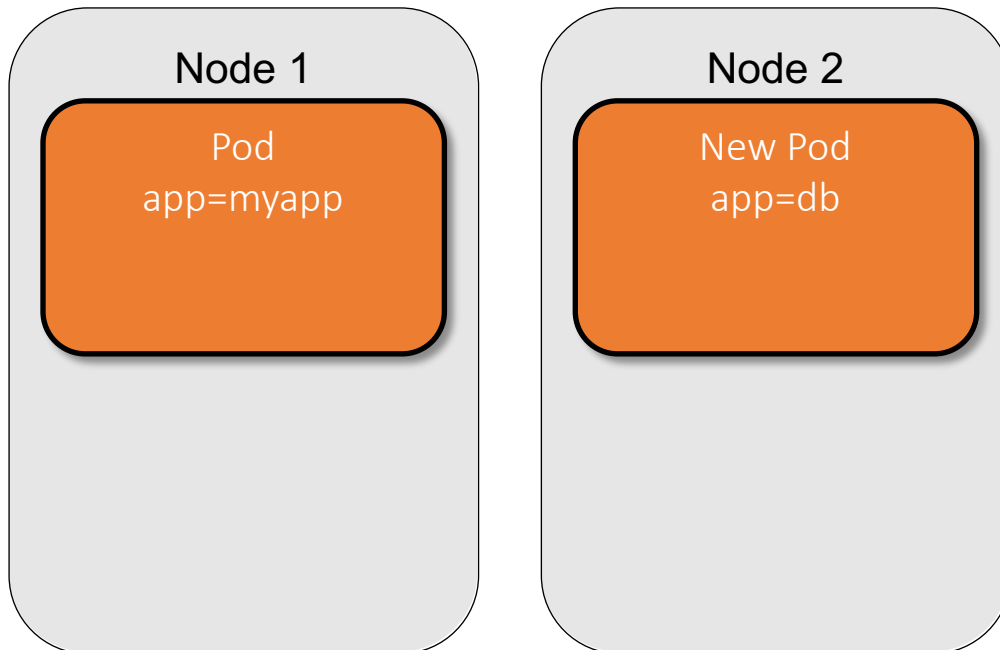
```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Anti-Affinity

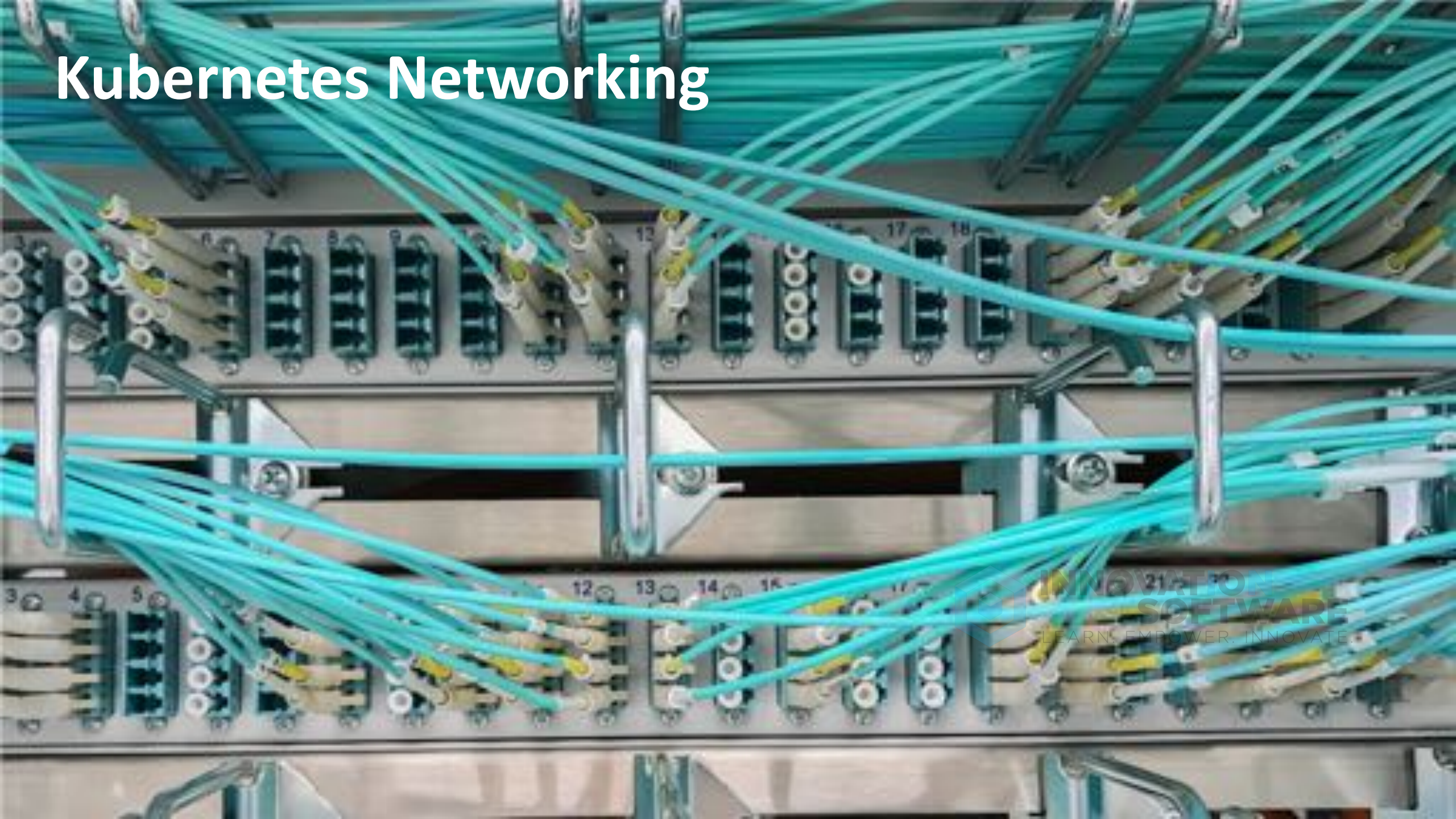
- Do not schedule Pods onto nodes with matching labels.
 - app=db
 - operator: In
 - kubernetes.io/hostname



```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```

INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.

Kubernetes Networking



Networking Problems and Solutions

1. Highly-coupled container-to-container communications => linux
2. Pod-to-Pod communications => various
3. Pod-to-Service communications => service
4. External-to-Service communications => service or ingress

Kubernetes Networking Model Requirements

- all containers can communicate with all other containers without NAT
- all nodes can communicate with all containers (and vice-versa) without NAT
- the IP that a container sees itself as is the same IP that others see it as

Container to Container Communication

- Containers live inside of pods
- Group of one or more containers that are always co-located and co-scheduled, and run in a shared context
- Containers within a POD share an IP address and port space, and can find each other via localhost (net namespace).
- Communicate with each other using standard inter-process communications (ipc namespace).



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod to Pod Requirements

- Pods can communicate with all other pods without NAT
- Regardless of which host they land on
- Every pod gets its own IP address
- No need to explicitly create links between pods



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Pod Networking Approaches

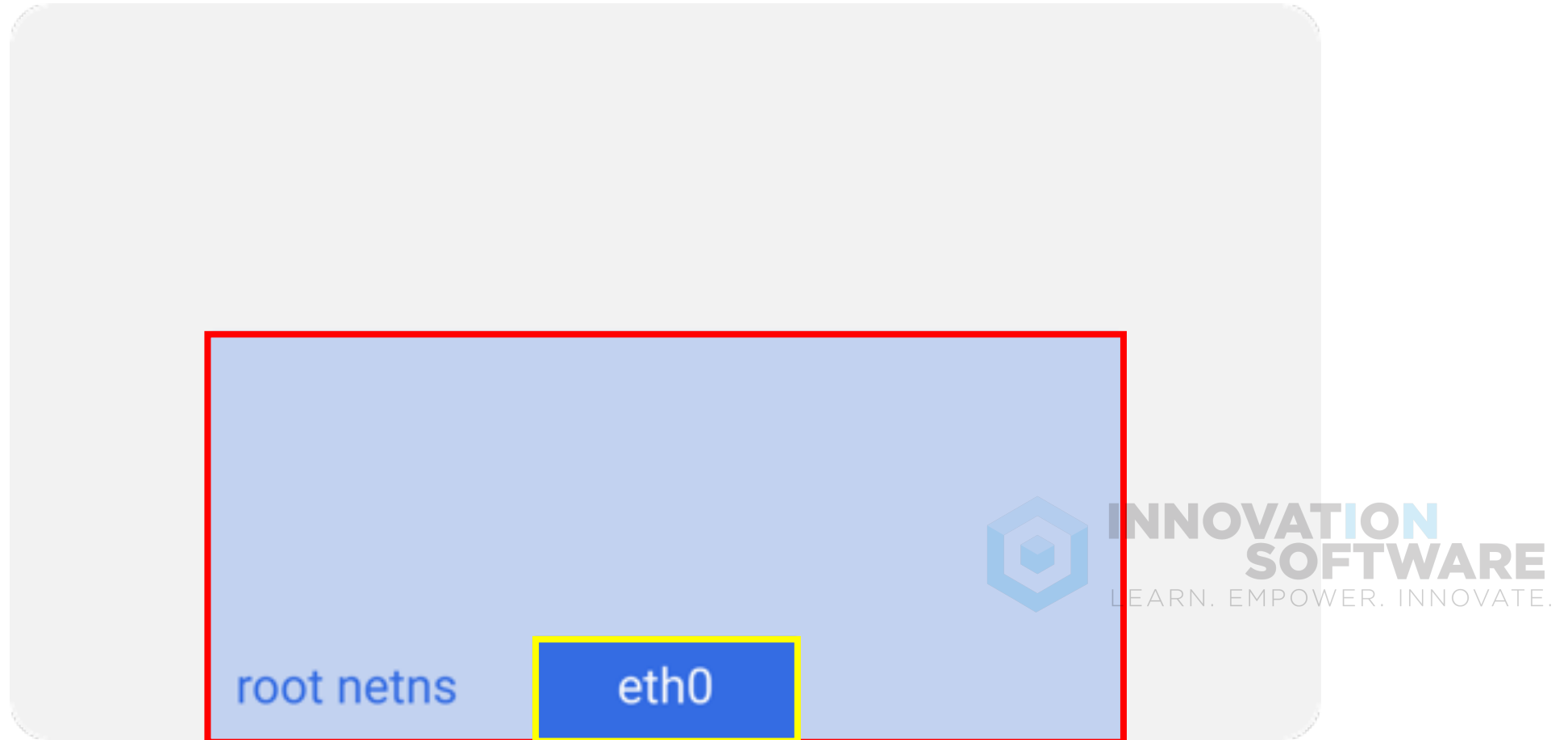
- **Using an overlay network**

- An overlay network obscures the underlying network architecture from the pod network through traffic encapsulation (for example vxlan).
- Encapsulation reduces performance, though exactly how much depends on your solution.

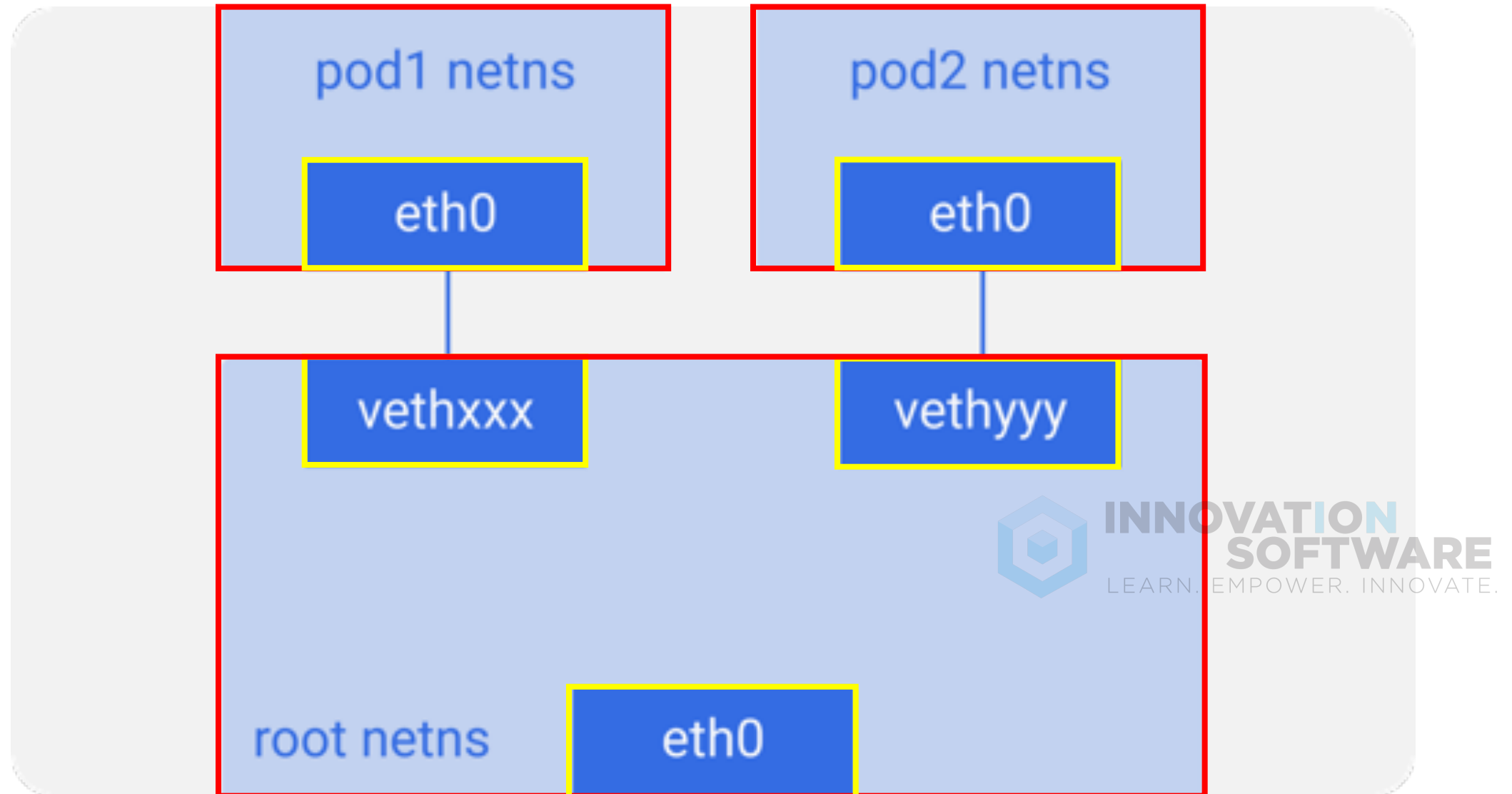
- **Without an overlay network**

- Configure the underlying network fabric (switches, routers, etc.) to be aware of pod IP addresses.
- This does not require the encapsulation provided by an overlay, and so can achieve better performance.

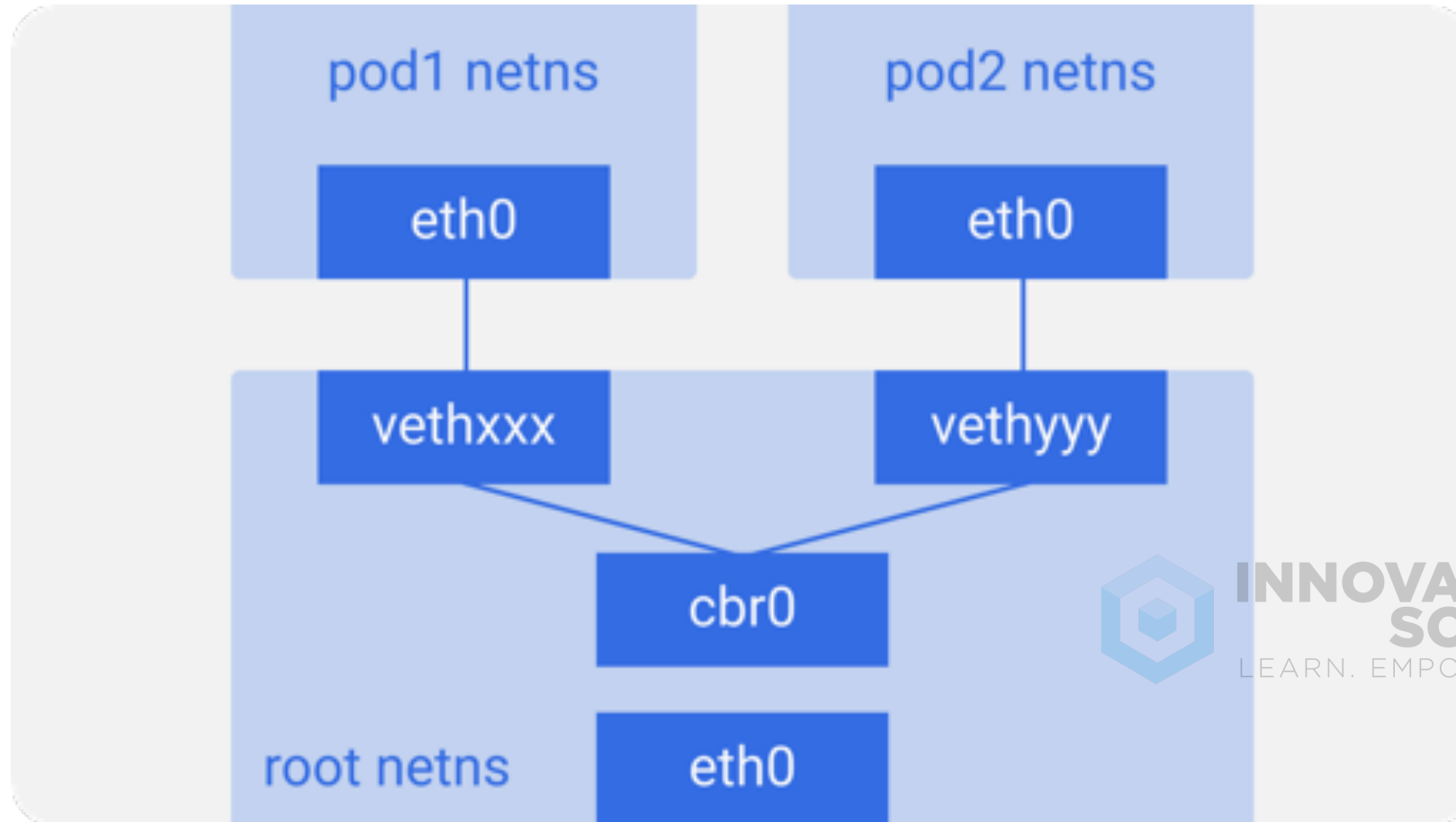
Node Namespace



Node Namespaces



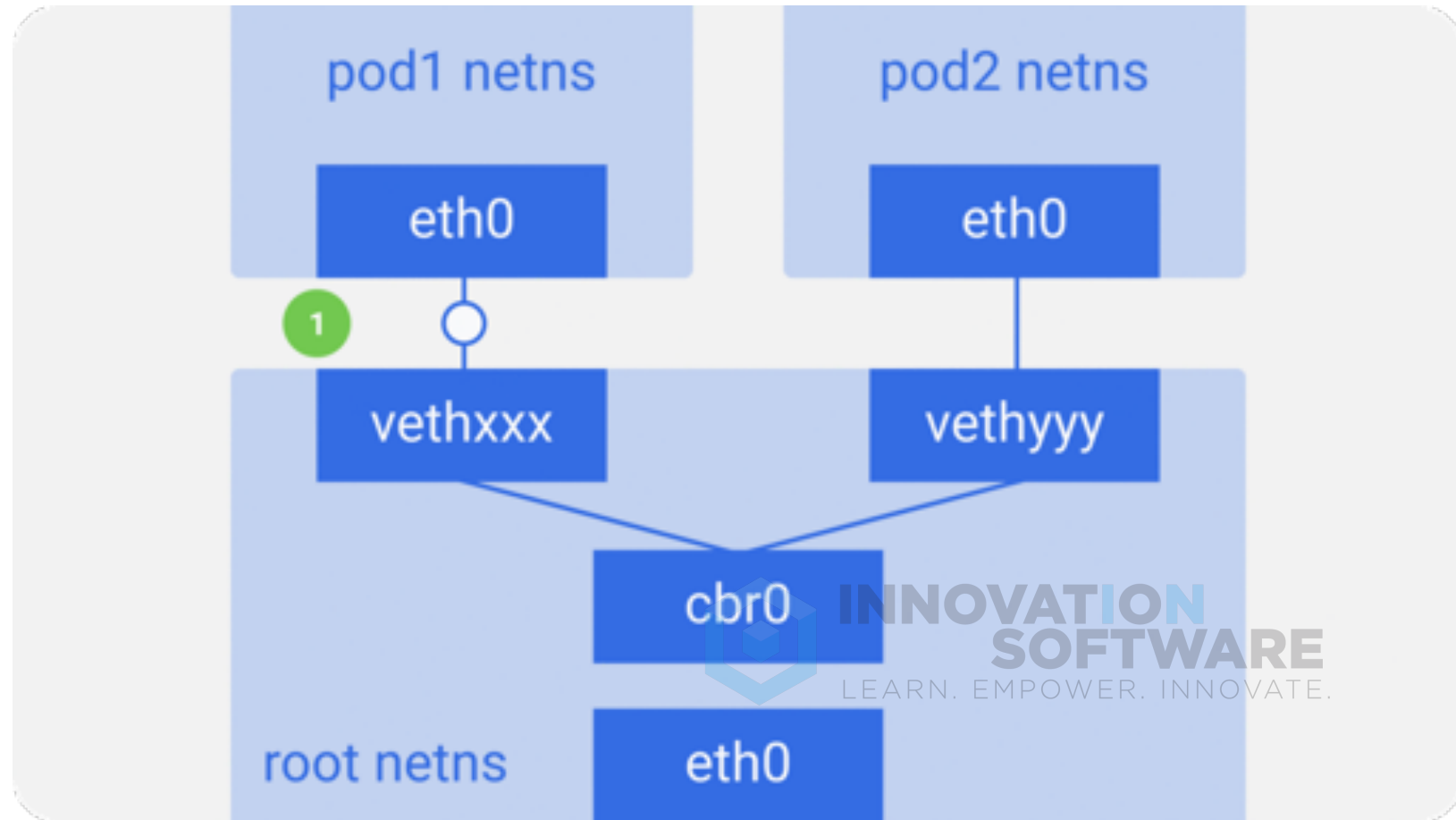
Pod to Host



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

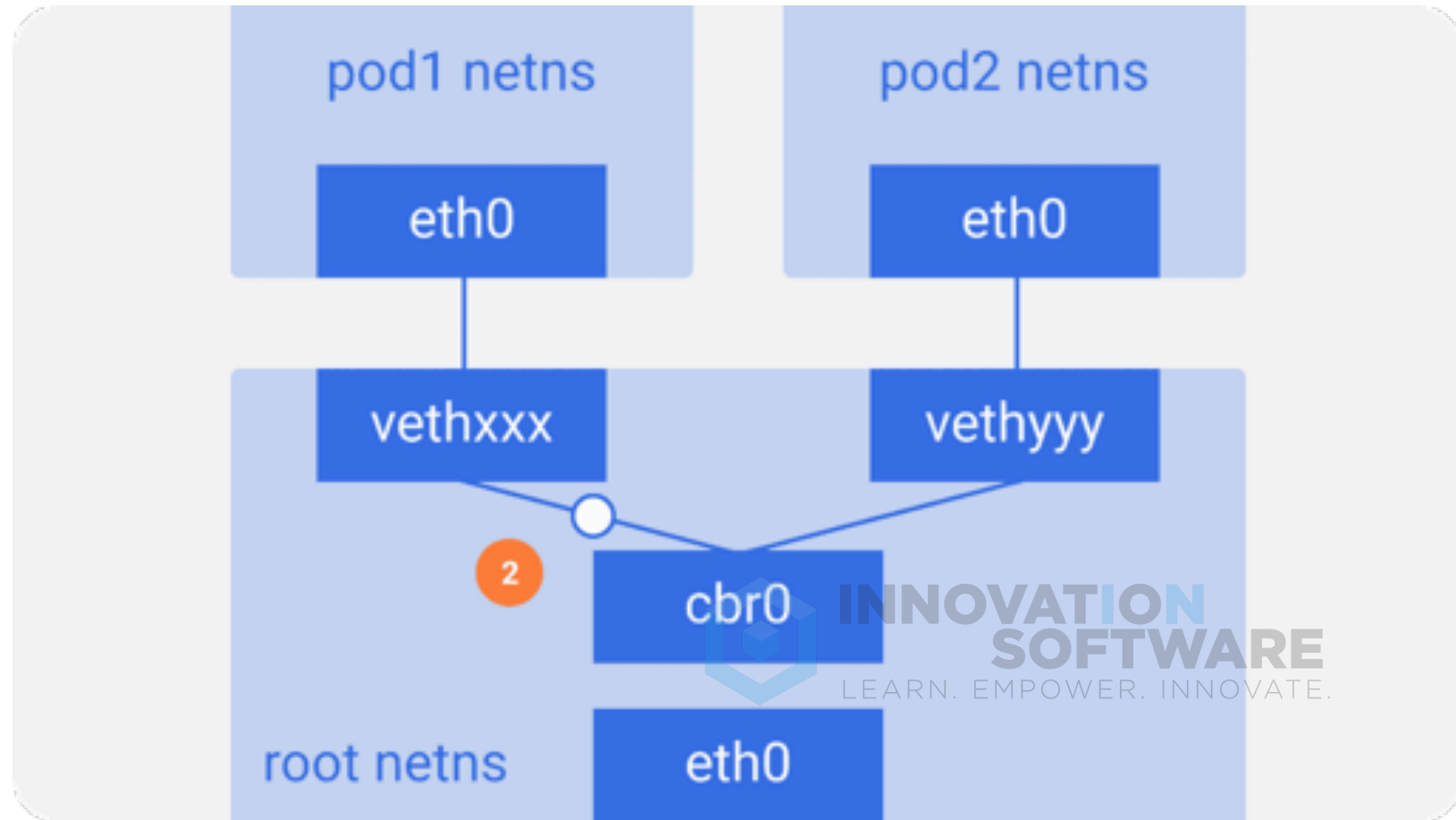
Single Host POD to POD Network

1. Packet leaves pod1's netns at eth0 and enters the root netns at vethxxx.



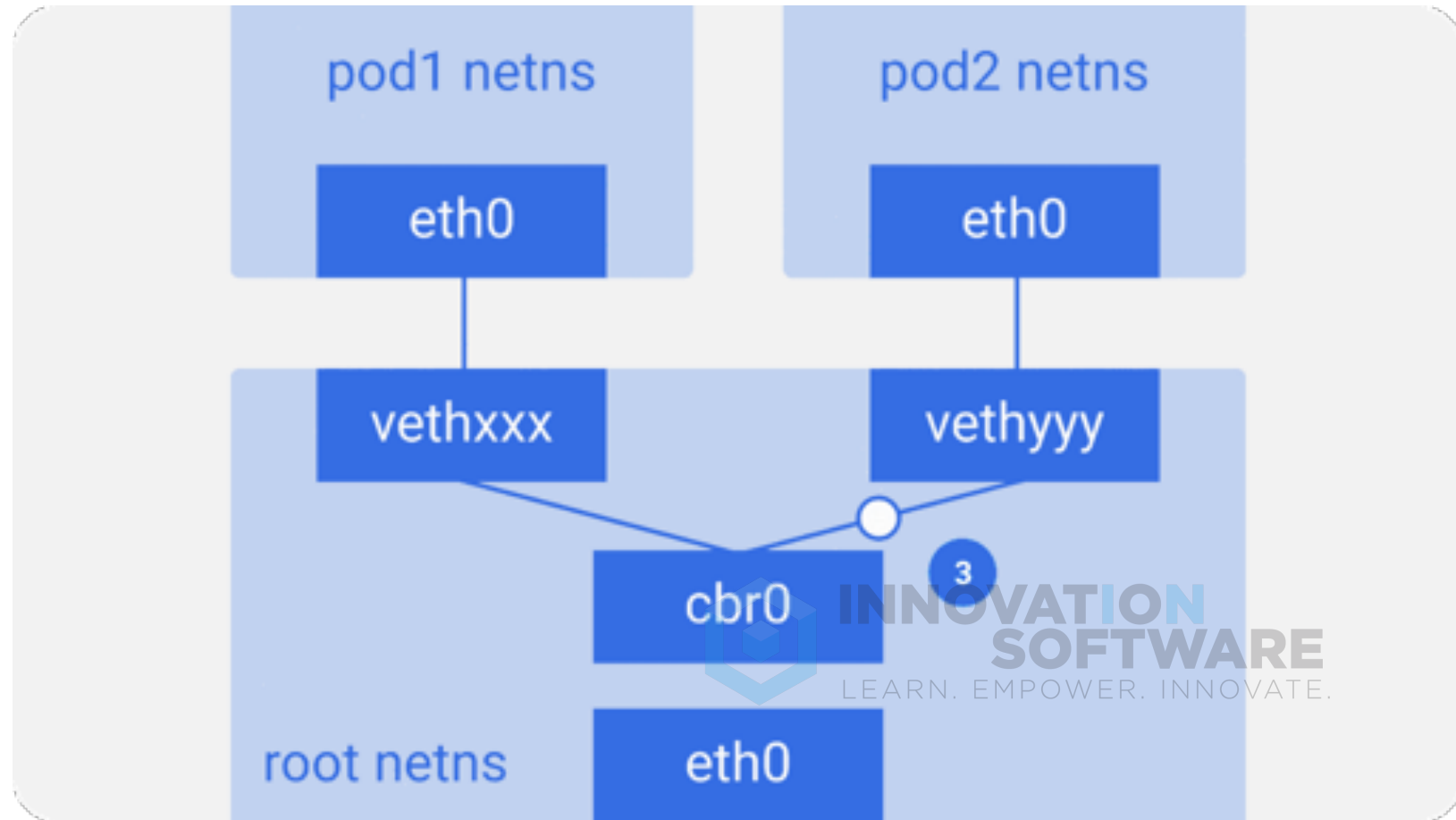
Single Host POD to POD Networking

2. It's passed on to cbr0, which discovers the destination using an ARP request, saying "who has this IP?"



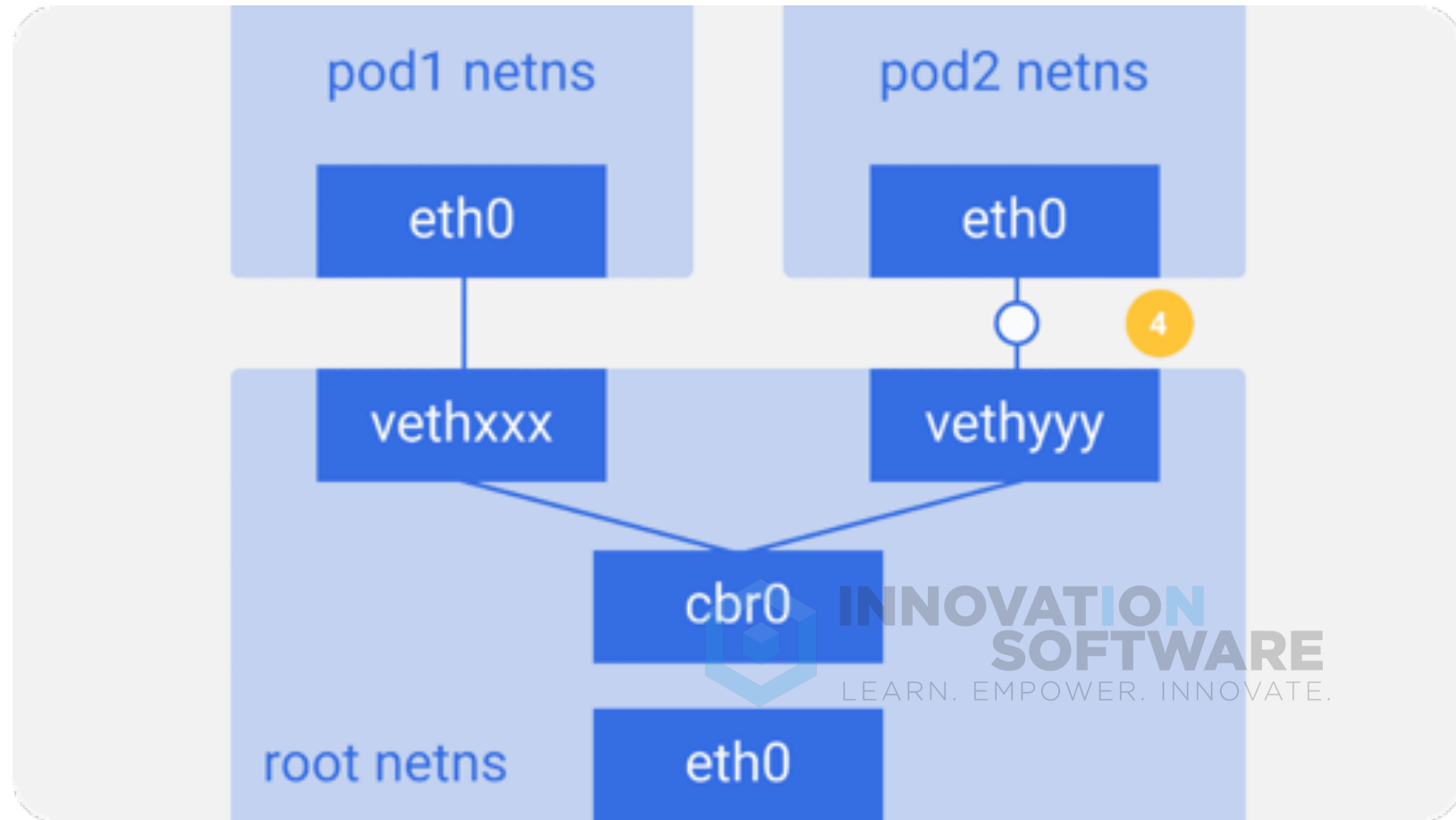
Single Host POD to POD Networking

3. vethyyy says it has that IP, so the bridge knows where to forward the packet.



Single Host POD to POD Networking

4. The packet reaches vethyyy, crosses the pipe-pair and reaches pod2's netns.



Kubernetes Inter-node Communication



INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.



Inter-node Communication

- PODs must be reachable across nodes
- Kubernetes doesn't care how:
 - L2 (ARP across nodes)
 - L3 (IP routing across nodes)
 - Overlay networks



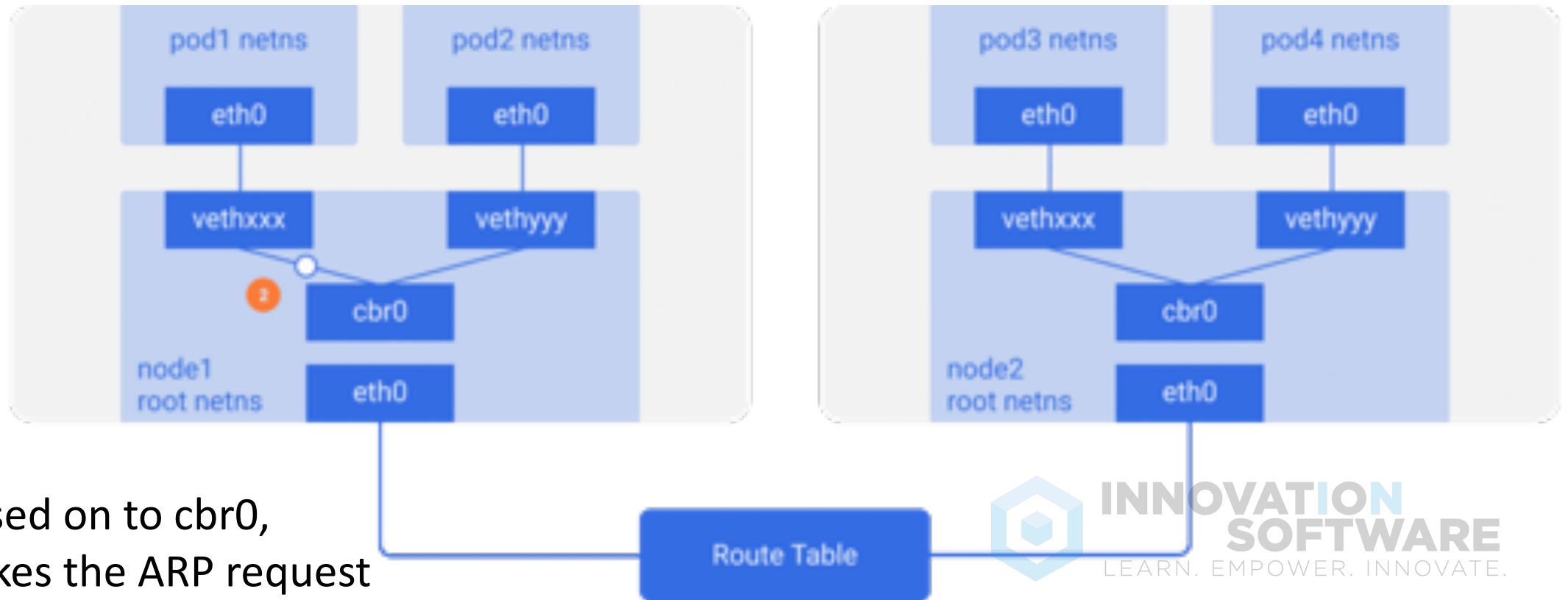
INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.

Inter-node Communication



1. It leaves POD1's netns at eth0 and enters the root netns at vethxxx.

Inter-node Communication



2. It's passed on to cbr0, which makes the ARP request to find the destination.



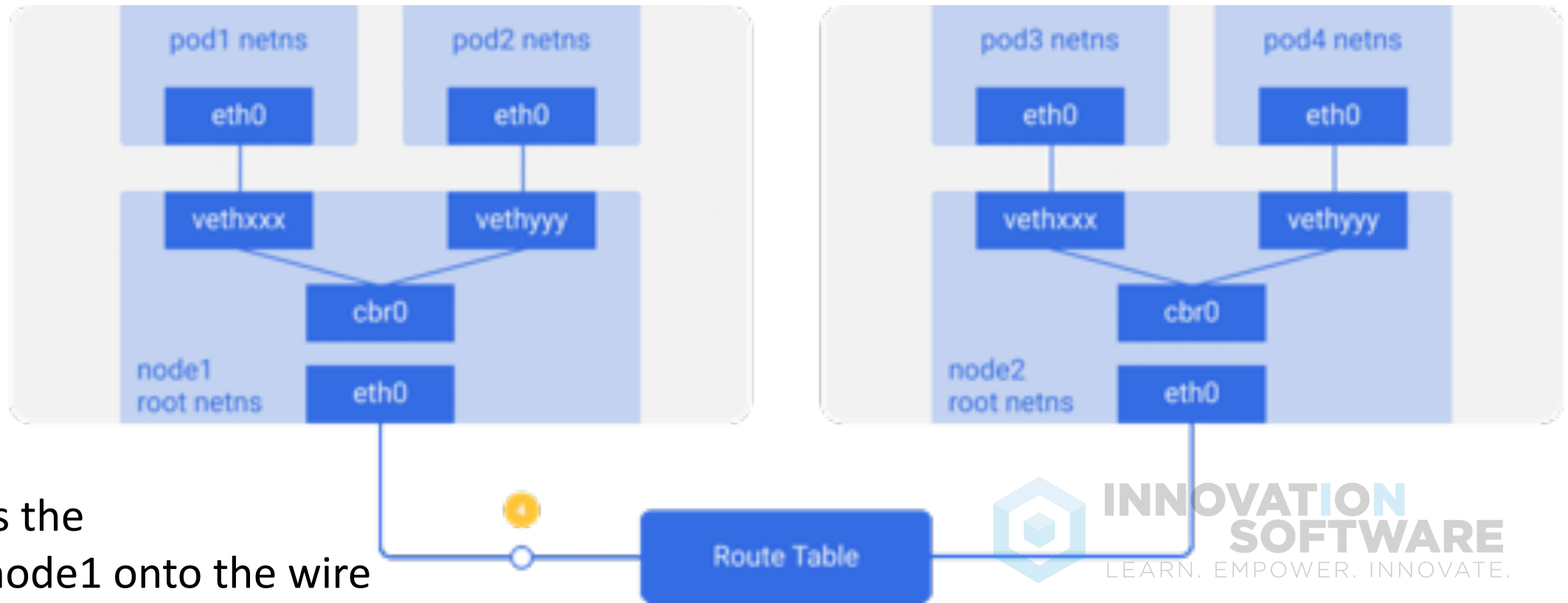
**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Inter-node Communication



3. It comes out of cbr0 to the main network interface eth0 since nobody on this node has the IP address for POD4.

Inter-node Communication

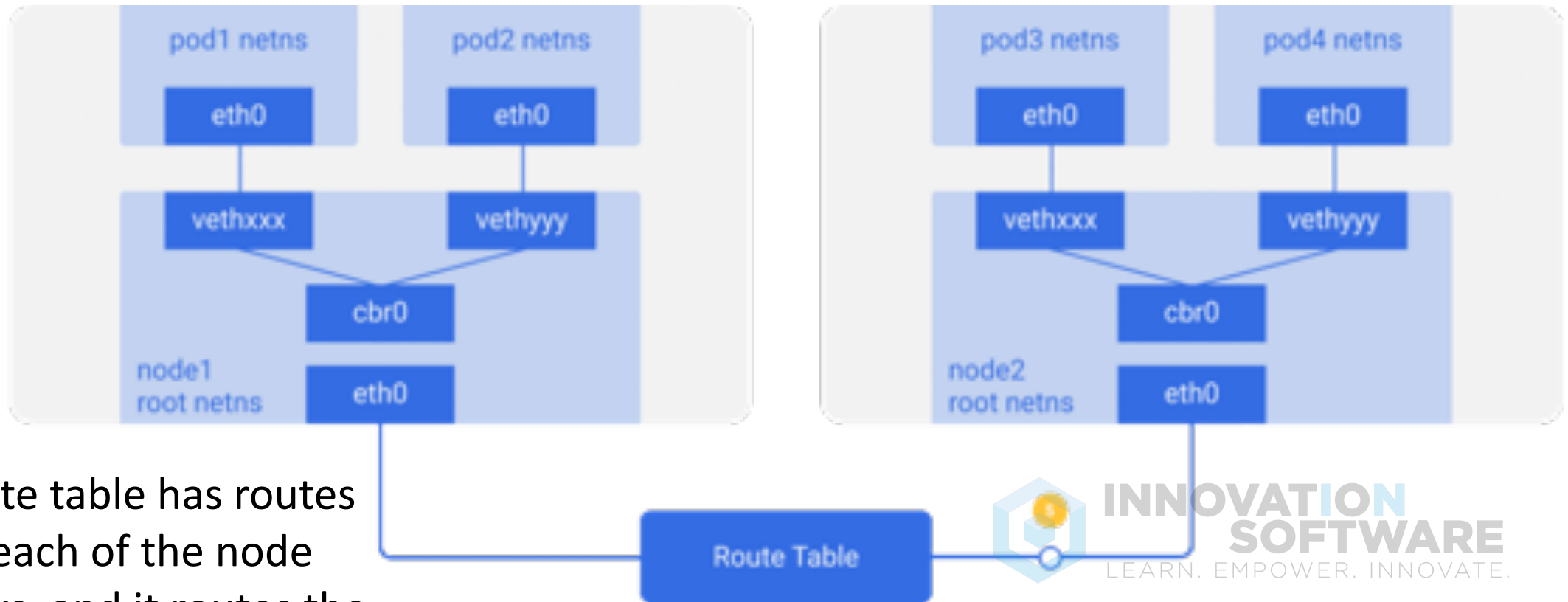


4. It leaves the machine node1 onto the wire with src=POD1 and dst=POD4.



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Inter-node Communication



5. The route table has routes setup for each of the node CIDR blocks, and it routes the packet to the node whose CIDR block contains the POD4 IP.

Inter-node Communication



6. Packet arrives at node2 at eth0. POD4 isn't the IP of eth0, the packet is forwarded to cbr0, nodes are configured with IP forwarding enabled.



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Inter-node Communication



6. The node's routing table is looked up for any routes matching the POD4 IP. It finds **cbr0** as the destination for this node's CIDR block.

Inter-node Communication



7. The bridge takes the packet, makes an ARP request and finds out that the IP belongs to vethyyy.



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Inter-node Communication



8. The packet crosses the pipe-pair and reaches POD4

Kubernetes Overlay Networking



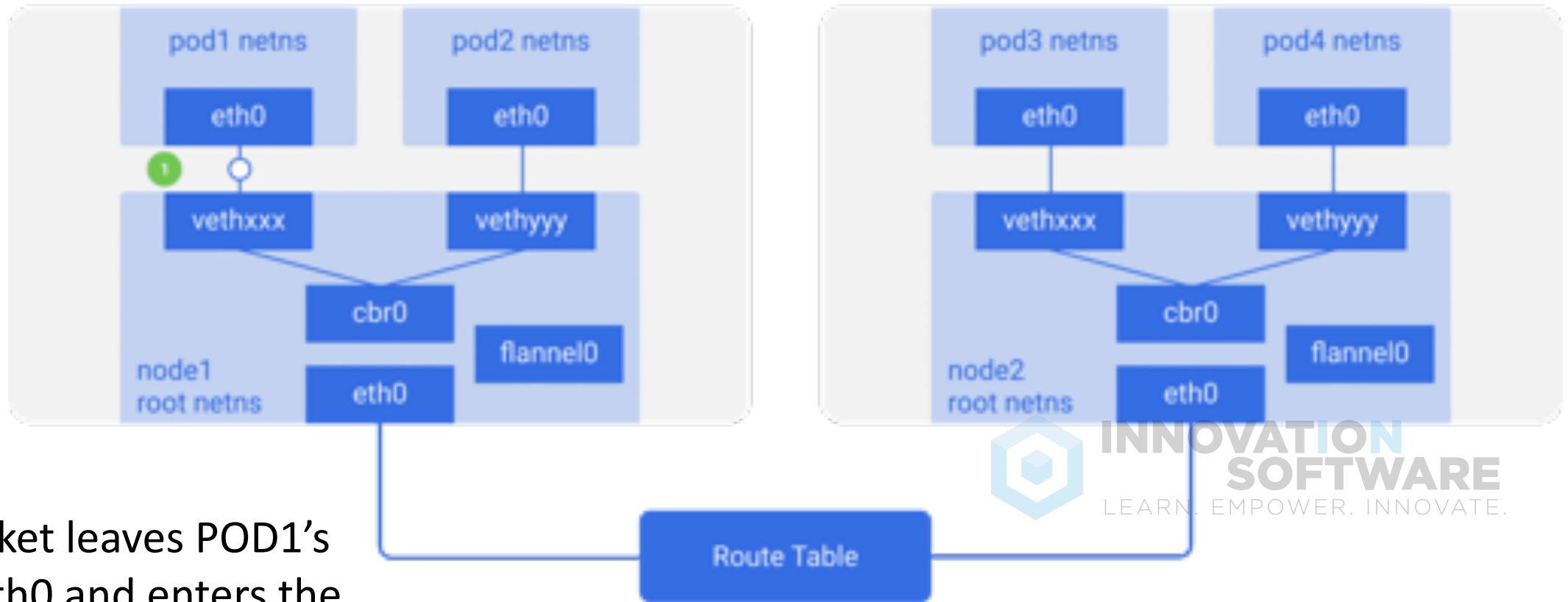
INNOVATION
SOFTWARE
LEARN. EMPOWER. INNOVATE.



Overlay Networking

- Not required but very common
- Provide virtual IP space
- Avoid route limits (Public Cloud)
- Encapsulating a packet-in-packet which traverses the native network across nodes.
- Can reduce throughput

Flannel Overlay Networking

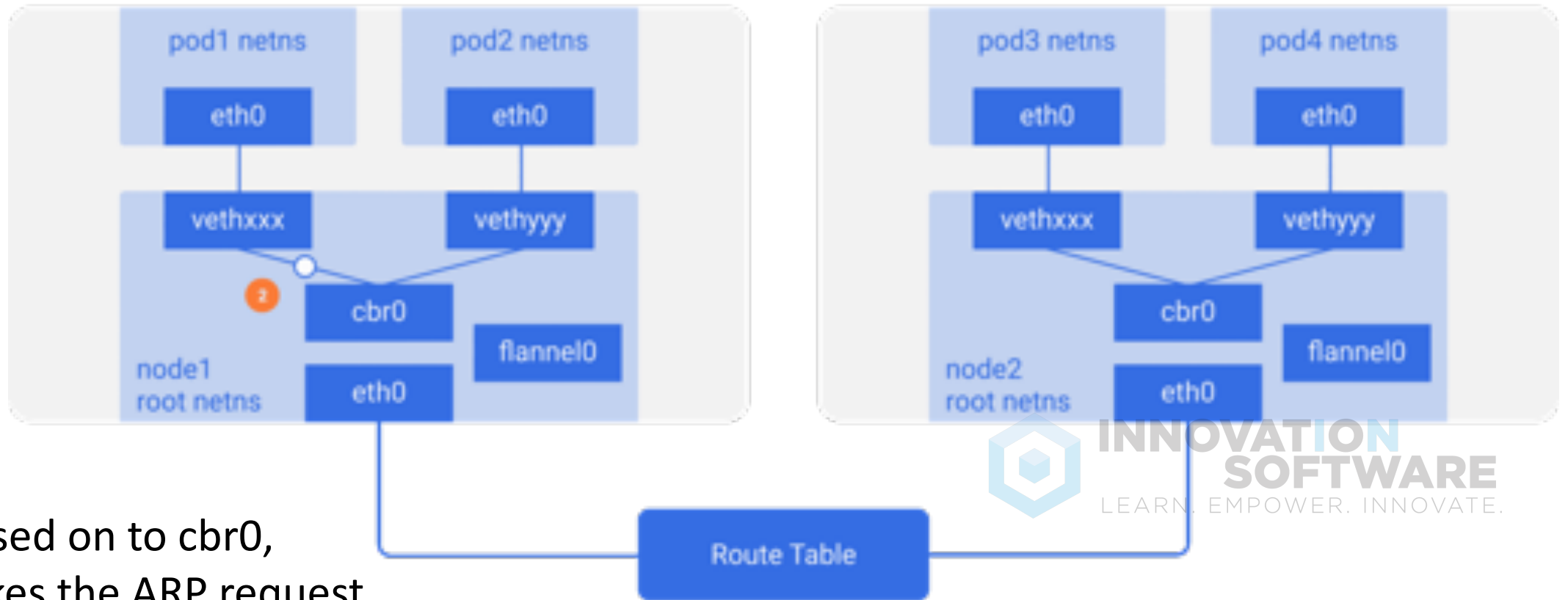


1. The packet leaves POD1's netns at eth0 and enters the root netns at vethxxx.



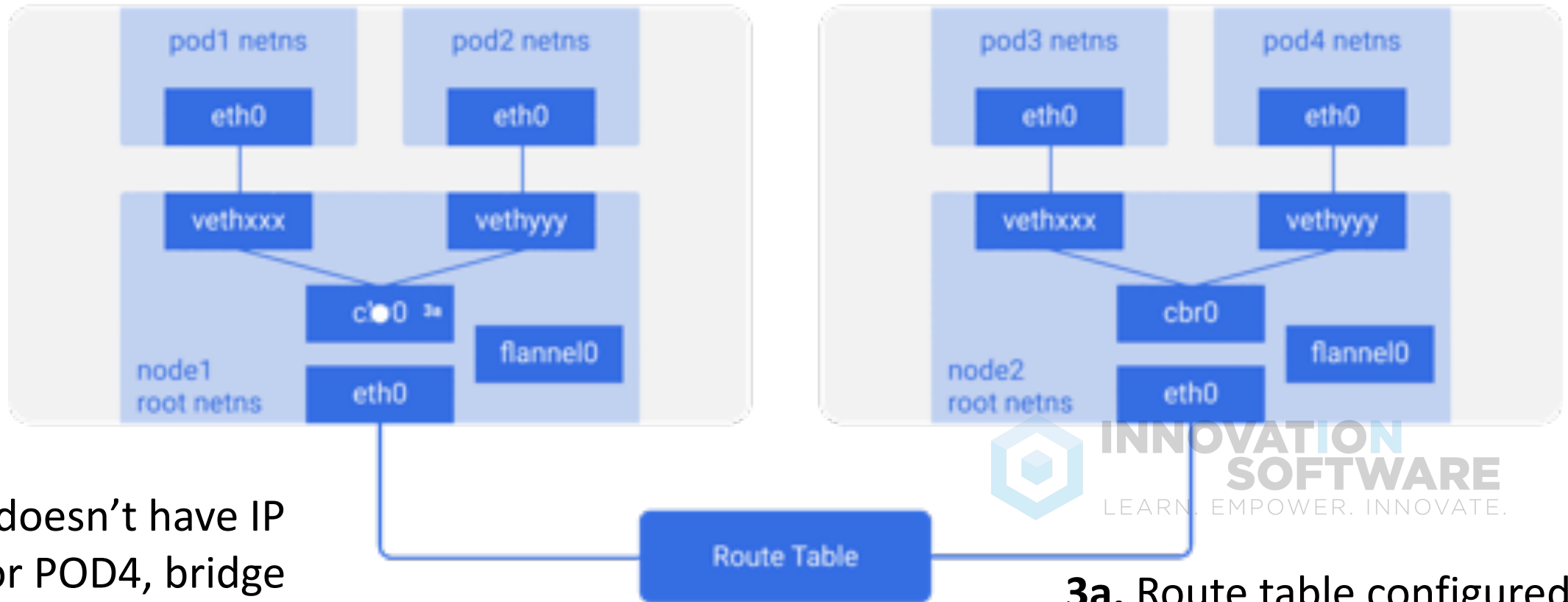
**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Flannel Overlay Networking



2. It's passed on to `cbr0`, which makes the ARP request to find the destination.

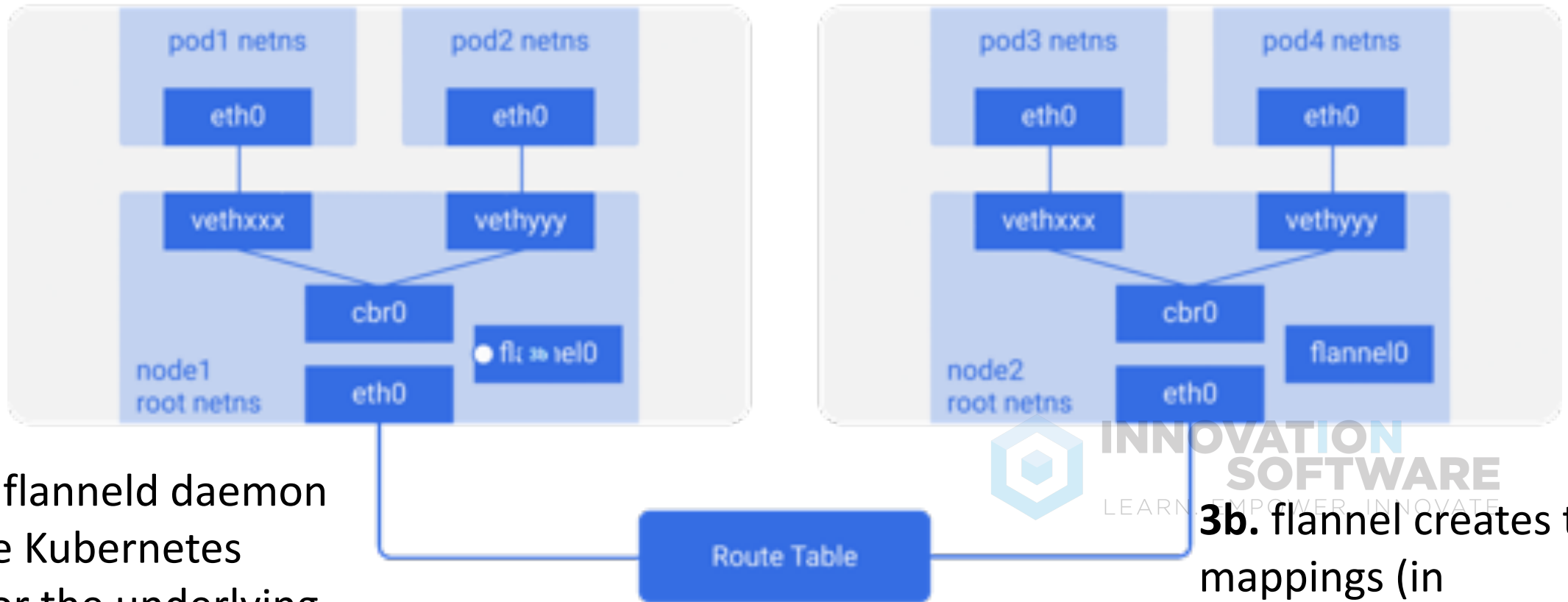
Flannel Overlay Networking



3a. Node doesn't have IP address for POD4, bridge sends to flannel0.

3a. Route table configured with flannel0 as the target for the pod network range .

Flannel Overlay Networking



3b. As the flanneld daemon talks to the Kubernetes apiserver or the underlying etcd, it knows about all the pod IPs, what nodes they're on.

3b. flannel creates the mappings (in userspace) for pods IPs to node IPs.

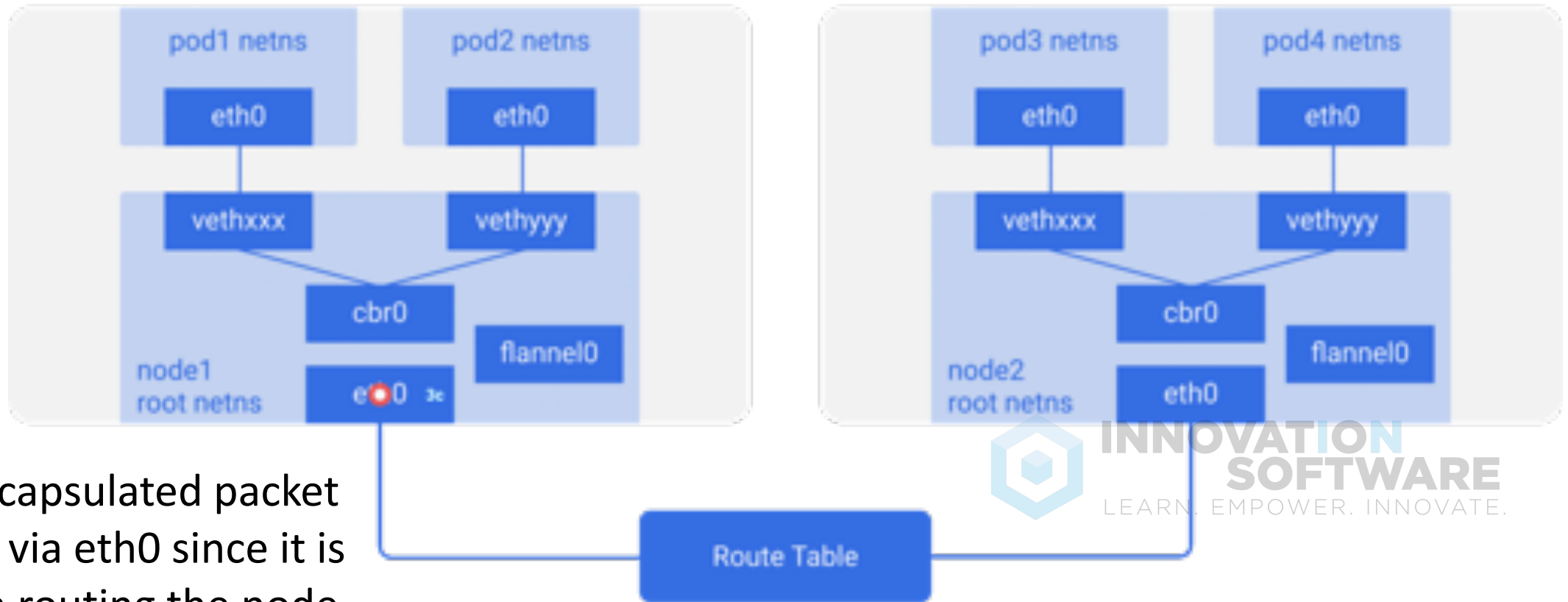
Flannel Overlay Networking



flannel0 takes this packet and wraps it in a UDP packet with extra headers changing the source and destinations IPs to the respective nodes, and sends it to a special vxlan port (generally 8472).

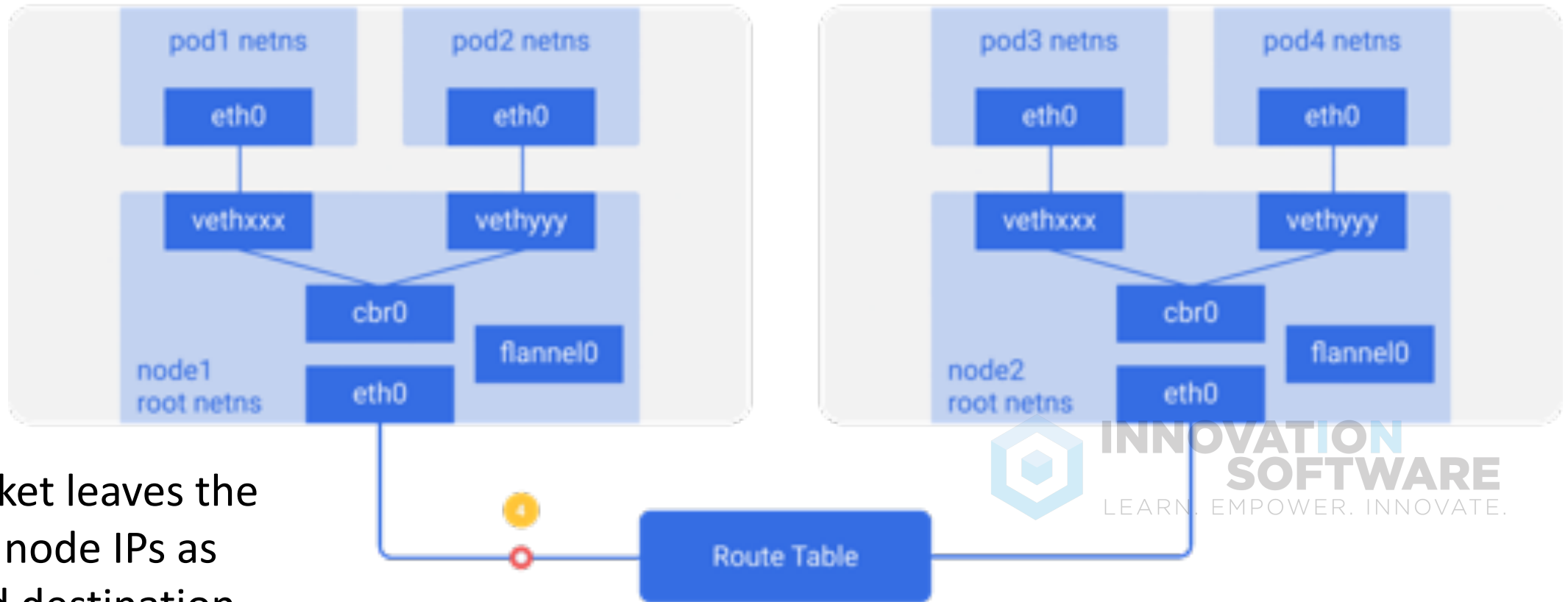


Flannel Overlay Networking



3c. The encapsulated packet is sent out via eth0 since it is involved in routing the node traffic.

Flannel Overlay Networking

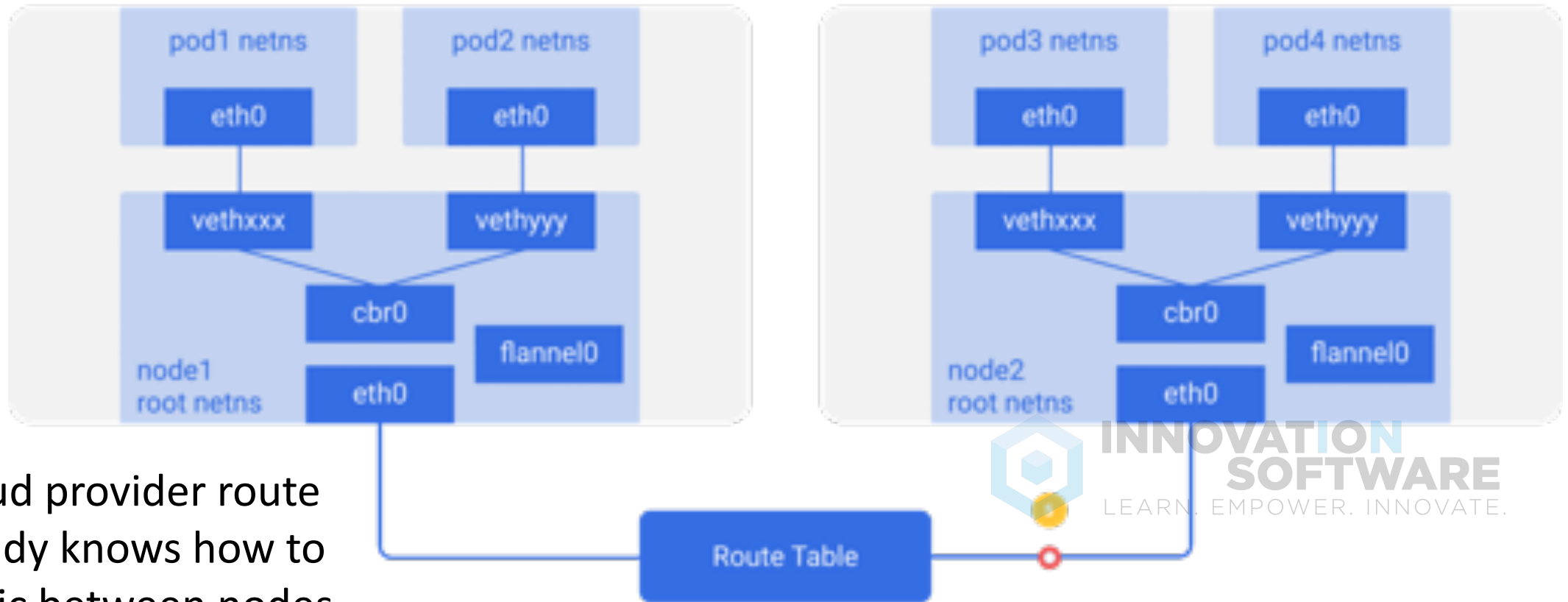


4. The packet leaves the node with node IPs as source and destination.



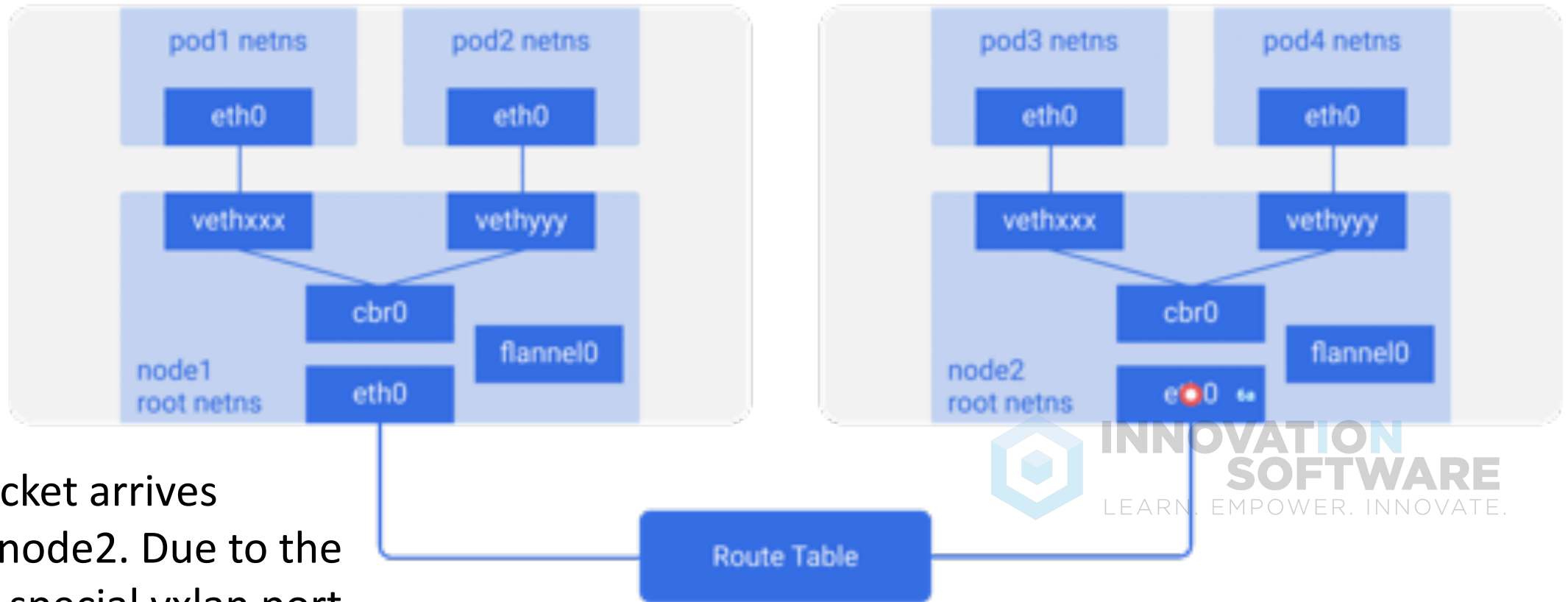
**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Flannel Overlay Networking



5. The cloud provider route table already knows how to route traffic between nodes, so it sends the packet to destination node2.

Flannel Overlay Networking

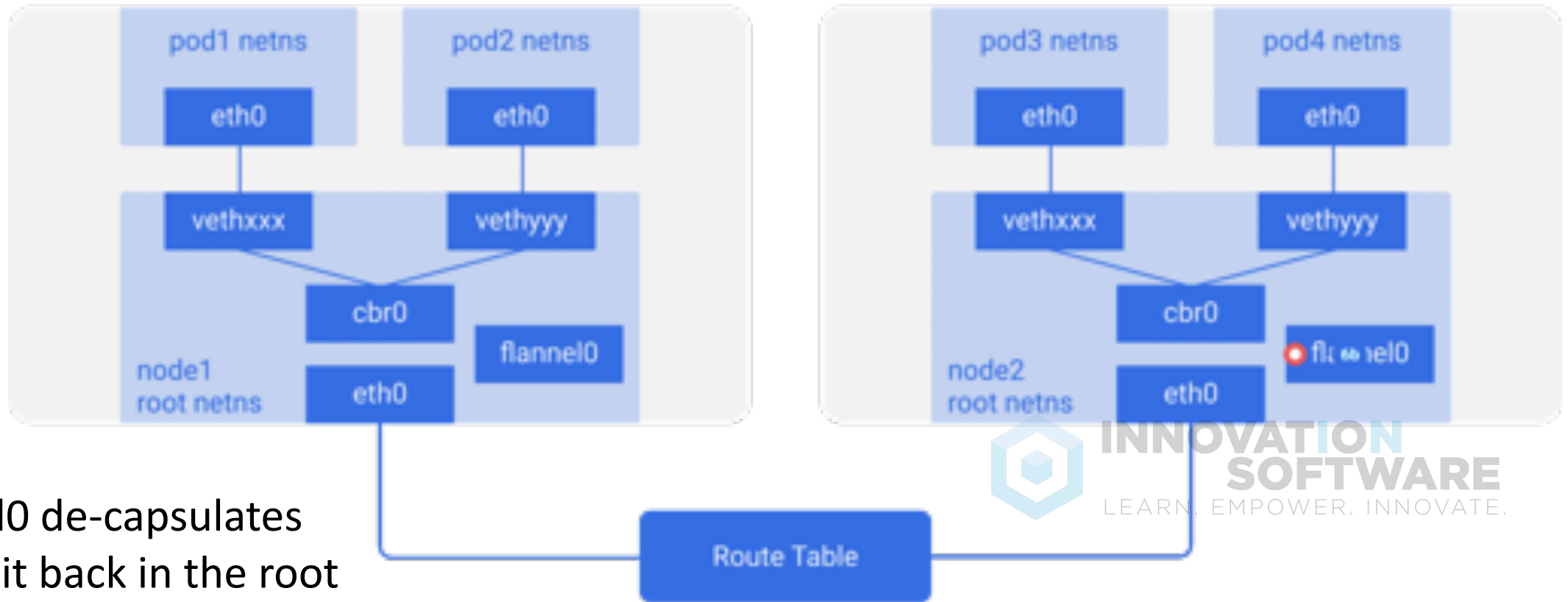


6a. The packet arrives at `eth0` of node2. Due to the port being special vxlan port, kernel sends the packet to `flannel0`.



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Flannel Overlay Networking

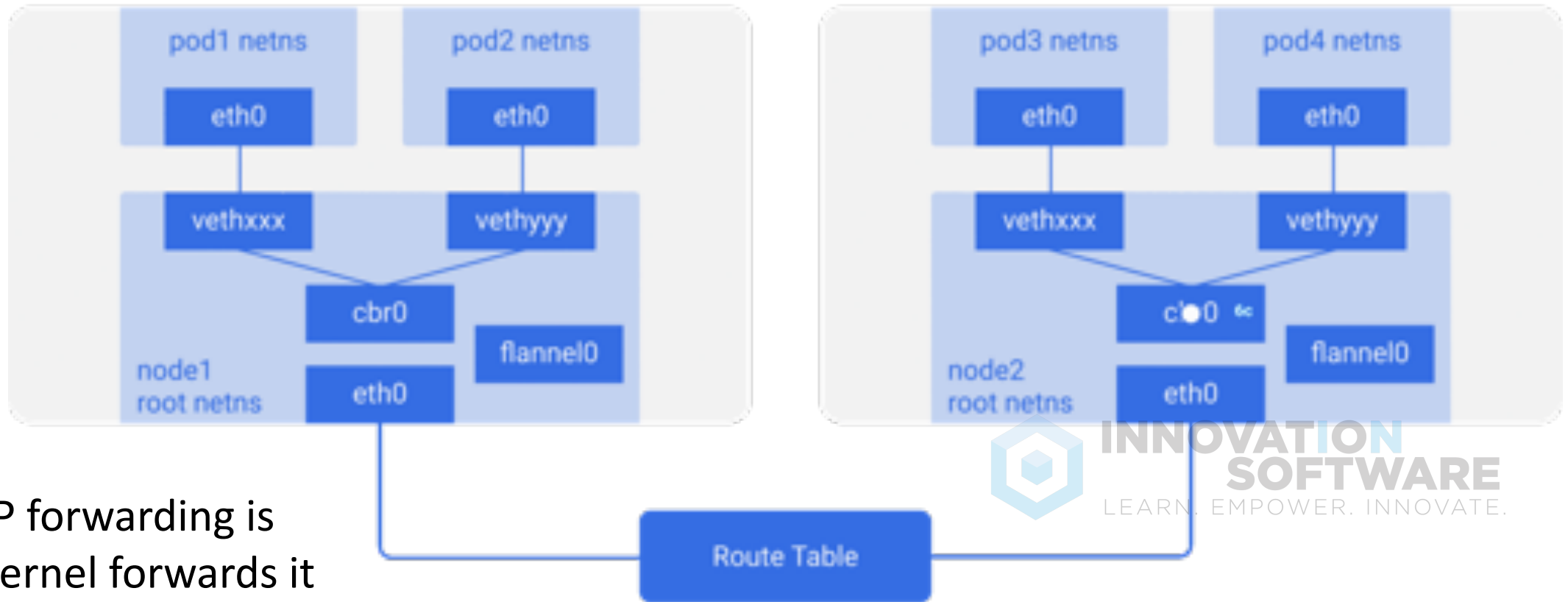


6b. flannel0 de-capsulates and emits it back in the root network namespace.



**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Flannel Overlay Networking

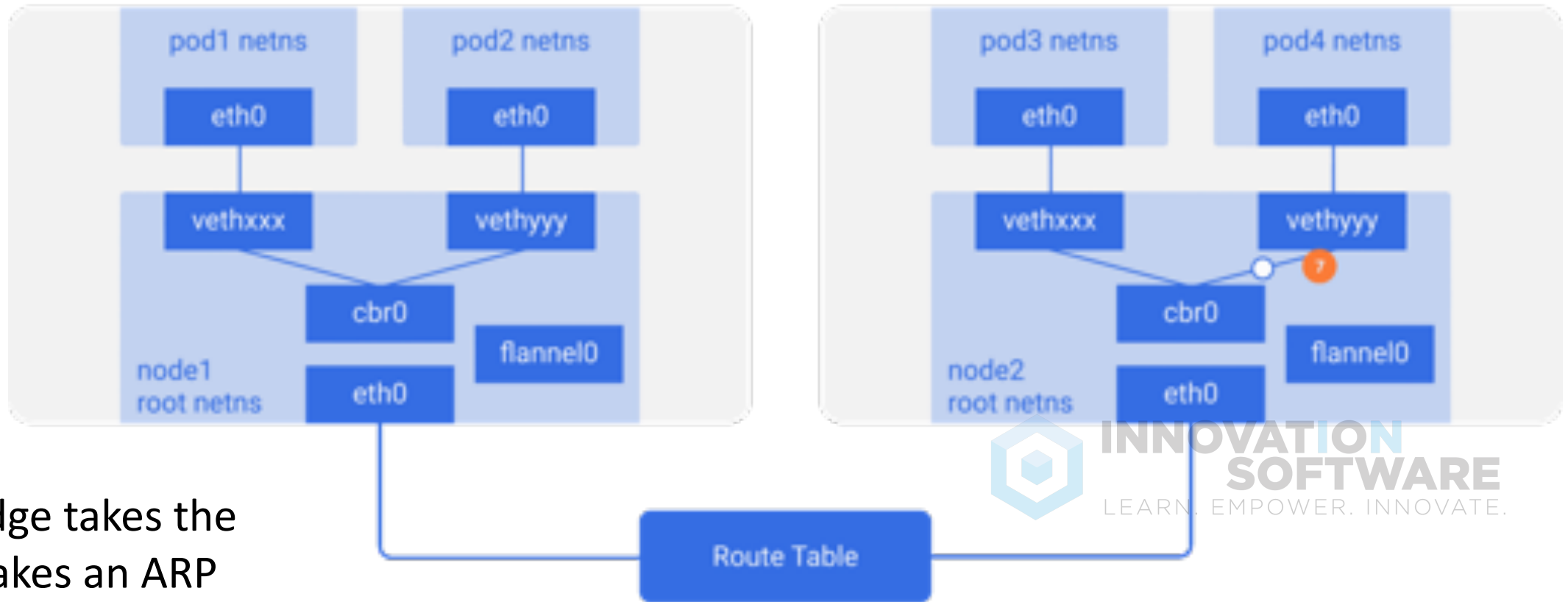


6c. Since IP forwarding is enabled, kernel forwards it to cbr0 as per the route tables.



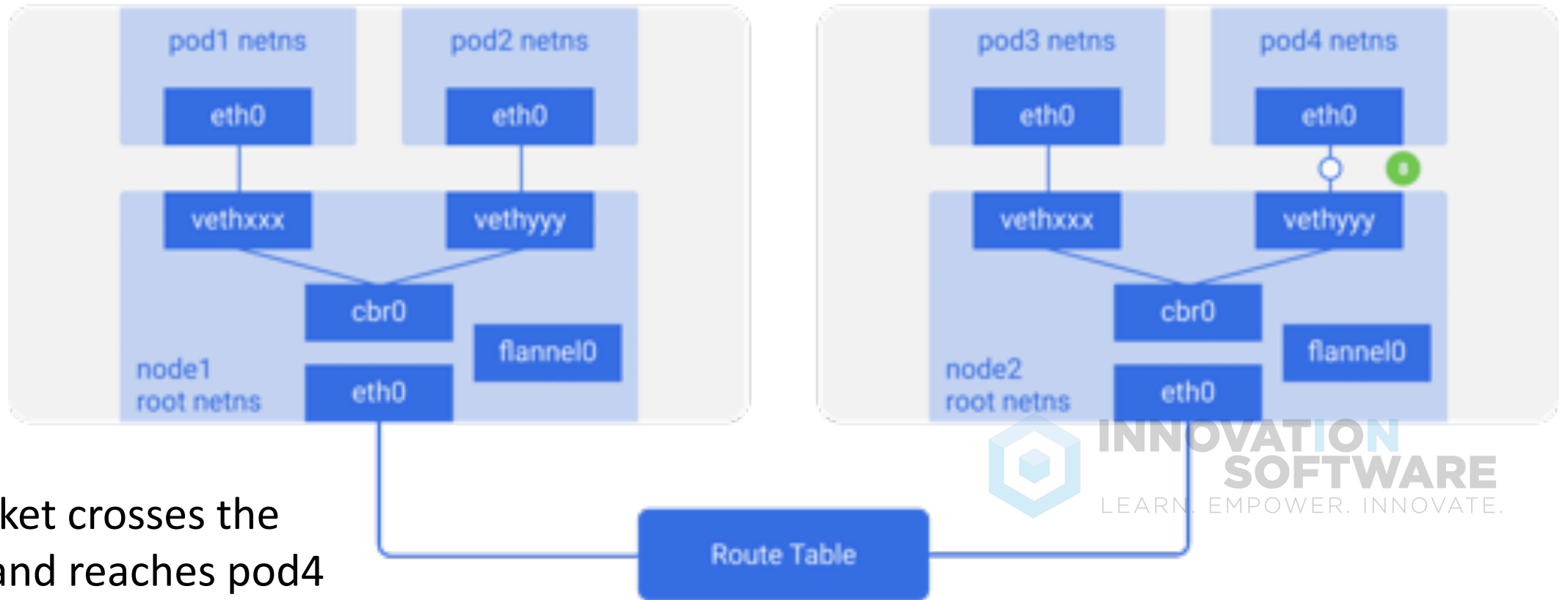
**INNOVATION
SOFTWARE**
LEARN. EMPOWER. INNOVATE.

Flannel Overlay Networking



7. The bridge takes the packet, makes an ARP request and finds out that the IP belongs to `vethyyy`.

Flannel Overlay Networking



8. The packet crosses the pipe-pair and reaches pod4